

Laboratórios de Análise de Discurso Político

Francisco Brandão

Lucas M Pavelski

2026-06-03

Table of contents

Introdução	7
Objetivos do curso	8
Motivação	8
Bibliografia	8
Programa detalhado	9
Aula 1 - 06/03/2026:	9
Aula 2 - 20/03/2026:	9
Aula 3 - 10/04/2026:	9
Aula 4 - 15/05/2026:	10
Aula 5 - 24/04/2026:	10
Aula 6 - 22/05/2026:	10
Aula 7 - 29/05/2026:	10
Aula 8 - 26/06/2026:	10
Avaliação	10
 1 Linguagem R	 11
1.1 Programando scripts em R:	11
1.2 Ambientes para programar em R	12
1.2.1 Rstudio	13
1.2.2 Google Colab	13
1.2.3 Console R	13
1.2.4 Scripts R	13
1.2.5 Notebooks R	14
1.3 Comandos básicos	15
1.3.1 Operadores aritméticos	15
1.3.2 Operadores relacionais e lógicos	15
1.3.3 Variáveis	17
1.3.4 Condicionais	20
1.3.5 Funções	21
1.3.6 Vetores	24
1.3.7 Data frames	27
1.3.8 Bibliotecas e pacotes	29
1.3.9 Strings (cadeias de caracteres)	31
1.4 Exemplo: Análise de Sentimentos com R	32
1.5 Para saber mais	35

1.6	Atividade prática	35
2	Palavras, morfemas e tokens	36
2.1	Obtendo dados textuais	36
2.1.1	Tipos de fontes de dados textuais	37
2.1.2	Exemplo: Carregando Discursos da Câmara	38
2.1.3	Criando o corpus	39
2.2	Níveis de análise textual	43
2.3	A estrutura das palavras: morfemas, lemas e radicais	44
2.3.1	Lematização e Anotação Morfossintática	44
2.3.2	Stemização (Corte do radical)	44
2.3.3	Caracteres e Codificação	45
2.4	Tokenização (Dividindo o texto)	45
2.4.1	A Matriz de Documentos e Termos (DFM)	46
2.5	Stopwords	53
2.6	Palavras no contexto (Keywords-in-context)	59
2.7	Exemplo Prático: Nuvem de Palavras por Partido	60
2.8	Para saber mais	68
2.9	Estudos guiados	68
2.9.1	Criando corpus a partir de uma planilha CSV	68
2.9.2	Quebrando o corpus em palavras (tokens)	70
2.9.3	Pré-processamento de tokens: removendo stopwords e stematização	71
2.9.4	Construindo DFM a partir de tokens:	73
2.9.5	Contando palavras mais utilizadas (em todos os discursos)	75
2.9.6	Contando palavras mais utilizadas por grupos (ex: partido)	76
2.9.7	Núvem de palavras	79
2.9.8	Núvem de palavras comparativa	80
2.10	Atividade prática	82
3	N-gramas, Collocations e TF-IDF	83
3.1	Dicionários	85
3.2	N-gramas	87
3.2.1	Skip-grams: ngramas com saltos	93
3.2.2	Matriz de Co-ocorrência de Características (Feature Co-occurrence Matrix, FCM)	96
3.3	Collocations	98
3.3.1	Exemplo	99
3.4	TF-IDF	101
3.4.1	Exemplo	102
3.5	Diversidade léxica (lexical diversity)	105
3.6	Leiturabilidade (readability)	106
3.7	Palavras-chave (Keyness)	107
3.8	Para saber mais	109

3.9	Estudos guiados	110
3.9.1	Análise de temas com dicionários	110
3.9.2	Gerando n-gramas (sequências de palavras)	111
3.9.3	Skip-grams para capturar relações entre palavras não adjacentes	112
3.9.4	Redes de co-ocorrência de palavras (FCM)	113
3.9.5	Identificando Collocations (expressões típicas)	114
3.9.6	Calculando TF-IDF para destacar termos distintivos	115
3.9.7	Medindo a diversidade léxica (vocabulário)	116
3.9.8	Análise de leiturabilidade (complexidade do texto)	117
3.9.9	Identificando keyness entre grupos	118
3.10	Atividade prática	120
4	Análise Sintática e Modelos de Tópicos	121
4.1	Análise sintática: classes gramaticais	123
4.1.1	Exemplo de análise sintática com Spacy	127
4.2	Análise sintática: relações de dependência	134
4.2.1	Exemplo de relações de dependência com Spacy	137
4.3	Modelos de tópicos: Latent Dirichlet Allocation (LDA)	141
4.3.1	Exemplo de análise de tópicos com LDA	142
4.4	Modelos de tópicos: análise de tópicos guiada	147
4.4.1	Exemplo de análise de tópicos guiada com seededlda	147
4.5	Para saber mais	149
4.6	Estudos guiados	149
4.6.1	Inicialização do Spacy para Português	149
4.6.2	Análise de classes gramaticais (POS Tagging)	150
4.6.3	Extraindo Relações de Dependência (Atores e Ações)	151
4.6.4	Modelagem de Tópicos (LDA)	154
4.6.5	Modelagem de Tópicos Guiada (Seeded LDA)	156
4.7	Atividade prática	158
5	Análise de tópicos estruturada e modelos de escala	159
5.1	Análise de tópicos estruturada	159
5.2	BERTopic	161
5.3	Escalonamento de texto	162
5.3.1	Wordscore	163
5.3.2	Wordfish	163
5.3.3	Análise de correspondência	163
5.4	Similaridade textual	164
6	Estudos guiados	165
6.1	STM	165
6.2	BERTopic	169
6.3	Escalonamento não-supervisionado (Wordfish)	169

6.4	Escalonamento supervisionado (Wordscores)	171
6.5	Análise de Correspondência (CA)	173
6.6	Similaridade Textual	176
6.7	Atividade prática	180
Glossário		182
	Introdução	182
	Aula 1	182
Referências		183
Appendices		185
A	Coletando Dados do legislativo	185
A.1	Coletando dados da API da Câmara dos Deputados	185
A.2	Coletando dados da API do Senado	206

Introdução

Câmara dos Deputados
Centro de Formação, Treinamento e Aperfeiçoamento
Programa de Pós-Graduação



MESTRADO PROFISSIONAL EM PODER LEGISLATIVO PLANO DE CURSO DE DISCIPLINA

DISCIPLINA: ANÁLISE DO DISCURSO POLÍTICO

Período: 1º semestre 2026

Carga horária total: 30 h/a

Código: MEST.7.09.17

PROFESSORES

E-mail

FRANCISCO BRANDÃO, Dr.

francisco.brandao@camara.leg.br

LUCAS MARCONDES PAVELSKI, Dr.

lucas.pavelski@camara.leg.br

CURRÍCULOS RESUMIDOS

FRANCISCO BRANDÃO, Dr.

Grupo de Pesquisa e Extensão (GPE): Ciência de Dados Aplicada ao Estudo do Poder Legislativo: abordagem computacional e métodos de análise

Doutor em Ciência Política pela Universidade da Califórnia, Santa Barbara (2018), nos campos de Política Comparada, Comunicação Política e Tecnologia da Informação e Sociedade. Possui graduação em Jornalismo pela Universidade de São Paulo (2000) e mestrado em Ciência Política pela Universidade de Brasília (2008). Entre 2022 e 2023, foi pesquisador associado da Universidade de Loughborough, no Reino Unido, no projeto PANCOPOP – Comunicação Populista em Tempos de Pandemia. Jornalista na Diretoria de Comunicação da Câmara dos Deputados.

Currículo Lattes: <http://lattes.cnpq.br/9403525530814720>

LUCAS MARCONDES PAVELSKI, Dr.

Doutor e Mestre em Engenharia Elétrica e Informática Industrial pela Universidade Tecnológica Federal do Paraná (2021) com ênfase em sistemas inteligentes. Possui graduação em Ciência da Computação pela Universidade Estadual do Centro-Oeste do Paraná (2012). Atualmente é Analista de TI na Câmara dos Deputados, atuando na Seção de Soluções para a Área Política, com experiência em Ciência de Dados e Engenharia de Software.

Currículo Lattes: <http://lattes.cnpq.br/9287626771427206>

EMENTA DA DISCIPLINA

História, conceitos importantes, questões teóricas e método da Análise de Discurso (AD). Poder como controle, controle do discurso, estratégias discursivas, discurso e poder. Discurso como produção social. Formações discursivas e formações ideológicas. Relação entre estrutura e agência na produção discursiva. Análise argumentativa; discurso e argumentação. As instâncias midiáticas do discurso político. Relações entre discurso político e discurso midiático. Processos hermenêuticos envolvidos na AD. Narrativas, accounts, justificações, estratégias retóricas e uso de significantes vazios na política e nas instâncias jurídicas e midiáticas.

OBJETIVO GERAL DA DISCIPLINA

O aluno deverá compreender e aplicar empiricamente os conceitos básicos relacionados com as principais teorias de análise do discurso e seu instrumental metodológico.

OBJETIVOS ESPECÍFICOS DA DISCIPLINA

Objetivos do curso

Apresentar um ferramental tecnológico para auxiliar na análise de discursos políticos utilizando técnicas de Processamento de Linguagem Natural (PLN).

Motivação

- O volume de dados textuais disponíveis cresceu exponencialmente com a digitalização da informação e o advento das redes sociais.
- Pesquisa quantitativa complementa a pesquisa qualitativa, permitindo análises em larga escala.
- Ferramentas de PLN permitem extrair insights valiosos de grandes volumes de texto.
- Aplicações práticas em diversas áreas, como ciência política, sociologia, marketing e saúde pública.
- Desenvolvimento de habilidades técnicas em análise de dados textuais, programação e modelagem estatística.
- Foco prático do Mestrado Profissional em Poder Legislativo.

Bibliografia

JURAFSKY, D; MARTIN, J. H. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models, 3rd edition, 2025.

Livro-texto com referencial teórico sobre processamento de linguagem natural, linguística computacional e análise de conteúdo (Jurafsky and Martin 2026).

Disponível em: <https://web.stanford.edu/~jurafsky/slp3/>

O'NEILL, L et al. Quantitative discourse analysis at Scale—AI, NLP and the transformer revolution. SoDa Laboratories Working Paper Series, v. 35, n. 11, p. 18-26, 2021.

Artigo apresentando NPL do estado-da-arte aplicada a estudos sociais (O'Neill et al. 2021) .

Disponível em: <https://ideas.repec.org/p/ajr/sodwps/2021-12.html>

SILGE, J.; ROBINSON, D. Text mining with R: a tidy approach. Beijing ; Boston: O'Reilly, 2017.

Livro-texto com referencial teórico e prático sobre mineração de textos com R e biblioteca tidytext (Silge and Robinson 2017).

Disponível em: <https://www.tidytextmining.com/>

WICKHAM, Hadley et al. R for data science. Sebastopol: O'Reilly, 2017.

Livro-texto com referencial prático sobre ciência de dados com R (Wickham 2023).

Disponível em: <https://r4ds.hadley.nz/> (inglês) e <https://pt.r4ds.hadley.nz/> (tradução em português).

CASELI, Helena de Medeiros; NUNES, Maria das Graças Volpe. Processamento de linguagem natural: conceitos, técnicas e aplicações em português. 2024.

Livro-texto com referencial teórico sobre processamento de linguagem natural em português (Caseli and Nunes 2024).

Disponível em: <https://brasileiraspln.com/livro-pln/2a-edicao/>

Programa detalhado

Aula 1 - 06/03/2026:

- Aplicações de PLN em análise de discurso político
- Linguagem R
- Operações básicas com textos em R

Aula 2 - 20/03/2026:

- Obtenção de dados textuais
- Dados do legislativo
- Morfemas, lemas e stemming
- Tokens e stopwords

Aula 3 - 10/04/2026:

- Dicionários
- n-gramas, Collocations
- TF-IDF
- Readability
- Keyness

Aula 4 - 15/05/2026:

- Análise de tópicos
- Análise sintática
- Embeddings
- Similaridade de Documentos

Aula 5 - 24/04/2026:

- Classificação de textos
- Modelos de linguagem

Aula 6 - 22/05/2026:

- Análise de sentimentos
- Reconhecimento de entidades nomeadas

Aula 7 - 29/05/2026:

- Modelos discursivos
- Retrieval Augmented Generation (RAG)

Aula 8 - 26/06/2026:

- Apresentações finais

Avaliação

A avaliação será feita com base na participação nas aulas e na apresentação final de um projeto prático de análise de discurso político utilizando técnicas de processamento de linguagem natural.

1 Linguagem R

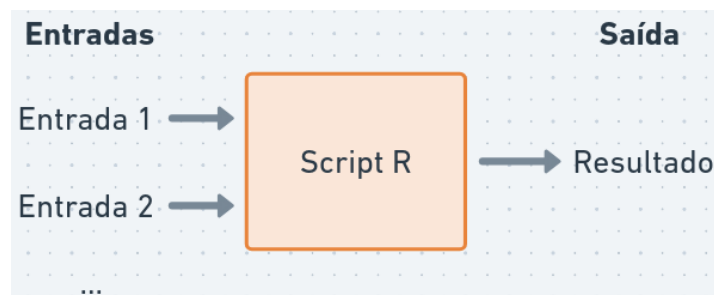
i Nesta aula

- Como utilizar a linguagem R
- Comandos básicos
- Utilizando bibliotecas
- Funções para processamento de texto

Link para Google Colab: <https://colab.research.google.com/drive/1LalxshzmyMmTyfuIaFbp7RYcoP5Qnvuo>

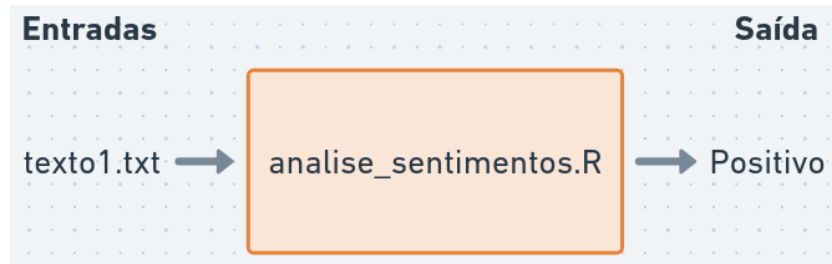
- **R** é uma linguagem de programação e ambiente de software para computação estatística e gráficos
- Criada por Ross Ihaka e Robert Gentleman em 1993
- Muito utilizada em ciência de dados, estatística e análise de texto
- Possui uma grande comunidade de usuários e desenvolvedores, com muitas bibliotecas para análise de textos

1.1 Programando scripts em R:



Script R: sequência de comandos R que podem ser executados para realizar uma tarefa específica (ex: análise de sentimentos)

Em PLN:



`analise_sentimentos.R` é um arquivo contendo instruções para:

- Ler arquivos de texto
- Processar textos (tokenização, remoção de stopwords, stemming)
- Analisar sentimentos (encontrar palavras positivas e negativas)
- Gerar saída (relatório, gráfico, tabela)

O **interpretador R (Rscript)** é um programa que executa os comandos do script e produz a saída desejada. Por exemplo, pelo terminal de comandos:

```
{ $ Rscript analise_sentimentos.R texto1.txt } > "Positivo"
```

1.2 Ambientes para programar em R

O interpretador R está disponível para download em <https://cran-r.c3sl.ufpr.br/>

```
## (código utilizado apenas para exibir páginas no Google Colab)
if (!requireNamespace("htmltools", quietly = TRUE)) {
  install.packages("htmltools")
}
htmltools::tags$iframe(src = "https://cran-r.c3sl.ufpr.br/", width = "100%", height = "315")
```

- No Windows, é necessário também instalar o RTools, disponível em <https://cran.r-project.org/bin/windows/Rtools/>, que inclui ferramentas para auxiliar a instalação de pacotes do R.

Podemos escrever comandos R em diversos ambientes:

1.2.1 Rstudio

- Ambiente de programação integrado (IDE) instalada localmente (Windows, Mac, Linux).
- Possui diversas funcionalidades para facilitar a escrita e execução de código R, como editor de scripts, console interativo, visualização de gráficos e gerenciamento de pacotes.

Disponível em: <https://posit.co/downloads/>.

Uma alternativa é o IDE Positron, disponível em: <https://positron.posit.co/download.html>.

1.2.2 Google Colab

- Ambiente online para programação em Python, mas também suporta R com algumas configurações adicionais (selecione “R” como linguagem de programação para criar um notebook R).
- Permite compartilhar notebooks interativos com código R e resultados.

Disponível em: <https://colab.research.google.com/>.

Outra alternativa é a posit.cloud, disponível em: <https://posit.cloud>.

1.2.3 Console R

- Interface de linha de comando para executar comandos R diretamente no terminal.
- Um comando é executado por vez (digite o comando e aperte Enter), e o resultado é exibido imediatamente.
- É um ambiente menos amigável para desenvolver scripts longos, mas útil para testes rápidos e para o aprendizado da linguagem.

Acessando o console R:

- No terminal, digite R para iniciar o console R e q() para sair.
- No RStudio, o console R está disponível na parte inferior da interface.
- No Google Colab, clique em “Terminal” e digite “R” para iniciar o console R.

1.2.4 Scripts R

- Arquivo de texto com extensão .R que contém uma sequência de comandos R.
- Permite organizar e reutilizar código, facilitando a execução de tarefas complexas.

Como escrever um script R:

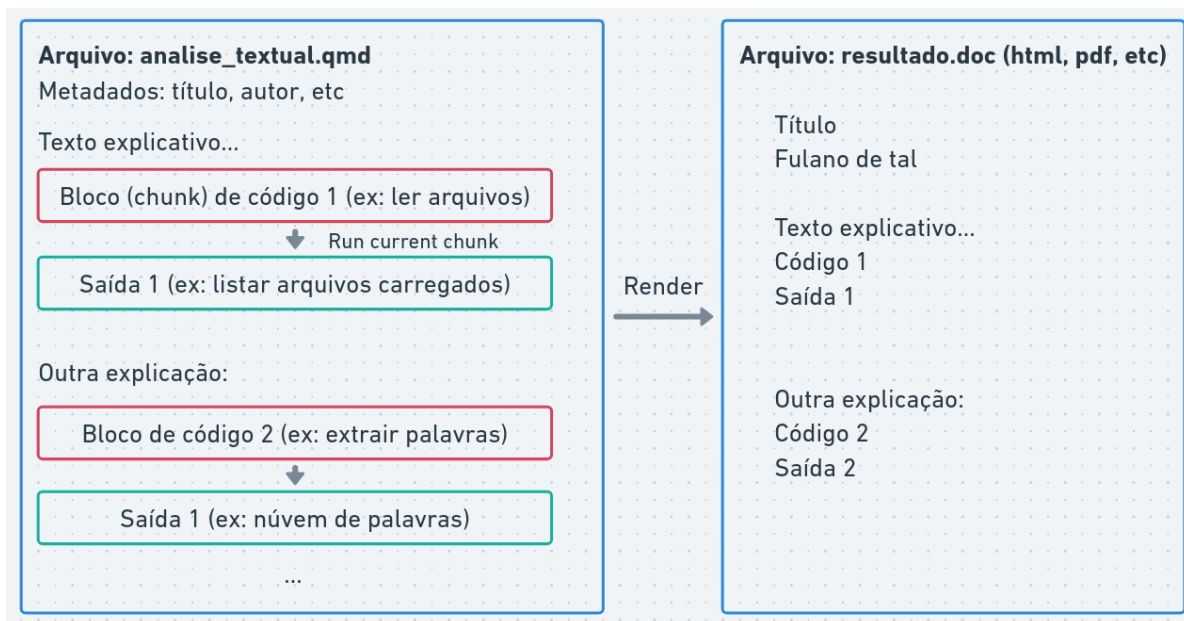
- No RStudio, crie um novo script em: **File > New File > R Script**. Clique em “Source” para executar o script inteiro ou selecione linhas específicas e clique em “Run” para executar apenas essas linhas.

1.2.5 Notebooks R

- Notebooks são documentos que combinam texto explicativo, código R e resultados (gráficos, tabelas) em um formato interativo.
- Permitem criar relatórios dinâmicos e interativos, facilitando a comunicação dos resultados.

Para criar e executar notebooks R, você pode usar o Rstudio ou o Google Colab:

- No Rstudio, crie um novo notebook: **File > New File > Quarto Document**.
- No Google Colab, clique em “File” > “New Notebook” e selecione “R” como linguagem de programação.



Notebooks possuem dois tipos de células:

- células de texto para escrever explicações e comentários
- células de código para escrever e executar códigos em R (o botão “play” executa cada célula individualmente).

1.3 Comandos básicos

- Executar cálculos
- Operadores relacionais e lógicos
- Variáveis
- Listas e vetores
- Funções
- Strings (cadeias de caracteres)

1.3.1 Operadores aritméticos

R pode ser utilizado para cálculos básicos, por exemplo:

```
print(4 + 3)
```

```
[1] 7
```

- `print(*)` é uma função que exibe o resultado de uma expressão na tela.

```
print(5 * 2 + 3)
```

```
[1] 13
```

```
print((6 - 1) / 2)
```

```
[1] 2.5
```

Qualquer código após `#` é considerado um comentário e não é executado:

```
print(8 / 3)      # Isto é um comentário, não será executado
```

```
[1] 2.666667
```

1.3.2 Operadores relacionais e lógicos

Operadores relacionais são utilizados para comparar valores.

Igualdade:

```
print(5 == 5)
```

[1] TRUE

Diferença:

```
print(1 != 2)
```

[1] TRUE

Maior que:

```
print(4 > 3)
```

[1] TRUE

Menor que:

```
print(2 < 5)
```

[1] TRUE

Maior ou igual a:

```
print(3 >= 3)
```

[1] TRUE

- Expressões lógicas: retornam verdadeiro (TRUE) ou falso (FALSE)

Conjunção “e”:

```
print(4 > 3 && 8 > 2)
```

[1] TRUE

- `&&` é o operador lógico “e” (conjunção) que retorna TRUE se ambas as expressões forem verdadeiras, caso contrário retorna FALSE:

- `TRUE && TRUE` retorna `TRUE`
- `TRUE && FALSE` retorna `FALSE`
- `FALSE && TRUE` retorna `FALSE`
- `FALSE && FALSE` retorna `FALSE`

Disjunção “ou”:

```
print(4 > 3 || 2 > 8)
```

[1] `TRUE`

- `||` é o operador lógico “ou” (disjunção), que retorna `TRUE` se pelo menos uma das expressões for verdadeira. Caso contrário, retorna `FALSE`.
- `TRUE || TRUE` retorna `TRUE`
- `TRUE || FALSE` retorna `TRUE`
- `FALSE || TRUE` retorna `TRUE`
- `FALSE || FALSE` retorna `FALSE`

Negação “não”

```
print(!(5 > 3))
```

[1] `FALSE`

- `!` é o operador lógico “não” (negação) que inverte o valor lógico de uma expressão. Se a expressão for `TRUE`, retorna `FALSE`, e se for `FALSE`, retorna `TRUE`.
- `!(TRUE)` retorna `FALSE`
- `!(FALSE)` retorna `TRUE`

```
print((TRUE && FALSE) || TRUE)
```

[1] `TRUE`

1.3.3 Variáveis

Variáveis são utilizadas para guardar valores na memória do computador:

```
a <- 5  
  
print(a)
```

[1] 5

Utilizamos o operador <- para atribuir valores a variáveis.

O valor armazenado pode ser utilizado em outras expressões:

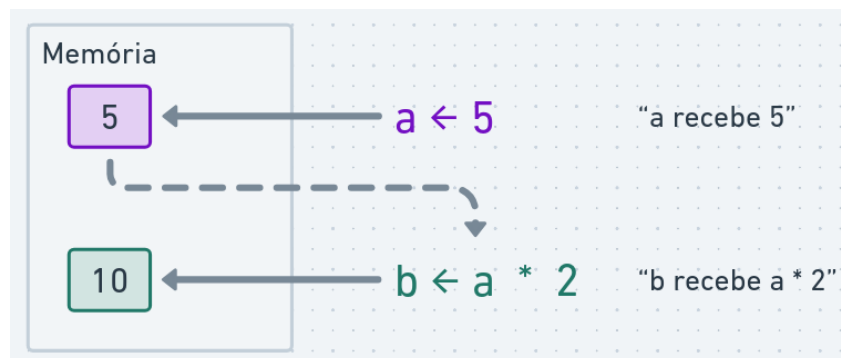
```
print(a * 2 + 3)
```

[1] 13

Ou podemos criar novas variáveis a partir de outras:

```
b <- a * 2  
  
print(b)
```

[1] 10



O valor armazenado na variável pode ser alterado:

```
a <- 10  
  
print(a)
```

```
[1] 10
```

Variáveis podem armazenar diferentes tipos:

```
p <- 1 == 1
q <- 4 < 3

print(p || q)
```

```
[1] TRUE
```

O nome de uma variável deve começar com uma letra e pode conter letras, números (exceto no começo), underscores (_) e pontos:

```
isto_é_uma_variável <- 10
aluno_aprovado_1 <- isto_é_uma_variável == 10

print(isto_é_uma_variável)
```

```
[1] 10
```

```
print(aluno_aprovado_1)
```

```
[1] TRUE
```

Exemplo de cálculo do índice de massa corporal (IMC):

```
peso <- 70 # peso em kg
altura <- 1.75 # altura em metros

imc <- peso / (altura * altura)

imc_saúdavel <- imc >= 18.5 && imc <= 24

print(imc)
```

```
[1] 22.85714
```

```
print(imc_saudavel)
```

```
[1] TRUE
```

1.3.4 Condicionais

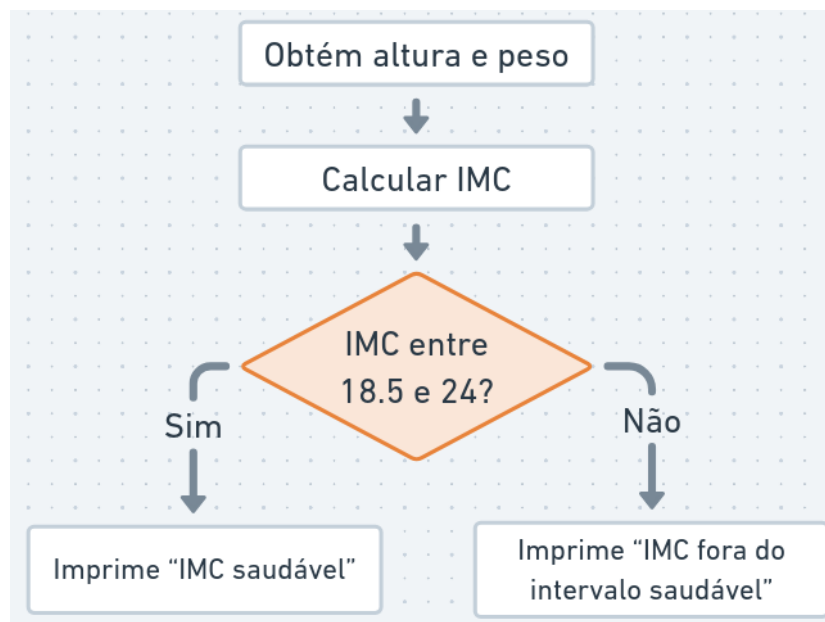
O comando `if` é utilizado para executar um bloco de código apenas se uma condição for verdadeira:

```
x <- 10

if (x > 5) {
  print("x é maior que 5")
}
```

```
[1] "x é maior que 5"
```

O comando `else` pode ser utilizado para executar um bloco de código alternativo caso a condição seja falsa:



```
peso <- 70 # peso em kg
altura <- 1.75 # altura em metros

imc <- peso / (altura * altura)

print(imc)
```

```
[1] 22.85714
```

```
imc_saudavel <- imc >= 18.5 && imc <= 24

if (imc_saudavel) {
  print("IMC saudável")
} else {
  print("IMC fora do intervalo saudável")
}
```

```
[1] "IMC saudável"
```

1.3.5 Funções

Uma **função** é um bloco de código que possui um nome e realiza uma tarefa específica, geralmente para operações comuns que se repetem.

Por exemplo, **print** é a função que usamos para exibir resultados na tela.

R possui várias funções para estatística e análise de dados. Alguns exemplos são:

Valor absoluto:

```
print(abs(-5))
```

```
[1] 5
```

Raiz quadrada:

```
raiz <- sqrt(16)

print(raiz)
```

[1] 4

Logaritmo:

```
log(100, base = 10)
```

[1] 2

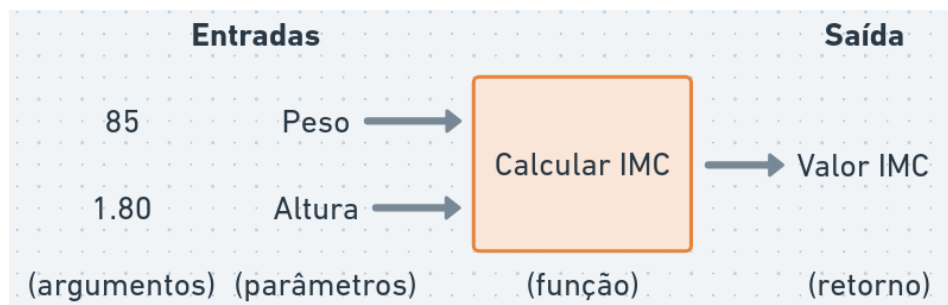
- O número 100 é o primeiro argumento da função.
- O número 10 é o segundo argumento, nomeado como base. Por padrão, a base é o número de Euler ($\exp(1)$), calculando o logaritmo natural.
- Para obter ajuda sobre uma função, execute `help(log)` ou `?log`.

Também é possível definir nossas próprias funções:

```
calcular_imc <- function(peso, altura) {  
  altura_quadrado <- altura * altura  
  return(peso / altura_quadrado)  
}
```

```
imc <- calcular_imc(85, 1.80)  
print(imc)
```

[1] 26.23457



Observe a sintaxe para declarar uma função:

```
nome_da_funcao <- function(argumentos) {  
  # corpo da função  
}
```

A expressão `return(...)` é utilizada para indicar o valor que a função deve retornar. Se `return()` não for especificado, a função retornará o resultado da última expressão avaliada em seu corpo.

Note que qualquer variável criada dentro de uma função é uma variável **local**, ou seja, ela não pode ser acessada fora do escopo da função:

```
print(altura_quadrado)
```

Error:

```
! objeto 'altura_quadrado' não encontrado
```

Após definir uma função, podemos utilizá-la em qualquer parte do código:

```
teste_imc <- function(peso, altura) {  
  imc <- calcular_imc(peso, altura)  
  imc_saudavel <- imc >= 18.5 && imc <= 24  
  return(imc_saudavel)  
}  
  
print(calcular_imc(80, 1.75) < calcular_imc(70, 1.65))
```

```
[1] FALSE
```

Em certos casos, utilizamos o resultado de uma função como argumento de outra função:

```
converter_para_celsius <- function(fahrenheit) {  
  celsius <- (fahrenheit - 32) * 5 / 9  
  return(celsius)  
}  
  
esta_congelando <- function(celsius) {  
  congelando <- celsius <= 0  
  return(congelando)  
}  
  
temperatura_inverno <- 25 # Graus Fahrenheit (aprox. -3.8 °C)  
  
resultado <- esta_congelando(converter_para_celsius(temperatura_inverno))  
  
print(resultado)
```

```
[1] TRUE
```

Em R, podemos utilizar o *operador pipe* (`|>`) para encadear funções de forma mais legível:

```
temperatura_inverno <- 25 # Graus Fahrenheit (aprox. -3.8 °C)

resultado <- temperatura_inverno |>
  converter_para_celsius() |>
  esta_congelando()

print(resultado)
```

```
[1] TRUE
```

- No exemplo, a expressão `temperatura_inverno` é passada como primeiro argumento para a função `converter_para_celsius()`, e o resultado dessa função é então passado como primeiro argumento para a função `esta_congelando()`.

O operador pipe (`|>`) torna o código mais legível, permitindo que as funções sejam encadeadas de maneira clara e fluida.

1.3.6 Vetores

Vetores são estruturas que armazenam uma sequência de elementos do mesmo tipo:

```
vetor_num <- c(1, 2, 3, 4, 5, 6, 7)

print(vetor_num)
```

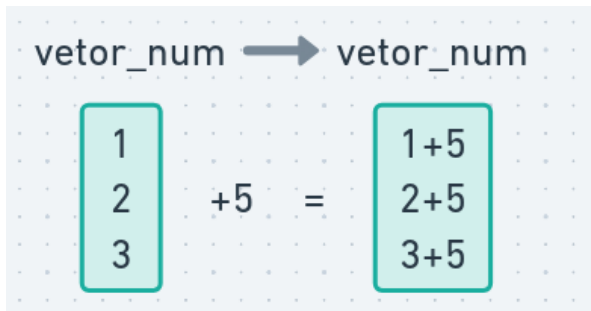
```
[1] 1 2 3 4 5 6 7
```

Operações aritméticas podem ser aplicadas a todos os elementos de um vetor simultaneamente:

```
vetor_num <- c(1, 2, 3)
vetor_num <- vetor_num + 5

print(vetor_num)
```

```
[1] 6 7 8
```

É possível criar sequências numéricas de forma simples:

```
vetor_seq <- 1:10  
print(vetor_seq)
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

Algumas funções operam sobre vetores:

```
print(sum(1:100))
```

```
[1] 5050
```

```
print(mean(1:100))
```

```
[1] 50.5
```

```
print(sd(1:100))
```

```
[1] 29.01149
```

Para acessar um elemento específico do vetor, utilizamos colchetes []:

```
vetor_pares <- c(2, 4, 6, 8, 10)  
print(vetor_pares[3])
```

```
[1] 6
```

Para alterar elementos do vetor também utilizamos colchetes:

```
vetor_seq <- 1:10
vetor_seq[3] <- 999

print(vetor_seq)
```

```
[1] 1 2 999 4 5 6 7 8 9 10
```

Podemos acessar múltiplos elementos do vetor:

```
print(vetor_pares[c(1, 4)])
```

```
[1] 2 8
```

Também podemos filtrar elementos do vetor com expressões lógicas:

```
pares_maior_5 <- vetor_pares > 5
elementos_maior_5 <- vetor_pares[pares_maior_5]

print(vetor_pares)
```

```
[1] 2 4 6 8 10
```

```
print(pares_maior_5)
```

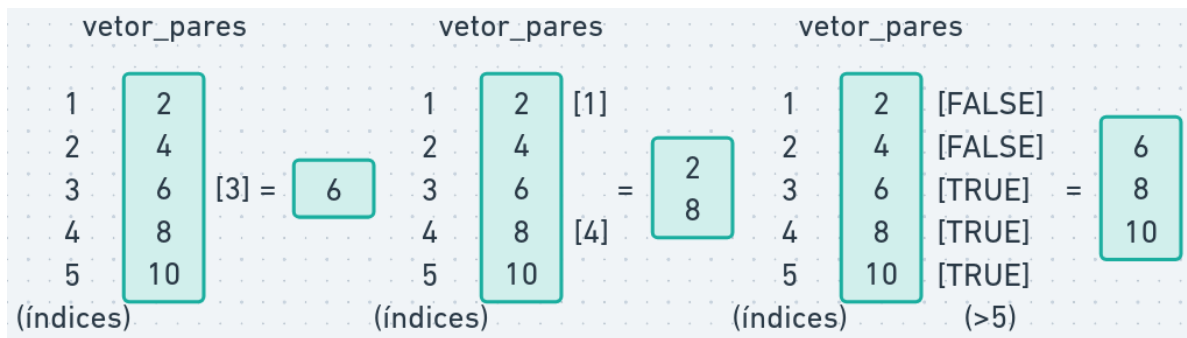
```
[1] FALSE FALSE TRUE TRUE TRUE
```

```
print(elementos_maior_5)
```

```
[1] 6 8 10
```

```
print(vetor_pares[vetor_pares > 5])
```

```
[1] 6 8 10
```



O operador `%in%` é utilizado para verificar se certos elementos estão presentes em um vetor:

```
vetor_a <- c(2, 3, 10)

print(3 %in% vetor_a)
```

```
[1] TRUE
```

```
vetor_b <- c(3, 4, 5, 6, 7)

print(vetor_a %in% vetor_b)
```

```
[1] FALSE TRUE FALSE
```

1.3.7 Data frames

R possui vários tipos de dados, os principais são:

- Numéricos: números inteiros ou decimais (ex: 5, 3.14)
- Lógicos: TRUE ou FALSE
- Strings: texto entre aspas (ex: "Olá, mundo!")

Data frames são estruturas de dados tabulares, onde cada coluna é um vetor e cada linha representa uma observação.

```
df <- data.frame(
  nome = c("João", "Maria", "Pedro"),
  idade = c(30, 25, 28),
  turma = c("A", "B", "A")
)

print(df)
```

	nome	idade	turma
1	João	30	A
2	Maria	25	B
3	Pedro	28	A

Data frames são amplamente utilizados para análise de dados, permitindo manipular e analisar conjuntos de dados tabulares.

Essencialmente, um data frame é uma lista de vetores de mesmo comprimento, em que cada vetor representa uma coluna. Cada coluna pode conter um tipo diferente de dado (numérico, lógico, texto etc.).

Acessando colunas:

```
df <- data.frame(
  nome = c("João", "Maria", "Pedro"),
  idade = c(30, 25, 28),
  turma = c("A", "B", "A")
)

print(df$idade)
```

```
[1] 30 25 28
```

```
print(df[, "idade"])
```

```
[1] 30 25 28
```

Acessando linhas:

```
print(df[1, ])
```

	nome	idade	turma
1	João	30	A

Acessando linha e coluna:

```
print(df[1, "idade"])
```

```
[1] 30
```

Para selecionar alunos com mais de 25 anos:

```
alunos_mais_velhos <- df[df$idade > 25, ]
print(alunos_mais_velhos)
```

```
      nome idade turma
1  João     30      A
3 Pedro     28      A
```

1.3.8 Bibliotecas e pacotes

Bibliotecas (ou pacotes) são conjuntos de funções, dados e documentação que estendem as funcionalidades do R. Geralmente são escritas por outros usuários e compartilhadas com a comunidade (ex: `tidyverse`).

Para instalar uma biblioteca, utilizamos o comando `install.packages()`.

```
if (!"tidyverse" %in% installed.packages()) {
  install.packages("tidyverse")
}
```

Pesquisadores do mundo todo contribuem com bibliotecas no repositório oficial do R, <https://cran-r.c3sl.ufpr.br/>.

Para acessar as funções de uma biblioteca, é necessário carregá-la com `library(...)`:

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.0      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2     4.0.2      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr       1.2.1
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

O tidyverse é uma coleção de pacotes para ciência de dados que compartilham uma filosofia de design.

Ele inclui pacotes para manipulação de dados (`dplyr`, `tidyr`), visualização (`ggplot2`) e muito mais.

Suas funções para manipulação de data frames (aqui chamados de tibbles), como `filter`, `select` e `mutate`, facilitam a análise de dados tabulares:

```
alunos <- tibble(
  nome = c("João", "Maria", "Pedro"),
  idade = c(30, 25, 28),
  nota_final = c(8.5, 9.0, 6.5),
  turma = c("A", "B", "A")
)

alunos_filtrados <- alunos |>
  filter(idade > 25) |> # filtrar os alunos com mais de 25 anos
  mutate(aprovado = nota_final >= 7) |> # nova coluna "aprovado" com base na nota final
  select(nome, idade, aprovado) # selecionar apenas as colunas nome e idade

print(alunos_filtrados)
```

```
# A tibble: 2 x 3
  nome  idade aprovado
<chr> <dbl> <lgl>
1 João    30 TRUE
2 Pedro   28 FALSE
```

Calculando a média de idade dos alunos aprovados:

```
media_idade_aprovados <- alunos |>
  filter(nota_final >= 7) |>
  summarise(media_idade = mean(idade))

print(media_idade_aprovados)
```

```
# A tibble: 1 x 1
  media_idade
<dbl>
1      27.5
```

Em resumo, vimos exemplos de funções:

- `filter()`: filtra linhas com base em condições lógicas
- `mutate()`: cria novas colunas ou modifica colunas existentes
- `select()`: seleciona colunas específicas
- `summarise()`: calcula estatísticas resumidas, como média, soma, etc

Existem várias outras funções para manipulação de data frames incluídas no **tidyverse**. Um resumo visual dessas funções está disponível em <https://nyu-cdsc.github.io/learningr/assets/data-transformation.pdf>.

1.3.9 Strings (cadeias de caracteres)

String é o tipo de dado utilizado para representar textos. Em R, strings são definidas com aspas simples ou duplas:

```
texto <- "Olá, mundo!"  
  
print(texto)
```

```
[1] "Olá, mundo!"
```

A biblioteca **stringr** (parte do **tidyverse**) possui diversas funções para a manipulação de strings (todas começam com o prefixo **str_**):

```
a <- "olá"  
b <- 'mundo'  
a_b_concatenados <- str_c(a, b, sep = " ")  
  
print(a_b_concatenados)
```

```
[1] "olá mundo"
```

- Concatenar significa unir duas ou mais strings. A função `str_c` do pacote **stringr** é usada para esta finalidade, e o argumento **sep** especifica o caractere separador a ser inserido entre as strings (neste caso, um espaço).

```
num_palavras <- str_length("quantos caracteres tem esta frase?")  
  
print(num_palavras)
```

[1] 34

- A função `str_length` retorna o número de caracteres em uma string, incluindo espaços e pontuação.

```
contem_texto <- str_detect("este texto contém palavras", "texto")  
  
print(contem_texto)
```

[1] TRUE

- A função `str_detect` verifica se uma string contém um padrão de texto. Ela retorna TRUE se o padrão for encontrado e FALSE caso contrário.

As funções do pacote `stringr` são vetorizadas, o que significa que elas podem ser aplicadas diretamente a um vetor de textos, retornando um resultado para cada elemento.

```
discursos <- c(  
  "este texto contém palavras",  
  "este outro texto não tem a palavra-chave",  
  "a palavra-chave está presente aqui"  
)  
contem_texto <- str_detect(discursos, "palavra-chave")  
  
print(contem_texto)
```

[1] FALSE TRUE TRUE

```
print(discursos[contem_texto])
```

```
[1] "este outro texto não tem a palavra-chave"  
[2] "a palavra-chave está presente aqui"
```

1.4 Exemplo: Análise de Sentimentos com R

```
library(tidyverse)
```

Criando um vetor de textos:


```

textos <- c(
  "Eu amo este produto, é incrível!",
  "Este serviço é péssimo, estou muito insatisfeito.",
  "O filme foi ótimo, adorei a história e os personagens.",
  "A comida estava horrível, não recomendo este restaurante.",
  "O atendimento foi excelente, muito atenciosos e rápidos.",
  "Não volto mais a este lugar."
)

print(textos)

```

```

[1] "Eu amo este produto, é incrível!"
[2] "Este serviço é péssimo, estou muito insatisfeito."
[3] "O filme foi ótimo, adorei a história e os personagens."
[4] "A comida estava horrível, não recomendo este restaurante."
[5] "O atendimento foi excelente, muito atenciosos e rápidos."
[6] "Não volto mais a este lugar."

```

Criando um data frame com os textos:

```

df_textos <- tibble(texto = textos)

print(df_textos)

```

```

# A tibble: 6 x 1
  texto
  <chr>
1 Eu amo este produto, é incrível!
2 Este serviço é péssimo, estou muito insatisfeito.
3 O filme foi ótimo, adorei a história e os personagens.
4 A comida estava horrível, não recomendo este restaurante.
5 O atendimento foi excelente, muito atenciosos e rápidos.
6 Não volto mais a este lugar.

```

Criando um vetor de palavras positivas e negativas:

```

palavras_positivas <- c("amo", "incrível", "ótimo",
                        "adorei", "excelente", "atenciosos", "rápidos")
palavras_negativas <- c("péssimo", "insatisfeito", "horrível",
                        "não recomendo", "ruim", "lento")

```

```
print(palavras_positivas)
```

```
[1] "amo"          "incrível"    "ótimo"       "adorei"      "excelente"
[6] "atenciosos"  "rápidos"
```

```
print(palavras_negativas)
```

```
[1] "péssimo"      "insatisfeito" "horrível"     "não recomendo"
[5] "ruim"         "lento"
```

Criando uma função para analisar o sentimento de um texto:

```
analisar_sentimento <- function(palavras) {
  palavras_positivas_encontradas <- sum(palavras %in% palavras_positivas)
  palavras_negativas_encontradas <- sum(palavras %in% palavras_negativas)
  if (palavras_positivas_encontradas > palavras_negativas_encontradas) {
    return("Positivo")
  } else if (palavras_negativas_encontradas > palavras_positivas_encontradas) {
    return("Negativo")
  } else {
    return("Neutro")
  }
}

print(analisar_sentimento(c("este", "produto", "é", "incrível")))
```

```
[1] "Positivo"
```

Aplicando a função de análise de sentimento a cada texto:

```
df_analise <- df_textos |>
  mutate(texto_min = str_to_lower(texto)) |> # converter para minúsculas
  mutate(palavras = str_extract_all(texto_min, boundary("word")))|> # dividir o texto em palavras
  mutate(sentimento = map_chr(palavras, analisar_sentimento)) |> # map_chr aplica a função
  select(texto, sentimento) # selecionar apenas as colunas texto e sentimento

print(df_analise)
```

```
# A tibble: 6 x 2
  texto                                sentimento
  <chr>                                <chr>
1 Eu amo este produto, é incrível!      Positivo
2 Este serviço é péssimo, estou muito insatisfeito. Negativo
3 O filme foi ótimo, adorei a história e os personagens. Positivo
4 A comida estava horrível, não recomendo este restaurante. Negativo
5 O atendimento foi excelente, muito atenciosos e rápidos. Positivo
6 Não volto mais a este lugar.          Neutro
```

```
df_contagem <- df_analise |>
  group_by(sentimento) |>      # agrupar por sentimento
  summarise(contagem = n())    # contar o número de textos em cada categoria de sentimento

print(df_contagem)
```

```
# A tibble: 3 x 2
  sentimento contagem
  <chr>         <int>
1 Negativo         2
2 Neutro           1
3 Positivo         3
```

1.5 Para saber mais

As três primeiras partes do livro *R for Data Science* (Wickham 2023) são uma ótima introdução à linguagem R e à manipulação de dados com a biblioteca `tidyverse`.

1.6 Atividade prática

1. Crie um vetor de textos contendo pelo menos 5 frases de discursos políticos.
2. Crie um data frame com esses textos.
3. Defina 3 vetores com palavras representando temas políticos (ex: economia, saúde, educação).
4. Crie uma função que analise um texto, identifique a qual tema ele mais se relaciona e retorne o tema predominante.
5. Aplique a função de análise de temas a cada texto e crie uma nova coluna no data frame indicando o tema predominante em cada texto.

2 Palavras, morfemas e tokens

i Nesta aula

- Fontes de dados textuais para política.
- O conceito de **Corpus** e a biblioteca *quanteda*.
- Níveis de análise: de caracteres a discursos.
- Processamento básico: Tokenização, *Stopwords* e *Stemming*.
- Exploração inicial: *Keyword-in-context* (KWIC) e Nuvens de Palavras.

Link para o Google Colab: <https://colab.research.google.com/drive/1bOM77Nea3m4OLQSDMUz4k-yj9Qq9q0Zk>

```
# instalando pacotes utilizados nesta aula
if (!"tidyverse" %in% installed.packages()) {
  install.packages("tidyverse")
}
if (!"quanteda" %in% installed.packages()) {
  install.packages("quanteda")
}
if (!"quanteda.textstats" %in% installed.packages()) {
  install.packages("quanteda.textstats")
}
if (!"quanteda.textplots" %in% installed.packages()) {
  install.packages("quanteda.textplots")
}
if (!"stopwords" %in% installed.packages()) {
  install.packages("stopwords")
}
```

2.1 Obtendo dados textuais

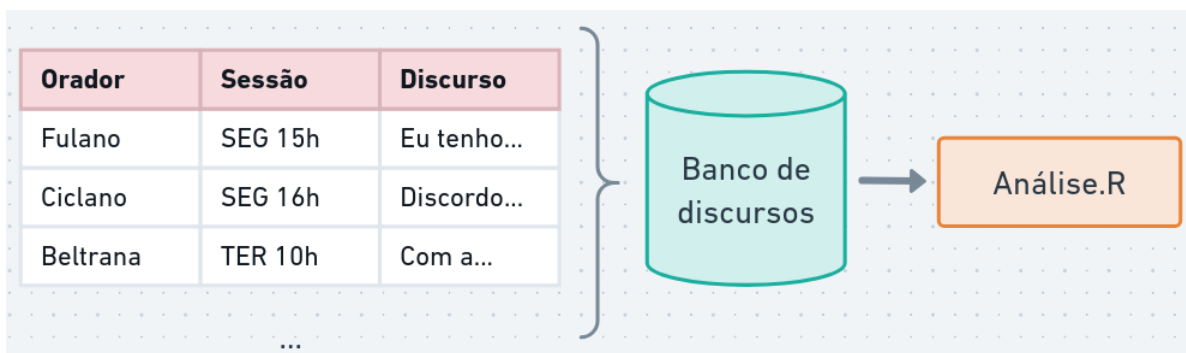
Para realizar uma análise de discurso mediada por computador, o primeiro passo é transformar o material empírico (discursos, leis, posts em redes sociais) em um formato que o computador consiga processar.



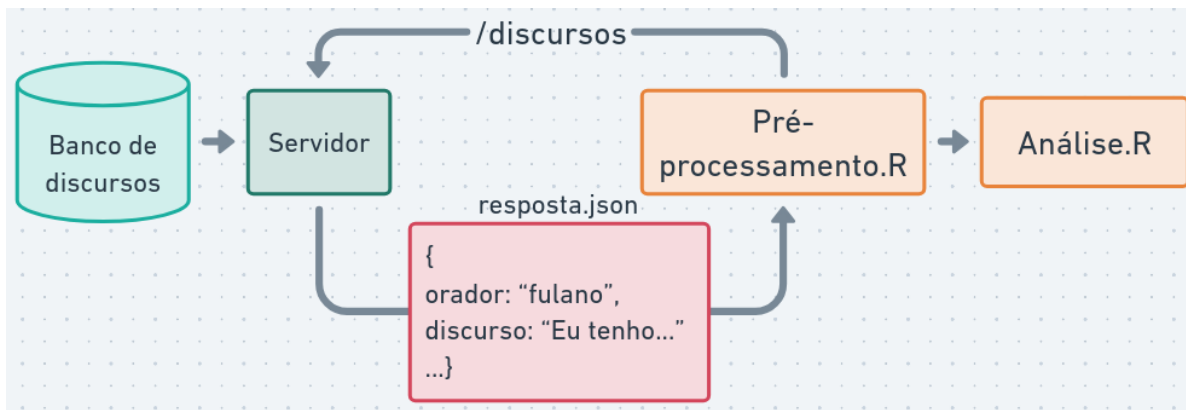
Quais são as fontes de textos interessantes para estudos políticos?

2.1.1 Tipos de fontes de dados textuais

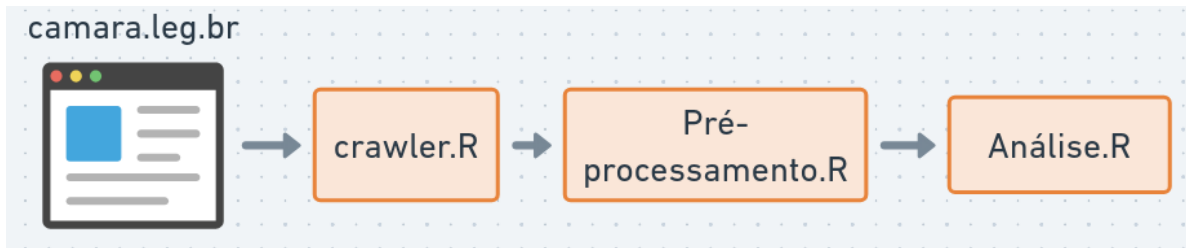
- **Banco de dados e APIs:** Muitas instituições (como a Câmara dos Deputados) oferecem **APIs**, que permitem baixar dados automaticamente com **metadados** valiosos: quem falou, quando, partido, etc.



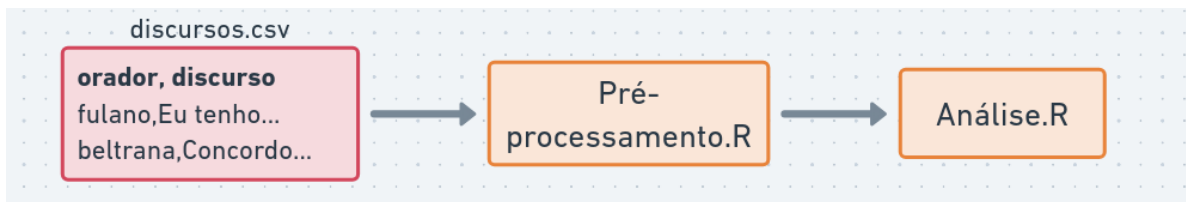
APIs permitem fazer requisições e coletar dados semi-estruturados (JSON ou XML).



Web scraping e OCR: Técnicas para extrair textos diretamente de sites ou PDFs. Útil quando os dados não estão organizados em bancos.



Arquivos estruturados (CSV, Excel): Forma comum de trabalhar com dados já coletados.



2.1.2 Exemplo: Carregando Discursos da Câmara

Nesta aula, para focar na análise textual, utilizaremos um conjunto de dados já extraído da API da Câmara dos Deputados, contendo discursos de parlamentares organizados em um arquivo CSV.

Para manipular esses dados, utilizaremos o pacote **tidyverse**, que é o padrão para organização de dados na linguagem R.

```

library(tidyverse)

# Carregando os dados
df_discursos <- read_csv("discursos.csv")

# Visualizando as primeiras linhas e colunas mais relevantes
df_discursos |> select(nome, partido, uf, transcricao) |> head()
  
```

```

# A tibble: 6 x 4
  nome      partido    uf  transcricao
  <chr>      <chr>    <chr> <chr>
1 Acácio Favacho MDB      AP  "DISCURSO NA ÍNTEGRA ENCAMINHADO PELO SR. ~
2 Acácio Favacho MDB      AP  "O SR. ACÁCIO FAVACHO (Bloco/MDB - AP. Sem~
3 Acácio Favacho MDB      AP  "DISCURSO NA ÍNTEGRA ENCAMINHADO PELO SR. ~
4 Adail Filho  REPUBLICANOS AM  "O SR. ADAIL FILHO (Bloco/REPUBLICANOS - A~
5 Adilson Barroso PL      SP  "O SR. ADILSON BARROSO (Bloco/PL - SP. Sem~
6 Adilson Barroso PL      SP  "O SR. ADILSON BARROSO (Bloco/PL - SP. Sem~
  
```

2.1.3 Criando o corpus

Na análise de textos, chamamos de **corpus** o conjunto de documentos que pretendemos estudar. Um corpus não é apenas uma “pilha de textos”, mas uma coleção organizada que preserva o vínculo entre o texto e seus **metadados** (as variáveis que contextualizam a fala, como o orador ou o partido).

Utilizaremos a biblioteca **quanteda**, uma das mais robustas para análise quantitativa de textos.

```
library(quanteda)

# Criando um identificador único para cada discurso
df_discursos <- df_discursos |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_"))

# Criando o objeto de corpus
corpus_discursos <- corpus(df_discursos,
                           text_field = "transcricao",
                           docid_field = "doc_id")

summary(corpus_discursos, n = 5)
```

Corpus consisting of 1849 documents, showing 5 documents:

	Text	Types	Tokens	Sentences	id	nome	uf
1_Acácio Favacho_MDB	256	428	16	204379	Acácio Favacho	AP	
2_Acácio Favacho_MDB	301	659	42	204379	Acácio Favacho	AP	
3_Acácio Favacho_MDB	318	588	29	204379	Acácio Favacho	AP	
4_Adail Filho_REPUBLICANOS	163	260	13	220714	Adail Filho	AM	
5_Adilson Barroso_PL	243	501	19	221328	Adilson Barroso	SP	
partido					uri		
MDB	https://dadosabertos.camara.leg.br/api/v2/deputados/204379						
MDB	https://dadosabertos.camara.leg.br/api/v2/deputados/204379						
MDB	https://dadosabertos.camara.leg.br/api/v2/deputados/204379						
REPUBLICANOS	https://dadosabertos.camara.leg.br/api/v2/deputados/220714						
PL	https://dadosabertos.camara.leg.br/api/v2/deputados/221328						
	nome_civil	sexo	cpf	data_nascimento			
ACÁCIO DA SILVA FAVACHO NETO	M	74287028287	1983-09-28				
ACÁCIO DA SILVA FAVACHO NETO	M	74287028287	1983-09-28				
ACÁCIO DA SILVA FAVACHO NETO	M	74287028287	1983-09-28				
ADAIL JOSÉ FIGUEIREDO PINHEIRO	M	77267796249	1992-02-16				

ADILSON BARROSO OLIVEIRA		M 05585378805		1964-06-14	
data_falecimento	uf_nascimento	municipio_nascimento	escolaridade		
NA	AP	Macapá	Superior		
NA	AP	Macapá	Superior		
NA	AP	Macapá	Superior		
NA	AM	Manaus	Superior	Incompleto	
NA	MG	Minas Novas	Superior		
situacao	data_situacao	ultimo_partido	hora_inicio	fase_evento	
Exercício	2023-02-01	MDB	2025-04-29 13:55:00	Encerramento	
Exercício	2023-02-01	MDB	2025-04-02 10:32:00	Homenagem	
Exercício	2023-02-01	MDB	2025-04-01 13:55:00	Encerramento	
Exercício	2023-02-01	REPUBLICANOS	2025-04-02 14:44:00	Breves Comunicações	
Exercício	2025-12-14	PL	2025-04-22 17:16:00	Breves Comunicações	
tipo_discurso					
DISCURSO ENCAMINHADO					
HOMENAGEM					
DISCURSO ENCAMINHADO					
PELA ORDEM					
BREVES COMUNICAÇÕES					

Medida provisória, Crédito extraordinário, Comunidade ribeirinha, Crise climática, Amazonas, Seguros, Defesa, Pescador, Biodiversidade, Família, vulnerabilidade, Governo federal, Ministra de Estado, Minas

O Deputado relatou
O Deputado discursou na sessão

O Deputado destacou a importância da aprovação da Medida Provisória 1.362, que concede o seguro-desemprego aos pescadores, beneficiando famílias vulneráveis e protegendo a biodiversidade local.
O Deputado fez críticas ao Supremo Tribunal Federal, afirmando que parte dos Ministros teria

As informações extras (metadados) ficam armazenadas como **docvars** (variáveis de documento).

```
docvars(corpus_discursos) |> head()
```

	id	nome	uf	partido
1	204379	Acácio Favacho	AP	MDB
2	204379	Acácio Favacho	AP	MDB

3	204379	Acácio Favacho	AP	MDB
4	220714	Adail Filho	AM	REPUBLICANOS
5	221328	Adilson Barroso	SP	PL
6	221328	Adilson Barroso	SP	PL

uri

1	https://dadosabertos.camara.leg.br/api/v2/deputados/204379
2	https://dadosabertos.camara.leg.br/api/v2/deputados/204379
3	https://dadosabertos.camara.leg.br/api/v2/deputados/204379
4	https://dadosabertos.camara.leg.br/api/v2/deputados/220714
5	https://dadosabertos.camara.leg.br/api/v2/deputados/221328
6	https://dadosabertos.camara.leg.br/api/v2/deputados/221328

	nome_civil	sexo	cpf	data_nascimento
1	ACÁCIO DA SILVA FAVACHO NETO	M	74287028287	1983-09-28
2	ACÁCIO DA SILVA FAVACHO NETO	M	74287028287	1983-09-28
3	ACÁCIO DA SILVA FAVACHO NETO	M	74287028287	1983-09-28
4	ADAIL JOSÉ FIGUEIREDO PINHEIRO	M	77267796249	1992-02-16
5	ADILSON BARROSO OLIVEIRA	M	05585378805	1964-06-14
6	ADILSON BARROSO OLIVEIRA	M	05585378805	1964-06-14

	data_falecimento	uf_nascimento	municipio_nascimento	escolaridade
1	NA	AP	Macapá	Superior
2	NA	AP	Macapá	Superior
3	NA	AP	Macapá	Superior
4	NA	AM	Manaus	Superior Incompleto
5	NA	MG	Minas Novas	Superior
6	NA	MG	Minas Novas	Superior

	situacao	data_situacao	ultimo_partido	hora_inicio
1	Exercício	2023-02-01	MDB	2025-04-29 13:55:00
2	Exercício	2023-02-01	MDB	2025-04-02 10:32:00
3	Exercício	2023-02-01	MDB	2025-04-01 13:55:00
4	Exercício	2023-02-01	REPUBLICANOS	2025-04-02 14:44:00
5	Exercício	2025-12-14	PL	2025-04-22 17:16:00
6	Exercício	2025-12-14	PL	2025-04-09 16:00:00

	fase_evento	tipo_discurso
1	Encerramento	DISCURSO ENCAMINHADO
2	Homenagem	HOMENAGEM
3	Encerramento	DISCURSO ENCAMINHADO
4	Breves Comunicações	PELA ORDEM
5	Breves Comunicações	BREVES COMUNICAÇÕES
6	Breves Comunicações	BREVES COMUNICAÇÕES

1
2
3

4 Medida provisória,Crédito extraordinário,Comunidade ribeirinha,Crise climática,Amazonas,Se
 defeso,Pescador,Biodiversidade,Família,vulnerabilidade,Governo federal,Ministra de Estado,Min
 5
 6 justificat
 presidente da República,Crescimento econômico,Imposto,Mérito,Voto contrário

1 O Deputado relat
 2 O Deputado discursou na sessão s
 3
 4 O Deputado destacou a importância da aprovação da Medida
 defeso aos pescadores, beneficiando famílias vulneráveis e protegendo a biodiversidade local
 5 O Deputado fez críticas ao Supremo Tribunal Federal, afirmando que parte dos Ministros ter
 6
 Presidente Bolsonaro e a necessidade de estimular o crescimento econômico por meio de menos t

Podemos filtrar o corpus com base nelas, o que é essencial para análises comparativas em
 ciência política:

```
# Exemplo: Criar um sub-corpus apenas com discursos de parlamentares do PT
corpus_pt <- corpus_subset(corpus_discursos, partido == "PT")
print(corpus_pt)
```

Corpus consisting of 373 documents and 21 docvars.

39_Airton Faleiro_PT :

"O SR. AIRTON FALEIRO (Bloco/PT - PA. Sem revisão do orador.)..."

40_Airton Faleiro_PT :

"O SR. AIRTON FALEIRO (Bloco/PT - PA. Sem revisão do orador.)..."

41_Airton Faleiro_PT :

"O SR. AIRTON FALEIRO (Bloco/PT - PA) - Sr. Presidente, em no..."

42_Airton Faleiro_PT :

"O SR. AIRTON FALEIRO (Bloco/PT - PA. Pela ordem. Sem revisão..."

43_Airton Faleiro_PT :

"O SR. AIRTON FALEIRO (Bloco/PT - PA. Pela ordem. Sem revisão..."

57_Alencar Santana_PT :

"O SR. ALENCAR SANTANA (Bloco/PT - SP. Sem revisão do orador...."

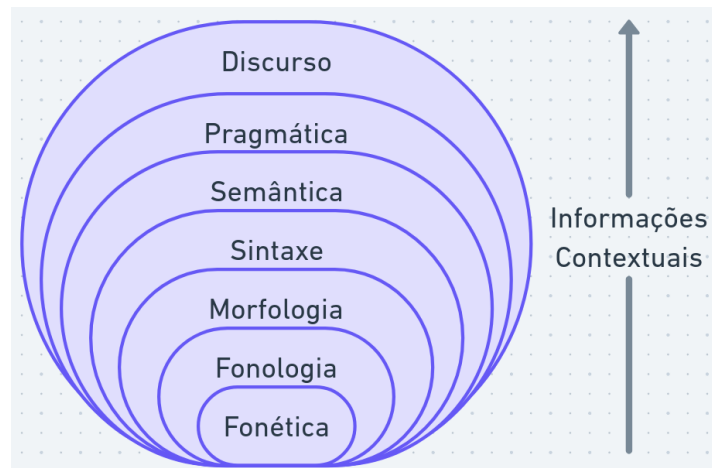
[reached max_ndoc ... 367 more documents]

Outras bibliotecas relacionadas que vamos utilizar são: `quanteda.textstats`, para gerar estatísticas, e `quanteda.textplots`, para visualizações gráficas.

```
library(quanteda.textstats)
library(quanteda.textplots)
```

2.2 Níveis de análise textual

O estudo da língua é dividido em diversas subáreas:



- **fonética e fonologia:** sons e sua organização
- **morfologia:** como morfemas se organizam para formar palavras
- **sintaxe:** como palavras se organizam para formar sintagmas e orações
- **semântica:** significado de palavras e frases
- **pragmática:** organização de orações para fins comunicativos
- **discurso:** estruturas do texto como todo

O processamento de textos pelo computador (PLN) foca principalmente no texto escrito (sintaxe, semântica e discurso). Contudo, a fala e a entonação também são vitais. Dada a natureza multimodal da análise de discurso, as marcas de oralidade carregam forte peso interpretativo.

Observe o exemplo abaixo das notas taquigráficas da Câmara:

<https://www.camara.leg.br/internet/sitaqweb/TextoHTML.asp?etapa=5&nuSessao=64.2025&nuQuarto=4327701&nuOrador=1&nuInsercao=1&dtHorarioQuarto=09:32&sfgFaseSessao=BC&Data=30/04/2025>

Que tipo de informação é transmitida por meio da fala, além do conteúdo semântico? A transcrição em texto consegue capturar sarcasmo, exaltação ou hesitação? Perceba como os taquígrafos utilizam parênteses e marcadores para tentar registrar reações do plenário.

2.3 A estrutura das palavras: morfemas, lemas e radicais

A forma como dividimos o texto depende do objetivo da nossa análise. Separar palavras em subunidades pode ser útil para lidar com: - neologismos, palavras incorretas ou desconhecidas; - complexidade do texto ou dialetos; - gêneros literários alternativos.

Por exemplo, Moffitt (2016) destaca o uso de neologismos em performances populistas. Na morfologia, a unidade mínima de significado é o **morfema** (ex: “casinhas” = casa + inha + s).

2.3.1 Lematização e Anotação Morfossintática

A **lematização** é o processo de reduzir uma palavra à sua forma original de dicionário (seu “lema”). Por exemplo, “gatos” e “gata” são convertidos para “gato”, e “fui” é convertido para o verbo “ir”.

No português, esse processo é complexo e exige dicionários e ferramentas avançadas (como o projeto MorphoBR ou a biblioteca `spacyr`), pois depende do contexto (ex: “canto” pode ser o verbo cantar ou o substantivo).

Isso enriquece a análise de discurso ao agrupar flexões verbais e nominais sob um mesmo guarda-chuva.

2.3.2 Stemização (Corte do radical)

Uma alternativa mais simples (e crua) muito usada computacionalmente é a **stemização** (Stemming), que é o processo de cortar as desinências da palavra, mantendo apenas o seu radical (raiz).

Isso reduz drasticamente o vocabulário do texto, facilitando análises de frequência, mas as palavras geradas podem ser ambíguas ou perder nuances de gênero/número que seriam relevantes para a análise.

Exemplo utilizando a biblioteca `quanteda`:

```
stems_testar <- char_wordstem(c("testar", "testando", "testarei"), language = "portuguese")
print(stems_testar)
```

```
[1] "test" "test" "test"
```

2.3.3 Caracteres e Codificação

A menor unidade de informação em um texto digital é o **caractere** (ou símbolo). O computador usa padrões, como o **UTF-8**, para catalogar caracteres (incluindo emojis e acentuação gráfica, essenciais em português).

```
print(tokenize_character("#feliz! \U0001f600"))
```

```
[[1]]  
[1] "#" "f" "e" "l" "i" "z" "!" " " " "
```

2.4 Tokenização (Dividindo o texto)

O problema de separar o texto em unidades mínimas para análise é conhecido como **tokenização**. Em geral, quebramos os discursos em palavras, mas podemos tokenizar por frases se o objetivo for analisar sentenças inteiras.

```
texto <- 'A deputada disse: "Não aceitaremos cortes". A oposição concordou.'  
  
# Tokenização removendo pontuação  
tokens_palavras <- tokens(texto, remove_punct = TRUE)  
print(tokens_palavras)
```

Tokens consisting of 1 document.

```
text1 :  
[1] "A"          "deputada"    "disse"       "Não"         "aceitaremos"  
[6] "cortes"     "A"           "oposição"    "concordou"
```

```
# Tokenização por sentenças  
tokens_frases <- tokens(texto, what = "sentence")  
print(tokens_frases)
```

Tokens consisting of 1 document.

```
text1 :  
[1] "A deputada disse: \"Não aceitaremos cortes\"."  
[2] "A oposição concordou."
```

2.4.1 A Matriz de Documentos e Termos (DFM)

Para o computador fazer contagens, ele precisa organizar os tokens em uma estrutura chamada **Matriz de Documentos e Termos (Document-Feature Matrix, DFM)**.

Pense no DFM como uma grande planilha: cada **linha** é um discurso e cada **coluna** é uma palavra do vocabulário. A célula contém a quantidade de vezes que aquela palavra apareceu naquele discurso.

```
dfm_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE) |>
  tokens_tolower() |> # converter para minúsculo
  dfm()

print(dfm_discursos)
```

Document-feature matrix of: 1,849 documents, 25,831 features (99.29% sparse) and 21 docvars.

docs	features
	discurso na íntegra encaminhado pelo sr deputado
1_Acácio Favacho_MDB	1 4 1 1 1 1 2
2_Acácio Favacho_MDB	0 2 0 0 0 3 1
3_Acácio Favacho_MDB	1 2 1 1 3 1 1
4_Adail Filho_REPUBLICANOS	0 2 0 0 1 2 0
5_Adilson Barroso_PL	0 3 0 0 0 1 0
6_Adilson Barroso_PL	0 5 0 0 0 1 2

docs	features
	acácio favacho sem
1_Acácio Favacho_MDB	1 1 1
2_Acácio Favacho_MDB	1 1 1
3_Acácio Favacho_MDB	1 1 1
4_Adail Filho_REPUBLICANOS	0 0 1
5_Adilson Barroso_PL	0 0 1
6_Adilson Barroso_PL	0 0 2

[reached max_ndoc ... 1,843 more documents, reached max_nfeat ... 25,821 more features]

Por exemplo, para ver a frequência da palavra “presidente”:

```
print(sum(dfm_discursos[, "presidente"]))
```

```
[1] 4737
```

A partir da DFM, podemos extrair estatísticas sobre os discursos, como `textstat_frequency` para contar palavras.

```
dfm_discursos |>
  textstat_frequency() |>
  head(50)
```

	feature	frequency	rank	docfreq	group
1	de	26004	1	1809	all
2	que	23622	2	1819	all
3	o	21855	3	1840	all
4	a	21616	4	1818	all
5	e	16707	5	1768	all
6	do	13012	6	1827	all
7	para	9192	7	1650	all
8	é	8970	8	1626	all
9	da	8935	9	1651	all
10	não	7469	10	1481	all
11	um	6501	11	1515	all
12	com	6101	12	1431	all
13	em	5588	13	1436	all
14	uma	5228	14	1435	all
15	no	5067	15	1412	all
16	presidente	4737	16	1754	all
17	os	4486	17	1347	all
18	na	3872	18	1294	all
19	sr	3756	19	1687	all
20	eu	3686	20	1196	all
21	por	3682	21	1279	all
22	nós	3566	22	1132	all
23	mais	3294	23	1158	all
24	se	3257	24	1147	all
25	dos	2732	25	1078	all
26	as	2701	26	1034	all
27	como	2689	27	1075	all
28	isso	2685	28	1102	all
29	sem	2665	29	1820	all
30	ao	2633	30	1149	all
31	aqui	2508	31	1043	all
32	está	2410	32	994	all
33	brasil	2404	33	982	all
34	à	2098	34	979	all

35	pela	2082	35	1388	all
36	muito	2030	36	1038	all
37	mas	1974	37	981	all
38	bloco	1863	38	1593	all
39	deputado	1862	39	811	all
40	foi	1845	40	819	all
41	tem	1713	41	862	all
42	revisão	1675	42	1661	all
43	também	1659	43	834	all
44	nosso	1615	44	790	all
45	governo	1606	45	640	all
46	essa	1590	46	807	all
47	das	1588	47	747	all
48	hoje	1584	48	876	all
49	casa	1573	49	770	all
50	porque	1564	50	813	all

Agrupando por partidos:

```
dfm_discursos |>
  textstat_frequency(10, groups = partido)
```

	feature	frequency	rank	docfreq	group
1	de	252	1	17	AVANTE
2	o	169	2	19	AVANTE
3	que	162	3	18	AVANTE
4	a	154	4	17	AVANTE
5	e	148	5	18	AVANTE
6	para	87	6	17	AVANTE
7	do	86	7	19	AVANTE
8	não	66	8	14	AVANTE
9	da	64	9	15	AVANTE
10	com	60	10	17	AVANTE
11	de	112	1	3	CIDADANIA
12	que	107	2	3	CIDADANIA
13	a	85	3	3	CIDADANIA
14	o	71	4	3	CIDADANIA
15	e	68	5	3	CIDADANIA
16	não	37	6	3	CIDADANIA
17	com	33	7	3	CIDADANIA
18	do	30	8	3	CIDADANIA
19	os	28	9	3	CIDADANIA

20	eu	27	10	3	CIDADANIA
21	de	1285	1	103	MDB
22	que	1193	2	104	MDB
23	o	1140	3	105	MDB
24	a	1044	4	103	MDB
25	do	819	5	105	MDB
26	e	717	6	96	MDB
27	é	426	7	83	MDB
28	da	422	8	91	MDB
29	para	418	9	90	MDB
30	não	343	10	73	MDB
31	o	189	1	6	MISSÃO
32	de	161	2	6	MISSÃO
33	que	156	3	6	MISSÃO
34	a	145	4	6	MISSÃO
35	do	132	5	6	MISSÃO
36	e	95	6	6	MISSÃO
37	não	78	7	6	MISSÃO
38	para	76	8	6	MISSÃO
39	é	52	9	6	MISSÃO
40	um	50	10	6	MISSÃO
41	o	1602	1	136	NOVO
42	de	1595	2	134	NOVO
43	que	1567	3	134	NOVO
44	a	1306	4	134	NOVO
45	e	909	5	130	NOVO
46	é	850	6	129	NOVO
47	do	817	7	134	NOVO
48	não	706	8	129	NOVO
49	para	572	9	125	NOVO
50	um	472	10	113	NOVO
51	de	684	1	57	PCdoB
52	que	617	2	56	PCdoB
53	a	615	3	57	PCdoB
54	o	509	4	55	PCdoB
55	e	437	5	55	PCdoB
56	da	361	6	57	PCdoB
57	do	348	7	52	PCdoB
58	para	241	8	50	PCdoB
59	é	193	9	45	PCdoB
60	presidente	156	10	55	PCdoB
61	o	489	1	43	PDT
62	de	443	2	38	PDT

63	que	428	3	40	PDT
64	a	388	4	40	PDT
65	e	289	5	38	PDT
66	do	252	6	42	PDT
67	para	224	7	35	PDT
68	é	218	8	42	PDT
69	da	158	9	34	PDT
70	não	136	10	33	PDT
71	que	5557	1	452	PL
72	de	5445	2	449	PL
73	o	5219	3	454	PL
74	a	4729	4	450	PL
75	e	3240	5	433	PL
76	do	2929	6	451	PL
77	é	2127	7	410	PL
78	não	2034	8	381	PL
79	para	2015	9	397	PL
80	da	1915	10	380	PL
81	de	646	1	33	PODE
82	o	569	2	37	PODE
83	que	561	3	36	PODE
84	a	538	4	34	PODE
85	e	442	5	35	PODE
86	do	337	6	37	PODE
87	para	243	7	34	PODE
88	é	227	8	33	PODE
89	da	220	9	33	PODE
90	não	171	10	27	PODE
91	de	775	1	52	PP
92	que	601	2	53	PP
93	o	587	3	53	PP
94	a	579	4	53	PP
95	e	479	5	52	PP
96	do	417	6	53	PP
97	da	310	7	49	PP
98	para	291	8	49	PP
99	com	203	9	42	PP
100	é	203	9	44	PP
101	de	201	1	6	PRD
102	que	177	2	6	PRD
103	a	116	3	6	PRD
104	e	86	4	6	PRD
105	o	85	5	6	PRD

106	da	58	6	6	PRD
107	do	56	7	6	PRD
108	com	56	7	6	PRD
109	para	52	9	6	PRD
110	é	52	9	6	PRD
111	de	1297	1	80	PSB
112	o	1097	2	81	PSB
113	que	1038	3	78	PSB
114	a	994	4	79	PSB
115	e	966	5	77	PSB
116	do	646	6	81	PSB
117	para	405	7	72	PSB
118	é	404	8	72	PSB
119	da	400	9	77	PSB
120	com	345	10	64	PSB
121	de	1638	1	92	PSD
122	a	1267	2	91	PSD
123	e	1130	3	85	PSD
124	que	1101	4	88	PSD
125	o	1030	5	94	PSD
126	do	710	6	91	PSD
127	da	582	7	87	PSD
128	para	486	8	82	PSD
129	é	384	9	75	PSD
130	com	340	10	68	PSD
131	de	498	1	14	PSDB
132	e	375	2	14	PSDB
133	a	327	3	14	PSDB
134	que	311	4	14	PSDB
135	o	261	5	14	PSDB
136	do	254	6	14	PSDB
137	para	169	7	14	PSDB
138	da	130	8	13	PSDB
139	com	116	9	14	PSDB
140	é	110	10	14	PSDB
141	que	1757	1	128	PSOL
142	de	1743	2	126	PSOL
143	a	1614	3	127	PSOL
144	o	1413	4	128	PSOL
145	e	1140	5	125	PSOL
146	do	909	6	126	PSOL
147	é	741	7	114	PSOL
148	da	701	8	124	PSOL

149	para	640	9	116	PSOL
150	não	613	10	115	PSOL
151	que	5310	1	371	PT
152	de	5270	2	364	PT
153	a	4661	3	370	PT
154	o	4618	4	370	PT
155	e	3597	5	362	PT
156	do	2657	6	372	PT
157	para	2018	7	339	PT
158	da	1910	8	347	PT
159	é	1842	9	335	PT
160	não	1547	10	298	PT
161	de	242	1	16	PV
162	o	198	2	16	PV
163	a	194	3	16	PV
164	que	194	3	16	PV
165	e	153	5	16	PV
166	do	137	6	16	PV
167	é	89	7	15	PV
168	da	78	8	16	PV
169	para	77	9	15	PV
170	com	75	10	13	PV
171	a	23	1	2	REDE
172	de	14	2	2	REDE
173	que	13	3	2	REDE
174	cultura	13	3	2	REDE
175	o	10	5	2	REDE
176	é	9	6	2	REDE
177	e	6	7	2	REDE
178	presidente	5	8	2	REDE
179	sem	4	9	2	REDE
180	do	4	9	2	REDE
181	de	1611	1	91	REPUBLICANOS
182	a	1265	2	90	REPUBLICANOS
183	o	1189	3	91	REPUBLICANOS
184	que	1179	4	90	REPUBLICANOS
185	e	1129	5	92	REPUBLICANOS
186	do	684	6	92	REPUBLICANOS
187	da	476	7	82	REPUBLICANOS
188	para	472	8	80	REPUBLICANOS
189	é	424	9	77	REPUBLICANOS
190	com	390	10	76	REPUBLICANOS
191	de	107	1	9	SOLIDARIEDADE

192	que	73	2	10	SOLIDARIEDADE
193	o	69	3	10	SOLIDARIEDADE
194	e	60	4	10	SOLIDARIEDADE
195	a	59	5	10	SOLIDARIEDADE
196	do	40	6	9	SOLIDARIEDADE
197	para	34	7	10	SOLIDARIEDADE
198	com	32	8	5	SOLIDARIEDADE
199	da	27	9	10	SOLIDARIEDADE
200	as	22	10	5	SOLIDARIEDADE
201	de	1985	1	117	UNIÃO
202	que	1520	2	114	UNIÃO
203	a	1513	3	116	UNIÃO
204	o	1341	4	117	UNIÃO
205	e	1241	5	113	UNIÃO
206	do	748	6	116	UNIÃO
207	para	645	7	108	UNIÃO
208	da	606	8	107	UNIÃO
209	é	530	9	99	UNIÃO
210	um	471	10	104	UNIÃO

2.5 Stopwords

Stopwords são palavras funcionais muito comuns (como “o”, “a”, “de”, “e”, “mas”) que, isoladamente, não ajudam a diferenciar os temas dos discursos. Elas são frequentemente removidas para reduzir o “ruído” nos gráficos e nas contagens.

No R, podemos usar o pacote `stopwords` para obter listas pré-definidas em português:

```
library(stopwords)
stopwords_portugues <- stopwords(language = "portuguese")
print(stopwords_portugues[1:10])
```

```
[1] "de" "a" "o" "que" "e" "do" "da" "em" "um" "para"
```

Atenção: A remoção de stopwords é ótima para descobrir os *temas* de um texto, mas pode ser problemática se você estiver analisando *estilos* ou *marcadores conversacionais* (o uso excessivo do pronome “nós” vs “eu”, por exemplo, desapareceria se você os tratasse como stopwords).

Vamos ver a diferença nas frequências após remover as stopwords:

```
dfm_discursos |>
  dfm_remove(stopwords_portugues) |>
  textstat_frequency(10, groups = partido)
```

	feature	frequency	rank	docfreq	group
1	é	44	1	13	AVANTE
2	presidente	43	2	19	AVANTE
3	sr	36	3	18	AVANTE
4	avante	30	4	17	AVANTE
5	aqui	26	5	10	AVANTE
6	casa	23	6	9	AVANTE
7	pastor	21	7	10	AVANTE
8	bahia	20	8	7	AVANTE
9	município	20	8	5	AVANTE
10	brasil	19	10	9	AVANTE
11	é	23	1	3	CIDADANIA
12	projeto	21	2	3	CIDADANIA
13	aqui	18	3	3	CIDADANIA
14	deputado	17	4	3	CIDADANIA
15	pessoas	17	4	2	CIDADANIA
16	todos	15	6	3	CIDADANIA
17	presidente	14	7	3	CIDADANIA
18	autistas	13	8	2	CIDADANIA
19	sr	12	9	3	CIDADANIA
20	deputados	12	9	2	CIDADANIA
21	é	426	1	83	MDB
22	presidente	195	2	86	MDB
23	sr	186	3	104	MDB
24	mdb	159	4	104	MDB
25	deputado	108	5	48	MDB
26	bloco	105	6	103	MDB
27	aqui	103	7	51	MDB
28	revisão	101	8	101	MDB
29	estado	99	9	34	MDB
30	casa	99	9	40	MDB
31	é	52	1	6	MISSÃO
32	supremo	43	2	4	MISSÃO
33	governo	33	3	4	MISSÃO
34	presidente	21	4	6	MISSÃO
35	gente	21	4	5	MISSÃO
36	vai	21	4	5	MISSÃO
37	agora	21	4	6	MISSÃO

38	deputados	20	8	5	MISSÃO
39	ministro	20	8	4	MISSÃO
40	tribunal	19	10	5	MISSÃO
41	é	850	1	129	NOVO
42	presidente	334	2	134	NOVO
43	sr	250	3	121	NOVO
44	aqui	218	4	88	NOVO
45	novo	180	5	99	NOVO
46	deputado	171	6	81	NOVO
47	ser	151	7	73	NOVO
48	porque	137	8	73	NOVO
49	brasil	136	9	64	NOVO
50	governo	132	10	59	NOVO
51	é	193	1	45	PCdoB
52	presidente	156	2	55	PCdoB
53	lei	80	3	13	PCdoB
54	brasil	76	4	29	PCdoB
55	aqui	71	5	30	PCdoB
56	pcdob	70	6	57	PCdoB
57	bloco	69	7	57	PCdoB
58	sr	67	8	36	PCdoB
59	quero	56	9	37	PCdoB
60	revisão	54	10	54	PCdoB
61	é	218	1	42	PDT
62	presidente	98	2	42	PDT
63	sr	89	3	42	PDT
64	pdt	62	4	43	PDT
65	brasil	56	5	21	PDT
66	bloco	47	6	43	PDT
67	grande	45	7	19	PDT
68	aqui	44	8	16	PDT
69	casa	43	9	19	PDT
70	revisão	41	10	41	PDT
71	é	2127	1	410	PL
72	presidente	1277	2	444	PL
73	sr	1090	3	446	PL
74	pl	651	4	413	PL
75	aqui	639	5	264	PL
76	brasil	605	6	235	PL
77	bloco	599	7	438	PL
78	deputado	526	8	208	PL
79	revisão	443	9	437	PL
80	governo	426	10	170	PL

81	é	227	1	33	PODE
82	sr	108	2	36	PODE
83	brasil	99	3	23	PODE
84	presidente	98	4	32	PODE
85	aqui	93	5	21	PODE
86	todos	70	6	22	PODE
87	estado	53	7	17	PODE
88	pode	53	7	37	PODE
89	bloco	48	9	36	PODE
90	hoje	47	10	18	PODE
91	é	203	1	44	PP
92	presidente	151	2	52	PP
93	sr	103	3	47	PP
94	estado	95	4	26	PP
95	aqui	66	5	29	PP
96	brasil	60	6	30	PP
97	bloco	59	7	51	PP
98	deputado	55	8	27	PP
99	pp	54	9	50	PP
100	quero	52	10	29	PP
101	é	52	1	6	PRD
102	pessoas	21	2	5	PRD
103	brasil	20	3	6	PRD
104	sr	19	4	6	PRD
105	presidente	19	4	6	PRD
106	muitas	19	4	6	PRD
107	todos	16	7	6	PRD
108	gente	16	7	3	PRD
109	todo	15	9	6	PRD
110	infelizmente	15	9	6	PRD
111	é	404	1	72	PSB
112	sr	142	2	81	PSB
113	acre	132	3	20	PSB
114	presidente	123	4	79	PSB
115	brasil	108	5	45	PSB
116	hoje	93	6	51	PSB
117	ser	88	7	32	PSB
118	psb	88	7	50	PSB
119	federal	82	9	33	PSB
120	justiça	77	10	28	PSB
121	é	384	1	75	PSD
122	presidente	236	2	91	PSD
123	sr	181	3	80	PSD

124	psd	125	4	77	PSD
125	brasil	95	5	47	PSD
126	estado	91	6	32	PSD
127	projeto	91	6	28	PSD
128	aqui	88	8	42	PSD
129	cidade	86	9	25	PSD
130	obrigado	83	10	54	PSD
131	é	110	1	14	PSDB
132	federal	55	2	12	PSDB
133	presidente	47	3	14	PSDB
134	estado	36	4	11	PSDB
135	aqui	34	5	13	PSDB
136	sul	33	6	7	PSDB
137	governo	32	7	8	PSDB
138	sr	31	8	11	PSDB
139	entorno	30	9	2	PSDB
140	grande	29	10	11	PSDB
141	é	741	1	114	PSOL
142	presidente	251	2	116	PSOL
143	deputado	239	3	59	PSOL
144	aqui	213	4	87	PSOL
145	psol	179	5	129	PSOL
146	sr	162	6	101	PSOL
147	bloco	140	7	129	PSOL
148	revisão	130	8	128	PSOL
149	glauber	129	9	34	PSOL
150	porque	125	10	63	PSOL
151	é	1842	1	335	PT
152	presidente	1066	2	351	PT
153	sr	712	3	327	PT
154	aqui	558	4	224	PT
155	brasil	546	5	218	PT
156	governo	448	6	166	PT
157	porque	394	7	196	PT
158	pt	383	8	339	PT
159	bloco	379	9	348	PT
160	casa	369	10	168	PT
161	é	89	1	15	PV
162	sr	34	2	16	PV
163	presidente	33	3	15	PV
164	governo	24	4	9	PV
165	brasil	23	5	7	PV
166	aqui	21	6	11	PV

167	federal	20	7	8	PV
168	casa	19	8	11	PV
169	bloco	17	9	16	PV
170	pv	17	9	16	PV
171	cultura	13	1	2	REDE
172	é	9	2	2	REDE
173	presidente	5	3	2	REDE
174	partido	3	4	1	REDE
175	viva	3	4	1	REDE
176	sr	2	6	2	REDE
177	além	2	6	1	REDE
178	povo	2	6	1	REDE
179	bloco	2	6	2	REDE
180	revisão	2	6	2	REDE
181	é	424	1	77	REPUBLICANOS
182	presidente	213	2	88	REPUBLICANOS
183	sr	180	3	88	REPUBLICANOS
184	brasil	132	4	54	REPUBLICANOS
185	deputado	92	5	57	REPUBLICANOS
186	ser	92	5	34	REPUBLICANOS
187	aqui	90	7	46	REPUBLICANOS
188	todos	87	8	45	REPUBLICANOS
189	hoje	83	9	49	REPUBLICANOS
190	povo	80	10	23	REPUBLICANOS
191	é	22	1	9	SOLIDARIEDADE
192	presidente	21	2	9	SOLIDARIEDADE
193	pessoas	18	3	3	SOLIDARIEDADE
194	sr	11	4	9	SOLIDARIEDADE
195	autismo	11	4	1	SOLIDARIEDADE
196	lei	11	4	5	SOLIDARIEDADE
197	projeto	11	4	5	SOLIDARIEDADE
198	solidariedade	11	4	9	SOLIDARIEDADE
199	revisão	9	9	9	SOLIDARIEDADE
200	ordem	8	10	8	SOLIDARIEDADE
201	é	530	1	99	UNIÃO
202	presidente	336	2	110	UNIÃO
203	sr	330	3	107	UNIÃO
204	brasil	189	4	77	UNIÃO
205	aqui	153	5	63	UNIÃO
206	pessoas	151	6	45	UNIÃO
207	casa	143	7	56	UNIÃO
208	união	138	8	104	UNIÃO
209	todos	124	9	59	UNIÃO

2.6 Palavras no contexto (Keywords-in-context)

Keywords-in-context (KWIC) é um método qualitativo excelente para a Análise de Discurso. Ele busca um termo específico e exibe a palavra central cercada pelas palavras que vêm antes e depois (o seu contexto).

Isso nos permite analisar os *enquadramentos* (frames) e *associações* feitas pelos oradores.

```
# Buscando o termo "anistia" e observando 3 palavras de contexto
kwic_anistia <- corpus_discursos |>
  tokens() |>
  kwic("anistia", window = 3) |>
  head(10)

kwic_anistia
```

Keyword-in-context with 10 matches.

[5_Adilson Barroso_PL, 88]	a Lei da Anistia , a extrema
[22_Adriana Ventura_NOVO, 363]	a favor da anistia . E a
[22_Adriana Ventura_NOVO, 374]	a favor da anistia não é passar
[39_Airton Faleiro_PT, 73]	que é a anistia . Eu,
[39_Airton Faleiro_PT, 395]	total, querem anistia total. Eu
[39_Airton Faleiro_PT, 619]	principais não têm anistia e nem têm
[39_Airton Faleiro_PT, 650]	no Brasil. Anistia , não!
[55_Alberto Fraga_PL, 361]	falar aqui de anistia , porque sou
[60_Alencar Santana_PT, 1081]	não" à anistia . Alguns setores
[60_Alencar Santana_PT, 1105]	que votar a anistia . Nós não

Você também pode usar o curinga * para buscar variações (ex: “golp*” pegará “golpe” e “golpista”):

```
golp_kwic <- corpus_discursos |>
  tokens() |>
  kwic("golp*", window = 3) |>
  head(10)

golp_kwic
```

Keyword-in-context with 10 matches.

[39_Airton Faleiro_PT, 255]	quem idealizou o		golpe	
[39_Airton Faleiro_PT, 262]	a mobilização do		golpe	
[39_Airton Faleiro_PT, 352]	, orquestrou o		golpe	
[39_Airton Faleiro_PT, 444]	quem executou o		golpe	
[39_Airton Faleiro_PT, 474]	impedir que esse		golpe	
[39_Airton Faleiro_PT, 534]	, executa um		golpe	
[39_Airton Faleiro_PT, 562]	tentar dar um		golpe	
[44_Alberto Fraga_PL, 124]	preparando mais um		golpe	
[62_Alencar Santana_PT, 29]	Prisão para os		golpistas	
[62_Alencar Santana_PT, 52]	Brasil sofreu um		golpe	

, quem financiou
, quem praticou
contra a democracia
contra a democracia
se efetivasse,
de Estado,
, ocupar e
. Isso aqui
. É isso
, praticado pelos

2.7 Exemplo Prático: Nuvem de Palavras por Partido

Vamos consolidar tudo que aprendemos criando uma visualização que compara as palavras mais ditas pelos deputados do PT e do PL.

Passo 1: Tokenização e Limpeza Vamos quebrar em palavras, remover pontuação, converter para minúsculo e remover stopwords do português.

```
tokens_limpos <- corpus_discursos |>  
  tokens(remove_punct = TRUE, remove_symbols = TRUE, remove_numbers = TRUE) |>  
  tokens_tolower() |>  
  tokens_remove(stopwords(language = "portuguese"))
```

Passo 2: Criando o DFM e removendo jargões políticos O plenário da Câmara é cheio de vícios de linguagem (“senhor”, “presidente”, “deputado”). Se não os removermos, a nossa análise será engolida por eles.

```
dfm_final <- dfm(tokens_limpos)

# Criando lista de palavras "irrelevantes" para o nosso contexto
palavras_irrelevantes <- c("senhor", "sr", "senhora", "sra", "sobre", "presidente",
                           "deputado", "deputada", "lei", "nº", "projeto", "é", "nesta", "de")

dfm_final <- dfm_final |> dfm_remove(palavras_irrelevantes)

dfm_final |>
  textstat_frequency(10, groups = partido)
```

	feature	frequency	rank	docfreq	group
1	avante	30	1	17	AVANTE
2	aqui	26	2	10	AVANTE
3	casa	23	3	9	AVANTE
4	pastor	21	4	10	AVANTE
5	bahia	20	5	7	AVANTE
6	município	20	5	5	AVANTE
7	brasil	19	7	9	AVANTE
8	deus	19	7	4	AVANTE
9	hoje	18	9	8	AVANTE
10	bloco	18	9	18	AVANTE
11	aqui	18	1	3	CIDADANIA
12	pessoas	17	2	2	CIDADANIA
13	todos	15	3	3	CIDADANIA
14	autistas	13	4	2	CIDADANIA
15	deputados	12	5	2	CIDADANIA
16	direitos	11	6	3	CIDADANIA
17	matéria	11	6	1	CIDADANIA
18	obrigado	9	8	2	CIDADANIA
19	cada	9	8	2	CIDADANIA
20	apoio	8	10	2	CIDADANIA
21	mdb	159	1	104	MDB
22	bloco	105	2	103	MDB
23	aqui	103	3	51	MDB
24	revisão	101	4	101	MDB
25	estado	99	5	34	MDB
26	casa	99	5	40	MDB
27	orador	96	7	96	MDB
28	porque	86	8	45	MDB
29	brasil	85	9	44	MDB

30	quero	83	10	45	MDB
31	supremo	43	1	4	MISSÃO
32	governo	33	2	4	MISSÃO
33	gente	21	3	5	MISSÃO
34	vai	21	3	5	MISSÃO
35	agora	21	3	6	MISSÃO
36	deputados	20	6	5	MISSÃO
37	ministro	20	6	4	MISSÃO
38	tribunal	19	8	5	MISSÃO
39	federal	18	9	5	MISSÃO
40	câmara	15	10	5	MISSÃO
41	aqui	218	1	88	NOVO
42	novo	180	2	99	NOVO
43	ser	151	3	73	NOVO
44	porque	137	4	73	NOVO
45	brasil	136	5	64	NOVO
46	governo	132	6	59	NOVO
47	vai	130	7	49	NOVO
48	revisão	126	8	123	NOVO
49	país	111	9	56	NOVO
50	orador	108	10	105	NOVO
51	brasil	76	1	29	PCdoB
52	aqui	71	2	30	PCdoB
53	pcdob	70	3	57	PCdoB
54	bloco	69	4	57	PCdoB
55	quero	56	5	37	PCdoB
56	revisão	54	6	54	PCdoB
57	ordem	51	7	39	PCdoB
58	casa	44	8	25	PCdoB
59	dia	43	9	25	PCdoB
60	importante	42	10	28	PCdoB
61	pdt	62	1	43	PDT
62	brasil	56	2	21	PDT
63	bloco	47	3	43	PDT
64	grande	45	4	19	PDT
65	aqui	44	5	16	PDT
66	casa	43	6	19	PDT
67	revisão	41	7	41	PDT
68	orador	40	8	40	PDT
69	estados	38	9	6	PDT
70	todos	36	10	14	PDT
71	pl	651	1	413	PL
72	aqui	639	2	264	PL

73	brasil	605	3	235	PL
74	bloco	599	4	438	PL
75	revisão	443	5	437	PL
76	governo	426	6	170	PL
77	porque	417	7	213	PL
78	orador	415	8	412	PL
79	hoje	399	9	204	PL
80	ordem	382	10	298	PL
81	brasil	99	1	23	PODE
82	aqui	93	2	21	PODE
83	todos	70	3	22	PODE
84	estado	53	4	17	PODE
85	pode	53	4	37	PODE
86	bloco	48	6	36	PODE
87	hoje	47	7	18	PODE
88	santo	44	8	13	PODE
89	espírito	43	9	12	PODE
90	dia	41	10	16	PODE
91	estado	95	1	26	PP
92	aqui	66	2	29	PP
93	brasil	60	3	30	PP
94	bloco	59	4	51	PP
95	pp	54	5	50	PP
96	quero	52	6	29	PP
97	revisão	51	7	50	PP
98	pessoas	48	8	20	PP
99	hoje	40	9	25	PP
100	orador	40	9	40	PP
101	pessoas	21	1	5	PRD
102	brasil	20	2	6	PRD
103	muitas	19	3	6	PRD
104	todos	16	4	6	PRD
105	gente	16	4	3	PRD
106	todo	15	6	6	PRD
107	infelizmente	15	6	6	PRD
108	fazer	14	8	4	PRD
109	vezes	14	8	6	PRD
110	casa	14	8	5	PRD
111	acre	132	1	20	PSB
112	brasil	108	2	45	PSB
113	hoje	93	3	51	PSB
114	ser	88	4	32	PSB
115	psb	88	4	50	PSB

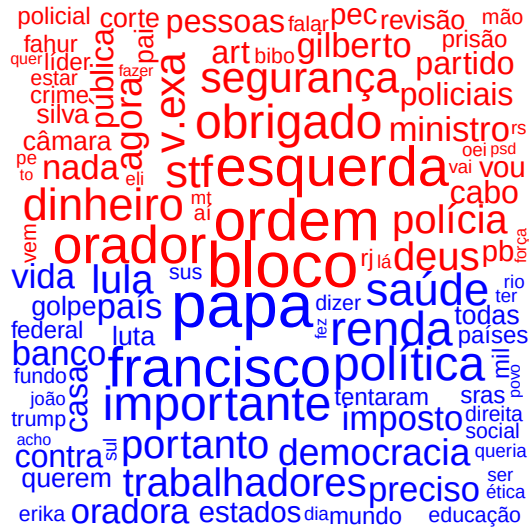
116	federal	82	6	33	PSB
117	justiça	77	7	28	PSB
118	governo	73	8	25	PSB
119	estado	67	9	36	PSB
120	recursos	65	10	31	PSB
121	psd	125	1	77	PSD
122	brasil	95	2	47	PSD
123	estado	91	3	32	PSD
124	aqui	88	4	42	PSD
125	cidade	86	5	25	PSD
126	obrigado	83	6	54	PSD
127	hoje	81	7	45	PSD
128	bloco	77	8	77	PSD
129	todos	75	9	40	PSD
130	revisão	74	10	74	PSD
131	federal	55	1	12	PSDB
132	estado	36	2	11	PSDB
133	aqui	34	3	13	PSDB
134	sul	33	4	7	PSDB
135	governo	32	5	8	PSDB
136	entorno	30	6	2	PSDB
137	grande	29	7	11	PSDB
138	quero	28	8	11	PSDB
139	dourados	26	9	3	PSDB
140	saúde	25	10	8	PSDB
141	aqui	213	1	87	PSOL
142	psol	179	2	129	PSOL
143	bloco	140	3	129	PSOL
144	revisão	130	4	128	PSOL
145	glauber	129	5	34	PSOL
146	porque	125	6	63	PSOL
147	brasil	124	7	57	PSOL
148	contra	112	8	57	PSOL
149	dia	109	9	49	PSOL
150	ordem	108	10	73	PSOL
151	aqui	558	1	224	PT
152	brasil	546	2	218	PT
153	governo	448	3	166	PT
154	porque	394	4	196	PT
155	pt	383	5	339	PT
156	bloco	379	6	348	PT
157	casa	369	7	168	PT
158	lula	357	8	149	PT

159	revisão	343	9	341	PT
160	país	320	10	151	PT
161	governo	24	1	9	PV
162	brasil	23	2	7	PV
163	aqui	21	3	11	PV
164	federal	20	4	8	PV
165	casa	19	5	11	PV
166	bloco	17	6	16	PV
167	pv	17	6	16	PV
168	revisão	16	8	16	PV
169	orador	16	8	16	PV
170	obrigado	14	10	9	PV
171	cultura	13	1	2	REDE
172	partido	3	2	1	REDE
173	viva	3	2	1	REDE
174	além	2	4	1	REDE
175	povo	2	4	1	REDE
176	bloco	2	4	2	REDE
177	revisão	2	4	2	REDE
178	orador	2	4	2	REDE
179	cada	2	4	2	REDE
180	sim	2	4	2	REDE
181	brasil	132	1	54	REPUBLICANOS
182	ser	92	2	34	REPUBLICANOS
183	aqui	90	3	46	REPUBLICANOS
184	todos	87	4	45	REPUBLICANOS
185	hoje	83	5	49	REPUBLICANOS
186	povo	80	6	23	REPUBLICANOS
187	pessoas	71	7	27	REPUBLICANOS
188	republicanos	71	7	55	REPUBLICANOS
189	casa	68	9	43	REPUBLICANOS
190	estado	67	10	34	REPUBLICANOS
191	pessoas	18	1	3	SOLIDARIEDADE
192	autismo	11	2	1	SOLIDARIEDADE
193	solidariedade	11	2	9	SOLIDARIEDADE
194	revisão	9	4	9	SOLIDARIEDADE
195	ordem	8	5	8	SOLIDARIEDADE
196	orador	7	6	7	SOLIDARIEDADE
197	pessoa	7	6	2	SOLIDARIEDADE
198	casa	7	6	4	SOLIDARIEDADE
199	nacional	7	6	3	SOLIDARIEDADE
200	quero	6	10	4	SOLIDARIEDADE
201	brasil	189	1	77	UNIÃO



67

PL



PT

2.8 Para saber mais

- Leia mais sobre morfemas, tokenização no Capítulo 2 do livro “Speech and Language Processing” (Jurafsky and Martin 2026) e os Capítulos 4 e 5 do livro “Processamento de Linguagem Natural” (Caseli and Nunes 2024).
- Estude os tutoriais completos da biblioteca `quanteda` em <https://tutorials.quanteda.io/>. Saiba mais sobre a biblioteca lendo o artigo de Benoit et al. (2018).

2.9 Estudos guiados

2.9.1 Criando corpus a partir de uma planilha CSV

```
library(tidyverse)
library(quanteda)
# caso não tenha as bibliotecas instaladas, execute install.packages(c("tidyverse", "quanteda"))

# Ler arquivo discursos_camara_abril_2025.csv (planilha) em uma tabela do R
# Imprimindo detalhes sobre a leitura do arquivo (show_col_types)
tabela_discursos <- read_csv("discursos_camara_abril_2025.csv", show_col_types = TRUE)
```

Rows: 1527 Columns: 21

-- Column specification -----

Delimiter: ","

chr (16): nome, uf, partido, uri, nome_civil, sexo, uf_nascimento, municipi...

dbl (1): id

lgl (1): data_falecimento

dtm (1): hora_inicio

date (2): data_nascimento, data_situacao

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# Criar coluna identificador do texto "doc_id", contendo o nome e partido do discurso
tabela_discursos <- tabela_discursos |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_"))

# Criar corpus, indicando a coluna com texto (transcricao) e identificador (doc_id)
corpus_discursos <- corpus(
  tabela_discursos,
  text_field = "transcricao",
  docid_field = "doc_id"
)

# Imprimir um resumo do corpus carregado
print(corpus_discursos)
```

Corpus consisting of 1,527 documents and 20 docvars.

1_Acácio Favacho_MDB :

"DISCURSO NA ÍNTEGRA ENCAMINHADO PELO SR. DEPUTADO ACÁCIO FAV..."

2_Acácio Favacho_MDB :

"O SR. ACÁCIO FAVACHO (Bloco/MDB - AP. Sem revisão do orador...."

3_Acácio Favacho_MDB :

"DISCURSO NA ÍNTEGRA ENCAMINHADO PELO SR. DEPUTADO ACÁCIO FAV..."

4_Adail Filho_REPUBLICANOS :

"O SR. ADAIL FILHO (Bloco/REPUBLICANOS - AM. Pela ordem. Sem ..."

5_Adilson Barroso_PL :

"O SR. ADILSON BARROSO (Bloco/PL - SP. Sem revisão do orador...."

6_Adilson Barroso_PL :

"O SR. ADILSON BARROSO (Bloco/PL - SP. Sem revisão do orador...."

[reached max_ndoc ... 1,521 more documents]

2.9.2 Quebrando o corpus em palavras (tokens)

```
library(tidyverse)
library(quanteda)
# caso não tenha as bibliotecas instaladas, execute install.packages(c("tidyverse", "quanteda"))

# Ler arquivo discursos_camara_abril_2025.csv (planilha) em uma tabela do R
tabela_discursos <- read_csv("discursos_camara_abril_2025.csv", show_col_types = FALSE)

# Criar coluna identificador do texto "doc_id", contendo o nome e partido do discurso
tabela_discursos <- tabela_discursos |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_"))

# Criar corpus, indicando a coluna com texto (transcricao) e identificador (doc_id)
corpus_discursos <- corpus(
  tabela_discursos,
  text_field = "transcricao",
  docid_field = "doc_id"
)

# Criar tokens
tokens_discursos <- corpus_discursos |>
  tokens()

# Imprimir tokens do primeiro discurso
print(tokens_discursos)
```

Tokens consisting of 1,527 documents and 20 docvars.

1_Acácio Favacho_MDB :

```
[1] "DISCURSO"      "NA"            "ÍNTEGRA"       "ENCAMINHADO"  "PELO"
[6] "SR"            "."             "DEPUTADO"      "ACÁCIO"       "FAVACHO"
[11] "("             "SEM"
[ ... and 416 more ]
```

2_Acácio Favacho_MDB :

```

[1] "O"      "SR"      "."      "ACÁCIO" "FAVACHO" "("      "Bloco"
[8] "/"      "MDB"     "-"      "AP"      "."
[ ... and 647 more ]

3_Acácio Favacho_MDB :
[1] "DISCURSO" "NA"      "ÍNTEGRA" "ENCAMINHADO" "PELO"
[6] "SR"        "."      "DEPUTADO" "ACÁCIO"      "FAVACHO"
[11] "("        "SEM"
[ ... and 576 more ]

4_Adail Filho_REPUBLICANOS :
[1] "O"      "SR"      "."      "ADAIL"      "FILHO"
[6] "("      "Bloco"   "/"      "REPUBLICANOS" "-"
[11] "AM"     "."
[ ... and 248 more ]

5_Adilson Barroso_PL :
[1] "O"      "SR"      "."      "ADILSON" "BARROSO" "("      "Bloco"
[8] "/"      "PL"      "-"      "SP"      "."
[ ... and 489 more ]

6_Adilson Barroso_PL :
[1] "O"      "SR"      "."      "ADILSON" "BARROSO" "("      "Bloco"
[8] "/"      "PL"      "-"      "SP"      "."
[ ... and 474 more ]

[ reached max_ndoc ... 1,521 more documents ]

```

2.9.3 Pré-processamento de tokens: removendo stopwords e stematização

```

library(tidyverse)
library(quanteda)
library(stopwords)
# caso não tenha as bibliotecas instaladas, execute install.packages(c("tidyverse", "quanteda"))

# Ler arquivo discursos_camara_abril_2025.csv (planilha) em uma tabela do R
tabela_discursos <- read_csv("discursos_camara_abril_2025.csv", show_col_types = FALSE)

# Criar coluna identificador do texto "doc_id", contendo o nome e partido do discurso
tabela_discursos <- tabela_discursos |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_"))

```

```

# Criar corpus, indicando a coluna com texto (transcricao) e identificador (doc_id)
corpus_discursos <- corpus(
  tabela_discursos,
  text_field = "transcricao",
  docid_field = "doc_id"
)

# Criar lista de stopwords comuns em português
lista_stopwords <- stopwords("portuguese")

# Adicionar novas palavras comuns de discursos na lista de stopwords
lista_stopwords <- c(lista_stopwords, "é", "sr", "sra", "senhor", "senhora", "senhores", "se")

# Criar tokens:
# removendo pontuação (remove_punct), números (remove_numbers) e símbolos (remove_symbols)
# transformando tokens para minúsculo (tokens_tolower)
# removendo stopwords (tokens_remove)
# stematizando palavras (tokens_wordstem)
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE, remove_symbols = TRUE) |>
  tokens_tolower() |>
  tokens_remove(lista_stopwords) |>
  tokens_wordstem(language = "portuguese")

# Imprimir tokens do primeiro discurso
print(tokens_discursos)

```

Tokens consisting of 1,527 documents and 20 docvars.

1_Acácio Favacho_MDB :

```

[1] "discurs" "íntegr" "encaminh" "deput" "acáci" "favach"
[7] "registr" "taquigráf" "nobr" "coleg" "car" "telespect"
[ ... and 209 more ]

```

2_Acácio Favacho_MDB :

```

[1] "acáci" "favach" "bloc" "mdb" "ap" "revisã" "orador"
[8] "bom" "dia" "gentil" "quer" "agradec"
[ ... and 266 more ]

```

3_Acácio Favacho_MDB :

```

[1] "discurs" "íntegr" "encaminh" "deput" "acáci" "favach"
[7] "registr" "taquigráf" "deput" "car" "telespect" "sistem"

```



```
[ ... and 279 more ]

4_Adail Filho_REPUBLICANOS :
  [1] "adail"      "filh"      "bloc"      "republican" "am"
  [6] "ordem"      "revisã"    "orador"    "sras"      "srs"
  [11] "deput"      "venh"
  [ ... and 104 more ]

5_Adilson Barroso_PL :
  [1] "adilson" "barros" "bloc" "pl" "sp" "revisã"
  [7] "orador" "coleg" "deput" "tod" "acompanh" "ouv"
  [ ... and 211 more ]

6_Adilson Barroso_PL :
  [1] "adilson" "barros" "bloc" "pl" "sp" "revisã"
  [7] "orador" "coleg" "venh" "aqu" "dar" "justific"
  [ ... and 190 more ]

[ reached max_ndoc ... 1,521 more documents ]
```

2.9.4 Construindo DFM a partir de tokens:

```
library(tidyverse)
library(quanteda)
library(quanteda.textstats)
library(stopwords)
# caso não tenha as bibliotecas instaladas, execute install.packages(c("tidyverse", "quanteda"))

# Ler arquivo discursos_camara_abril_2025.csv (planilha) em uma tabela do R
tabela_discursos <- read_csv("discursos_camara_abril_2025.csv", show_col_types = FALSE)

# Criar coluna identificador do texto "doc_id", contendo o nome e partido do discurso
tabela_discursos <- tabela_discursos |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_"))

# Criar corpus, indicando a coluna com texto (transcricao) e identificador (doc_id)
corpus_discursos <- corpus(
  tabela_discursos,
  text_field = "transcricao",
  docid_field = "doc_id"
)
```

```

# Criar lista de stopwords comuns em português
lista_stopwords <- stopwords("portuguese")

# Adicionar novas palavras comuns de discursos na lista de stopwords
lista_stopwords <- c(lista_stopwords, "é", "sr", "sra", "senhor", "senhora", "senhores", "se

# Criar tokens:
# removendo pontuação (remove_punct), números (remove_numbers) e símbolos (remove_symbols)
# transformando tokens para minúsculo (tokens_tolower)
# removendo stopwords (tokens_remove)
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE, remove_symbols = TRUE) |>
  tokens_tolower() |>
  tokens_remove(lista_stopwords)

# Construir DFM:
dfm_discursos <- dfm(tokens_discursos)

# Imprimir DFM:
print(dfm_discursos)

```

Document-feature matrix of: 1,527 documents, 23,772 features (99.41% sparse) and 20 docvars.

docs	features					
		discurso	íntegra	encaminhado	deputado	acácio
1_Acácio Favacho_MDB		1	1	1	2	1
2_Acácio Favacho_MDB		0	0	0	1	1
3_Acácio Favacho_MDB		1	1	1	1	1
4_Adail Filho_REPUBLICANOS		0	0	0	0	0
5_Adilson Barroso_PL		0	0	0	0	0
6_Adilson Barroso_PL		0	0	0	2	0

docs	features					
		favacho	registro	taquigráfico	nobres	colegas
1_Acácio Favacho_MDB		1	1	1	1	1
2_Acácio Favacho_MDB		1	0	0	0	0
3_Acácio Favacho_MDB		1	1	1	0	0
4_Adail Filho_REPUBLICANOS		0	0	0	0	0
5_Adilson Barroso_PL		0	0	0	0	2
6_Adilson Barroso_PL		0	0	0	0	1

[reached max_ndoc ... 1,521 more documents, reached max_nfeat ... 23,762 more features]

2.9.5 Contando palavras mais utilizadas (em todos os discursos)

```
library(tidyverse)
library(quanteda)
library(quanteda.textstats)
library(stopwords)
# caso não tenha as bibliotecas instaladas, execute install.packages(c("tidyverse", "quanteda"))

# Ler arquivo discursos_camara_abril_2025.csv (planilha) em uma tabela do R
tabela_discursos <- read_csv("discursos_camara_abril_2025.csv", show_col_types = FALSE)

# Criar coluna identificador do texto "doc_id", contendo o nome e partido do discurso
tabela_discursos <- tabela_discursos |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_"))

# Criar corpus, indicando a coluna com texto (transcricao) e identificador (doc_id)
corpus_discursos <- corpus(
  tabela_discursos,
  text_field = "transcricao",
  docid_field = "doc_id"
)

# Criar lista de stopwords comuns em português
lista_stopwords <- stopwords("portuguese")

# Adicionar novas palavras comuns de discursos na lista de stopwords
lista_stopwords <- c(lista_stopwords, "é", "sr", "sra", "senhor", "senhora", "senhores", "sen")

# Criar tokens:
# removendo pontuação (remove_punct), números (remove_numbers) e símbolos (remove_symbols)
# transformando tokens para minúsculo (tokens_tolower)
# removendo stopwords (tokens_remove)
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE, remove_symbols = TRUE) |>
  tokens_tolower() |>
  tokens_remove(lista_stopwords)

# Construir DFM:
dfm_discursos <- dfm(tokens_discursos)

# Construir tabela com 20 palavras mais utilizadas:
palavras_discursos <- dfm_discursos |>
```

```
textstat_frequency(n = 20)

# Imprimir tabela:
print(palavras_discursos)
```

	feature	frequency	rank	docfreq	group
1	brasil	2032	1	819	all
2	aqui	2012	2	860	all
3	bloco	1546	3	1336	all
4	deputado	1518	4	661	all
5	revisão	1377	5	1369	all
6	casa	1338	6	649	all
7	governo	1324	7	526	all
8	hoje	1314	8	730	all
9	porque	1253	9	659	all
10	ser	1209	10	590	all
11	todos	1204	11	626	all
12	projeto	1165	12	484	all
13	orador	1112	13	1102	all
14	quero	1097	14	637	all
15	pessoas	1091	15	467	all
16	estado	1088	16	480	all
17	país	1066	17	551	all
18	federal	984	18	490	all
19	ordem	941	19	797	all
20	povo	929	20	434	all

2.9.6 Contando palavras mais utilizadas por grupos (ex: partido)

```
library(tidyverse)
library(quanteda)
library(quanteda.textstats)
library(stopwords)

# caso não tenha as bibliotecas instaladas, execute install.packages(c("tidyverse", "quanteda"))

# Ler arquivo discursos_camara_abril_2025.csv (planilha) em uma tabela do R
tabela_discursos <- read_csv("discursos_camara_abril_2025.csv", show_col_types = FALSE)

# Criar coluna identificador do texto "doc_id", contendo o nome e partido do discurso
tabela_discursos <- tabela_discursos |>
```

```

mutate(doc_id = str_c(row_number(), nome, partido, sep = "_"))

# Criar corpus, indicando a coluna com texto (transcricao) e identificador (doc_id)
corpus_discursos <- corpus(
  tabela_discursos,
  text_field = "transcricao",
  docid_field = "doc_id"
)

# Criar lista de stopwords comuns em português
lista_stopwords <- stopwords("portuguese")

# Adicionar novas palavras comuns de discursos na lista de stopwords
lista_stopwords <- c(lista_stopwords, "é", "sr", "sra", "senhor", "senhora", "senhores", "se")

# Criar tokens:
# removendo pontuação (remove_punct), números (remove_numbers) e símbolos (remove_symbols)
# transformando tokens para minúsculo (tokens_tolower)
# removendo stopwords (tokens_remove)
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE, remove_symbols = TRUE) |>
  tokens_tolower() |>
  tokens_remove(lista_stopwords)

# Construir DFM:
dfm_discursos <- dfm(tokens_discursos)

# Construir tabela com 3 palavras mais utilizadas por partidos:
palavras_discursos <- dfm_discursos |>
  textstat_frequency(n = 3, groups = partido)

# Imprimir tabela:
print(palavras_discursos)

```

	feature	frequency	rank	docfreq	group
1	avante	30	1	17	AVANTE
2	aqui	26	2	10	AVANTE
3	casa	23	3	9	AVANTE
4	projeto	21	1	3	CIDADANIA
5	aqui	18	2	3	CIDADANIA
6	deputado	17	3	3	CIDADANIA
7	mdb	101	1	67	MDB

8	aqui	85	2	38	MDB
9	casa	72	3	26	MDB
10	supremo	43	1	4	MISSÃO
11	governo	33	2	4	MISSÃO
12	gente	21	3	5	MISSÃO
13	novo	101	1	56	NOVO
14	aqui	101	1	45	NOVO
15	ser	86	3	37	NOVO
16	lei	80	1	13	PCdoB
17	brasil	74	2	28	PCdoB
18	pcdob	68	3	55	PCdoB
19	pdt	54	1	36	PDT
20	brasil	43	2	17	PDT
21	aqui	42	3	14	PDT
22	aqui	510	1	212	PL
23	brasil	507	2	185	PL
24	pl	500	3	322	PL
25	brasil	99	1	23	PODE
26	aqui	93	2	21	PODE
27	todos	70	3	22	PODE
28	estado	95	1	26	PP
29	aqui	66	2	29	PP
30	brasil	60	3	30	PP
31	pessoas	21	1	5	PRD
32	brasil	20	2	6	PRD
33	muitas	19	3	6	PRD
34	acre	126	1	16	PSB
35	brasil	84	2	35	PSB
36	psb	70	3	40	PSB
37	psd	125	1	77	PSD
38	estado	89	2	31	PSD
39	brasil	84	3	41	PSD
40	federal	55	1	12	PSDB
41	estado	36	2	11	PSDB
42	aqui	34	3	13	PSDB
43	deputado	194	1	41	PSOL
44	aqui	142	2	65	PSOL
45	psol	134	3	96	PSOL
46	brasil	485	1	203	PT
47	aqui	460	2	201	PT
48	governo	414	3	156	PT
49	governo	24	1	9	PV
50	brasil	23	2	7	PV

51	aqui	21	3	11	PV
52	cultura	13	1	2	REDE
53	partido	3	2	1	REDE
54	viva	3	2	1	REDE
55	brasil	126	1	52	REPUBLICANOS
56	ser	89	2	32	REPUBLICANOS
57	deputado	88	3	54	REPUBLICANOS
58	pessoas	18	1	3	SOLIDARIEDADE
59	autismo	11	2	1	SOLIDARIEDADE
60	lei	11	2	5	SOLIDARIEDADE
61	brasil	176	1	69	UNIÃO
62	pessoas	141	2	39	UNIÃO
63	união	123	3	89	UNIÃO

2.9.7 Núvem de palavras

```
library(tidyverse)
library(quanteda)
library(quanteda.textstats)
library(quanteda.textplots)
library(stopwords)
# caso não tenha as bibliotecas instaladas, execute install.packages(c("tidyverse", "quanteda"))

# Ler arquivo discursos_camara_abril_2025.csv (planilha) em uma tabela do R
tabela_discursos <- read_csv("discursos_camara_abril_2025.csv", show_col_types = FALSE)

# Criar coluna identificador do texto "doc_id", contendo o nome e partido do discurso
tabela_discursos <- tabela_discursos |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_"))

# Criar corpus, indicando a coluna com texto (transcricao) e identificador (doc_id)
corpus_discursos <- corpus(
  tabela_discursos,
  text_field = "transcricao",
  docid_field = "doc_id"
)

# Criar lista de stopwords comuns em português
lista_stopwords <- stopwords("portuguese")

# Adicionar novas palavras comuns de discursos na lista de stopwords
```

```

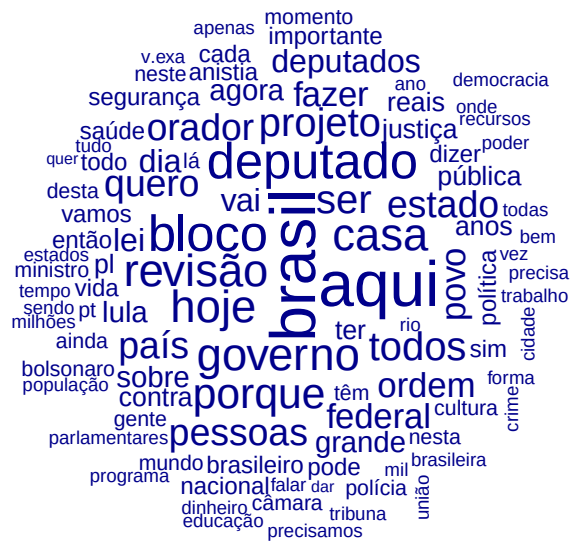
lista_stopwords <- c(lista_stopwords, "é", "sr", "sra", "senhor", "senhora", "senhores", "se

# Criar tokens:
# removendo pontuação (remove_punct), números (remove_numbers) e símbolos (remove_symbols)
# transformando tokens para minúsculo (tokens_tolower)
# removendo stopwords (tokens_remove)
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE, remove_symbols = TRUE) |>
  tokens_tolower() |>
  tokens_remove(lista_stopwords)

# Construir DFM:
dfm_discursos <- dfm(tokens_discursos)

# Desenhar núvem de palavras:
textplot_wordcloud(dfm_discursos, max_words = 100)

```



2.9.8 Núvem de palavras comparativa


```

library(tidyverse)
library(quanteda)
library(quanteda.textstats)
library(quanteda.textplots)
library(stopwords)
# caso não tenha as bibliotecas instaladas, execute install.packages(c("tidyverse", "quanteda"))

# Ler arquivo discursos_camara_abril_2025.csv (planilha) em uma tabela do R
tabela_discursos <- read_csv("discursos_camara_abril_2025.csv", show_col_types = FALSE)

# Criar coluna identificador do texto "doc_id", contendo o nome e partido do discurso
tabela_discursos <- tabela_discursos |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_"))

# Criar corpus, indicando a coluna com texto (transcricao) e identificador (doc_id)
corpus_discursos <- corpus(
  tabela_discursos,
  text_field = "transcricao",
  docid_field = "doc_id"
)

# Criar lista de stopwords comuns em português
lista_stopwords <- stopwords("portuguese")

# Adicionar novas palavras comuns de discursos na lista de stopwords
lista_stopwords <- c(lista_stopwords, "é", "sr", "sra", "senhor", "senhora", "senhores", "se")

# Criar tokens:
# removendo pontuação (remove_punct), números (remove_numbers) e símbolos (remove_symbols)
# transformando tokens para minúsculo (tokens_tolower)
# removendo stopwords (tokens_remove)
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE, remove_symbols = TRUE) |>
  tokens_tolower() |>
  tokens_remove(lista_stopwords)

# Construir DFM:
dfm_discursos <- dfm(tokens_discursos) |>
  dfm_group(groups = sexo)

# Desenhar núvem de palavras comparando grupos (gêneros):
textplot_wordcloud(

```

)

[illegible]

2.10 Atividade prática

1. Carregue o arquivo `discursos.csv` e familiarize-se com as colunas.
2. Crie um *corpus* filtrando apenas os discursos de deputados de um gênero ou estado.
3. Realize a tokenização e remova as *stopwords*.
4. Crie uma Matriz de Documentos e Termos (DFM).
5. Conte as palavras mais comuns utilizando `textstat_frequency` e visualize os resultados utilizando uma nuvem de palavras.
6. *Desafio*: Tente utilizar a função `kwic` para descobrir em que contexto os parlamentares do gênero ou estado utilizaram a palavra “saúde” ou “educação”.

3 N-gramas, Collocations e TF-IDF

i Nesta aula

- Dicionários.
- N-gramas e Skip-gramas
- Collocations.
- Estatísticas textuais: TF-IDF, diversidade léxica, leituraabilidade, keyness.

Link para o Google Colab: <https://colab.research.google.com/drive/10yt3BrJ7XV97xgv8ivNdILTJT6x9jnNS?usp=sharing>

Carregando as bibliotecas necessárias:

```
if (!"tidyverse" %in% installed.packages()) {  
  install.packages("tidyverse")  
}  
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
v dplyr      1.2.0      v readr      2.2.0  
v forcats    1.0.1      v stringr    1.6.0  
v ggplot2    4.0.2      v tibble     3.3.1  
v lubridate  1.9.5      v tidyr      1.3.2  
v purrr      1.2.1
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()  
x dplyr::lag()     masks stats::lag()
```

i Use the conflicted package (<<http://conflicted.r-lib.org/>>) to force all conflicts to become

```
if (!"quanteda" %in% installed.packages()) {  
  install.packages("quanteda")  
}  
library(quanteda)
```

Package version: 4.3.1
Unicode version: 15.0
ICU version: 72.1
Parallel computing: 12 of 12 threads used.
See <https://quanteda.io> for tutorials and examples.

```
if (!"quanteda.textstats" %in% installed.packages()) {  
  install.packages("quanteda.textstats")  
}  
library(quanteda.textstats)  
  
if (!"quanteda.textplots" %in% installed.packages()) {  
  install.packages("quanteda.textplots")  
}  
library(quanteda.textplots)  
  
if (!"stopwords" %in% installed.packages()) {  
  install.packages("stopwords")  
}  
library(stopwords)
```

Carregando dados de discursos parlamentares de abril de 2025:

```
dados_discursos <- read_csv("discursos_camara_abril_2025.csv") |>  
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_"))
```

Rows: 1527 Columns: 21

-- Column specification -----

Delimiter: ","

chr (16): nome, uf, partido, uri, nome_civil, sexo, uf_nascimento, municipi...

dbl (1): id

lgl (1): data_falecimento

dtm (1): hora_inicio

date (2): data_nascimento, data_situacao

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
corpus_discursos <- dados_discursos |>  
  corpus(text_field = "transcricao")
```

3.1 Dicionários

Dicionários são listas pré-definidas de palavras ou expressões associadas organizadas em uma hierarquia ou taxonomia.

Por exemplo, localizações geográficas podem ser organizadas em níveis para análise de menções em diferentes níveis: setores, bairros, cidades, estados, países etc. Vejamos um exemplo de análise de menções de regiões administrativas em planos de governo no DF:

```
dados_csv <- 'ano,candidato,plano
2014,Candidato A,"Vamos investir muito em Ceilândia e melhorar o transporte no Plano Piloto.
2018,Candidato B,"A saúde em Samambaia é a nossa prioridade. Ceilândia precisa de mais escola.
2022,Candidato C,"Nosso plano de governo foca no desenvolvimento do Gama e de Sobradinho. O plano
```

```
dados <- read_csv(I(dados_csv)) |> mutate(doc_id = str_c(ano, candidato))
```

Rows: 3 Columns: 3

```
-- Column specification -----
Delimiter: ","
chr (2): candidato, plano
dbl (1): ano
```

- i Use ``spec()`` to retrieve the full column specification for this data.
- i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
corpus_planos <- corpus(dados, text_field = "plano")

tokens_planos <- tokens(corpus_planos, remove_punct = TRUE) |>
  tokens_tolower()

tokens_planos
```

Tokens consisting of 3 documents and 2 docvars.

2014Candidato A :

```
[1] "vamos"      "investir"   "muito"      "em"         "ceilândia"
[6] "e"          "melhorar"   "o"          "transporte" "no"
[11] "plano"      "piloto"
[ ... and 12 more ]
```

2018Candidato B :

```
[1] "a"          "saúde"      "em"         "samambaia" "é"
```

```
[6] "a"          "nossa"      "prioridade" "ceilândia"  "precisa"
[11] "de"         "mais"
[ ... and 17 more ]
```

2022Candidato C :

```
[1] "nosso"      "plano"      "de"         "governo"
[5] "foca"       "no"         "desenvolvimento" "do"
[9] "gama"       "e"         "de"         "sobradinho"
[ ... and 11 more ]
```

Podemos montar um dicionário para representar diferentes regiões administrativas, incluindo setores, diferentes grafias e, possivelmente, ruas, quadras, viadutos, locais comuns, etc.

```
dicionario_ras <- dictionary(list(
  ceilandia = c("ceilândia", "ceilandia"),
  plano_piloto = c("plano piloto", "brasília", "brasilíia"),
  taguatinga = c("taguatinga"),
  samambaia = c("samambaia"),
  gama = c("gama"),
  sobradinho = c("sobradinho"),
  jardim_botanico = c("jardim botânico", "jardim botanico", "mangueiral", "tororó")
))

# Buscar os tokens que correspondem ao dicionário
tokens_ras <- tokens_lookup(tokens_planos, dictionary = dicionario_ras)

# Criar a DFM para contabilizar as menções
matriz_contagem <- dfm(tokens_ras)

matriz_contagem
```

Document-feature matrix of: 3 documents, 7 features (42.86% sparse) and 2 docvars.

		features						
docs		ceilandia	plano_piloto	taguatinga	samambaia	gama	sobradinho	
2014Candidato A		1	1	1	0	0	0	
2018Candidato B		1	0	0	1	1	0	
2022Candidato C		0	1	1	0	1	1	

		features	
docs		jardim_botanico	
2014Candidato A		1	
2018Candidato B		1	
2022Candidato C		0	

Dicionários também podem conter múltiplos níveis de hierarquia. Por exemplo:

```
pol_dict <- dictionary(list(
  Esquerda = list(
    Economia = c("redistribuição", "estatização", "direitos trabalhistas", "justiça social",
    Social = c("inclusão", "minorias", "diversidade", "identidade", "coletivo", "direitos hu
    Papel_Estado = c("estado indutor", "serviços públicos", "bem-estar social", "regulação")
  ),
  Direita = list(
    Economia = c("livre mercado", "privatização", "responsabilidade fiscal", "eficiência", "
    Social = c("família", "valores", "segurança", "tradição", "liberdade individual", "pátri
    Papel_Estado = c("estado mínimo", "desestatização", "desburocratização", "parceria públi
  )
))

corpus_discursos |>
  tokens() |>
  tokens_tolower() |>
  tokens_lookup(pol_dict, levels = 1:2) |>
  dfm() |>
  colSums()
```

esquerda.economia	esquerda.social	esquerda.papel_estado
103	271	39
direita.economia	direita.social	direita.papel_estado
61	989	6

Existem vários dicionários disponíveis para análise de texto, como dicionários de sentimentos, dicionários de categorias temáticas, dicionários de palavras positivas e negativas, entre outros. Veremos mais exemplos nas próximas aulas.

A Câmara dos Deputados mantém um dicionário de temas para classificar os discursos em diferentes categorias temáticas. O dicionário é organizado em uma hierarquia de temas e subtemas, permitindo uma análise detalhada dos tópicos abordados nos discursos.

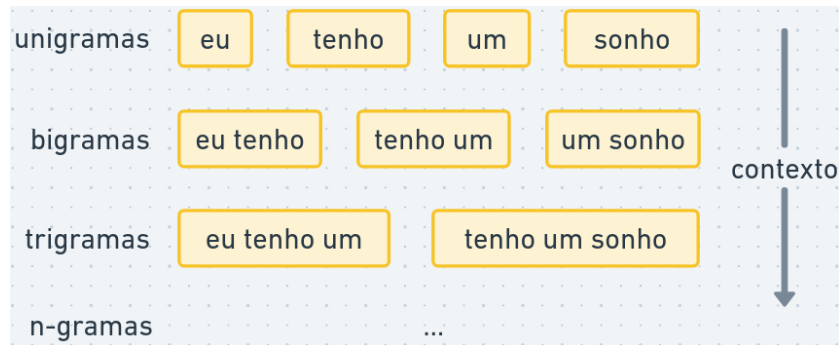
<https://dadosabertos.camara.leg.br/swagger/api.html?tab=staticfile#tesauro>

3.2 N-gramas

Na aula anterior, vimos como criar um corpus a partir de dados de texto e como separar o texto em palavras ou partes de palavras.

Por mais que palavras isoladas possuam significados interessantes, o significado de um texto é mais do que a soma das palavras que o compõem. Por exemplo, a expressão “teto de vidro” tem um significado diferente da soma dos significados de “teto”, “de” e “vidro”.

Os *n-gramas* consistem em sequências contíguas de “n” palavras. Por exemplo, na frase “eu tenho um sonho”:



3.2.0.1 Exemplo

```
bigramas_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE) |>
  tokens_ngrams(3)

bigramas_discursos |>
  dfm() |>
  textstat_frequency(n = 200)
```

	feature	frequency	rank	docfreq	group
1	sem_revisão_do	1103	1	1102	all
2	revisão_do_orador	1103	1	1102	all
3	ordem_sem_revisão	742	3	741	all
4	pela_ordem_sem	728	4	727	all
5	do_orador_sr	448	5	448	all
6	orador_sr_presidente	448	5	448	all
7	projeto_de_lei	330	7	185	all
8	sem_revisão_da	262	8	261	all
9	revisão_da_oradora	262	8	261	all
10	câmara_dos_deputados	224	10	171	all
11	do_orador_presidente	217	11	217	all
12	bilhões_de_reais	209	12	114	all

13	voz_do_brasil	204	13	198	all
14	obrigado_sr_presidente	192	14	167	all
15	do_orador_obrigado	172	15	172	all
16	rio_grande_do	169	16	88	all
17	rio_de_janeiro	168	17	95	all
18	o_povo_brasileiro	164	18	121	all
19	senhoras_e_senhores	163	19	123	all
20	a_esta_tribuna	160	20	152	all
21	a_voz_do	160	20	153	all
22	presidente_hugo_motta	159	22	111	all
23	programa_a_voz	155	23	149	all
24	grande_do_sul	149	24	73	all
25	sr_presidente_eu	149	24	138	all
26	mais_uma_vez	147	26	119	all
27	de_lei_nº	141	27	82	all
28	discurso_na_integra	130	28	130	all
29	na_integra_encaminhado	130	28	130	all
30	sem_registro_taquigráfico	130	28	130	all
31	supremo_tribunal_federal	129	31	76	all
32	8_de_janeiro	129	31	87	all
33	meios_de_comunicação	128	33	121	all
34	pelo_sr_deputado	125	34	125	all
35	do_brasil_e	125	34	122	all
36	no_programa_a	125	34	121	all
37	sras_e_srs	124	37	114	all
38	integra_encaminhado_pelo	121	38	121	all
39	encaminhado_pelo_sr	121	38	121	all
40	o_presidente_lula	119	40	86	all
41	mais_do_que	117	41	98	all
42	que_o_brasil	117	41	98	all
43	milhões_de_reais	116	43	61	all
44	do_presidente_lula	116	43	86	all
45	de_comunicação_dest	115	45	110	all
46	rs_pela_ordem	114	46	114	all
47	conselho_de_ética	114	46	56	all
48	comunicação_dest	113	48	110	all
49	rj_pela_ordem	113	48	113	all
50	e_srs_deputados	111	50	102	all
51	nós_temos_que	110	51	86	all
52	o_projeto_de	110	51	93	all
53	do_nosso_país	107	53	94	all
54	na_comissão_de	107	53	79	all
55	a_todos_os	105	55	92	all

56	que_o_governo	104	56	86	all
57	sr_presidente_sras	103	57	96	all
58	o_sr_presidente	103	57	72	all
59	de_todos_os	103	57	91	all
60	do_rio_de	103	57	56	all
61	alexandre_de_moraes	102	61	55	all
62	do_estado_do	101	62	76	all
63	da_câmara_dos	101	62	85	all
64	que_é_a	101	62	89	all
65	que_a_gente	101	62	73	all
66	que_nós_estamos	100	66	87	all
67	da_segurança_pública	99	67	48	all
68	ordem_do_dia	98	68	38	all
69	de_segurança_pública	96	69	61	all
70	deputado_glauber_braga	96	69	55	all
71	presidente_sras_e	93	71	88	all
72	o_que_é	93	71	79	all
73	muito_obrigado_sr	92	73	84	all
74	o_que_está	91	74	83	all
75	imposto_de_renda	91	74	45	all
76	para_que_a	90	76	80	all
77	cada_vez_mais	89	77	73	all
78	orador_obrigado_presidente	89	77	89	all
79	que_é_o	89	77	78	all
80	do_projeto_de	89	77	50	all
81	o_deputado_glauber	89	77	38	all
82	senhor_presidente_senhoras	87	82	85	all
83	do_povo_brasileiro	87	82	72	all
84	do_deputado_glauber	86	84	49	all
85	para_que_o	84	85	74	all
86	no_nosso_país	84	85	71	all
87	presidente_senhoras_e	81	87	79	all
88	meu_pronunciamento_seja	81	87	79	all
89	o_papa_francisco	81	87	39	all
90	a_oportunidade_de	80	90	59	all
91	como_líder_sem	80	90	80	all
92	líder_sem_revisão	80	90	80	all
93	estamos_falando_de	80	90	55	all
94	para_que_possamos	79	94	69	all
95	o_governo_federal	79	94	65	all
96	presidente_eu_quero	79	94	78	all
97	o_nosso_país	79	94	68	all
98	o_que_nós	79	94	66	all

99	da_oradora_sr	79	94	79	all
100	oradora_sr_presidente	79	94	79	all
101	do_governo_federal	77	101	67	all
102	nós_não_podemos	77	101	63	all
103	para_o_brasil	77	101	65	all
104	o_governo_do	76	104	63	all
105	de_são_paulo	76	104	51	all
106	rj_sem_revisão	76	104	76	all
107	para_o_nosso	75	107	65	all
108	todo_o_brasil	73	108	57	all
109	um_projeto_de	72	109	60	all
110	em_todos_os	71	110	66	all
111	do_rio_grande	71	110	47	all
112	de_que_o	71	110	65	all
113	bloco_pl_rj	71	110	56	all
114	presidente_da_república	70	114	53	all
115	que_o_presidente	70	114	64	all
116	estado_do_rio	70	114	37	all
117	essa_é_a	69	117	59	all
118	que_nós_temos	69	117	59	all
119	sr_presidente_srs	69	117	66	all
120	é_por_isso	68	120	58	all
121	por_isso_que	68	120	58	all
122	no_estado_do	68	120	55	all
123	sp_sem_revisão	68	120	68	all
124	que_está_acontecendo	68	120	61	all
125	sr_presidente_o	68	120	66	all
126	o_presidente_hugo	67	126	43	all
127	em_nosso_país	67	126	52	all
128	nos_meios_de	67	126	63	all
129	era_o_que	66	129	66	all
130	presidente_charles_fernandes	66	129	60	all
131	o_que_o	66	129	61	all
132	o_governo_lula	66	129	48	all
133	a_segurança_pública	66	129	37	all
134	tinha_a_dizer	65	134	65	all
135	que_esta_casa	65	134	59	all
136	de_comunicação_da	65	134	64	all
137	certeza_de_que	65	134	62	all
138	bloco_pl_rs	65	134	47	all
139	a_polícia_federal	65	134	44	all
140	que_meu_pronunciamento	65	134	65	all
141	desligamento_do_microfone	64	141	57	all

142	quero_dizer_que	64	141	63	all
143	ba_pela_ordem	64	141	64	all
144	sr_presidente_a	64	141	61	all
145	da_comissão_de	63	145	42	all
146	lei_aldir_blanc	63	145	44	all
147	pessoas_com_deficiência	63	145	37	all
148	eu_gostaria_de	62	148	56	all
149	rs_sem_revisão	62	148	62	all
150	orador_obrigado_sr	62	148	62	all
151	a_v.exa_que	61	151	59	all
152	tem_que_ser	61	151	51	all
153	governo_do_presidente	61	151	49	all
154	pec_da_segurança	61	151	41	all
155	esse_é_o	60	155	55	all
156	para_o_povo	60	155	49	all
157	isso_é_um	60	155	56	all
158	muito_obrigado_presidente	60	155	56	all
159	é_um_absurdo	60	155	53	all
160	compromisso_com_a	60	155	53	all
161	em_todo_o	60	155	54	all
162	da_oradora_presidente	59	162	59	all
163	é_isso_que	59	162	51	all
164	o_presidente_da	59	162	52	all
165	questão_de_ordem	59	162	23	all
166	que_tinha_a	58	166	58	all
167	esta_tribuna_para	58	166	56	all
168	na_câmara_dos	58	166	54	all
169	o_que_tinha	57	169	57	all
170	venho_a_esta	57	169	57	all
171	sp_pela_ordem	56	171	56	all
172	fazer_com_que	56	171	44	all
173	pronunciamento_seja_divulgado	56	171	56	all
174	golpe_de_estado	56	171	34	all
175	do_papa_francisco	56	171	35	all
176	é_uma_vergonha	55	176	41	all
177	das_pessoas_com	55	176	29	all
178	todos_os_dias	55	176	47	all
179	eu_quero_dizer	55	176	50	all
180	é_preciso_que	55	176	41	all
181	de_reais_para	54	181	42	all
182	não_é_uma	54	181	48	all
183	e_o_que	54	181	50	all
184	de_todo_o	54	181	41	all

185	presidente_srs_deputados	54	181	51	all
186	bloco_psol_rj	54	181	52	all
187	registro_taquigráfico_senhor	53	187	53	all
188	taquigráfico_senhor_presidente	53	187	53	all
189	não_pode_ser	53	187	47	all
190	é_muito_importante	53	187	47	all
191	dia_de_hoje	52	191	46	all
192	esse_tipo_de	52	191	48	all
193	os_estados_unidos	52	191	31	all
194	subo_a_esta	52	191	51	all
195	que_é_um	52	191	50	all
196	oficiais_de_justiça	52	191	25	all
197	a_ordem_do	52	191	32	all
198	na_cidade_de	52	191	41	all
199	que_não_é	52	191	48	all
200	no_conselho_de	52	191	32	all

* É possível passar um vetor de números para o tamanho do n-grama. Ex: 2:4 geraria bigramas, trigramas e quadrigramas.

Vantagens de utilizar n-gramas:

- Capturam expressões e frases comuns que podem ter significados específicos.
- Adicionam mais contexto ao modelo, o que pode melhorar a precisão em tarefas como classificação de texto ou análise de sentimentos.
- Identificam negações, como “não gosto”, que podem alterar o significado de uma frase.

Desvantagens de utilizar n-gramas:

- Há um retorno decrescente. Como há muitas combinações possíveis de palavras, os dados podem ficar esparsos (muitos casos raros).
- Rigidez estrutural e semântica. “Devemos focar na economia” e “Precisamos focar na economia” têm o mesmo significado, mas são quadrigramas distintos.

3.2.1 Skip-grams: ngramas com saltos

Outra limitação dos n-gramas é que as palavras sejam adjacentes. A linguagem é rica em intercalações e modalizações.

Skip-grams permitem “pular” palavras intermediárias para capturar contexto com relações estruturais e semânticas de longa distância.

```

skipgramas_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE) |>
  tokens_remove(stopwords("portuguese"), padding = TRUE) |>
  tokens_ngrams(2, skip = 0:6)

skipgramas_discursos |>
  tokens_select("*lula*") |>
  dfm() |>
  textstat_frequency(n = 100)

```

	feature	frequency	rank	docfreq	group
1	presidente_lula	296	1	164	all
2	governo_lula	135	2	90	all
3	luiz_lula	27	3	23	all
4	inácio_lula	27	3	23	all
5	lula_é	22	5	19	all
6	sr_lula	21	6	18	all
7	descondenado_lula	13	7	9	all
8	desgoverno_lula	12	8	11	all
9	lula_faz	11	9	9	all
10	lula_vai	9	10	8	all
11	lula_mandou	8	11	7	all
12	lula_hoje	7	12	5	all
13	lula_presidente	7	12	7	all
14	lula_agora	6	14	6	all
15	lula_voltou	6	14	6	all
16	povo_lula	6	14	6	all
17	acorda_lula	6	14	3	all
18	lula_então	5	18	5	all
19	é_lula	5	18	4	all
20	lula_sobre	4	20	4	all
21	brasileiro_lula	4	20	4	all
22	lula_nunca	4	20	3	all
23	lula_sempre	4	20	4	all
24	lula_porque	4	20	4	all
25	lula_fez	4	20	4	all
26	lula_sr	4	20	4	all
27	lula_aqui	4	20	4	all
28	lula_vamos	4	20	3	all
29	substituir_lula	4	20	2	all
30	celular_roubado	3	30	2	all
31	lula_dá	3	30	3	all

32	lula_morresse	3	30	3	all
33	lula_deu	3	30	3	all
34	brasil_lula	3	30	2	all
35	parabéns_lula	3	30	3	all
36	lula_conseguiu	3	30	3	all
37	lula_vem	3	30	3	all
38	lula_ainda	3	30	3	all
39	esperar_lula	3	30	3	all
40	dnit_lula	3	30	3	all
41	celular_anunciada	3	30	3	all
42	celular_parecem	3	30	3	all
43	justiça_lula	3	30	3	all
44	lula_onde	3	30	3	all
45	terceiro_lula	3	30	3	all
46	gestão_lula	3	30	3	all
47	lula_pode	3	30	3	all
48	lula_ter	3	30	3	all
49	reeleição_lula	3	30	3	all
50	lula_transforma	3	30	3	all
51	plenário_lula	3	30	1	all
52	lula_todos	3	30	3	all
53	lula_assume	3	30	3	all
54	próprio_lula	3	30	3	all
55	lula_atenção	3	30	3	all
56	lula_atrás	3	30	3	all
57	lula_contra	3	30	3	all
58	lula_apanas	3	30	3	all
59	lula_iria	3	30	1	all
60	instituto_lula	3	30	2	all
61	lula_continua	2	61	1	all
62	lula_vez	2	61	2	all
63	neste_lula	2	61	2	all
64	segurança_lula	2	61	2	all
65	bem_lula	2	61	2	all
66	lula_governo	2	61	2	all
67	lula_ganhou	2	61	2	all
68	lula_através	2	61	2	all
69	vai_lula	2	61	2	all
70	lula_olhando	2	61	2	all
71	lula_quer	2	61	2	all
72	lula_fazem	2	61	2	all
73	lula_colocar	2	61	2	all
74	lula_adora	2	61	2	all

75	vez_lula	2	61	2	all
76	deste_lula	2	61	2	all
77	celulares_apreendidos	2	61	1	all
78	lula_apostam	2	61	2	all
79	país_lula	2	61	2	all
80	lula_retire	2	61	2	all
81	lula_grande	2	61	2	all
82	lula_operação	2	61	2	all
83	lula_sim	2	61	2	all
84	lula_perderia	2	61	2	all
85	lula_lá	2	61	2	all
86	pública_lula	2	61	2	all
87	celular_porque	2	61	1	all
88	celular_hoje	2	61	2	all
89	celular_é	2	61	1	all
90	porque_lula	2	61	2	all
91	aparelhos_celulares	2	61	2	all
92	lula_lula	2	61	2	all
93	elegeu_lula	2	61	2	all
94	lula_fazer	2	61	2	all
95	lula_nenhum	2	61	2	all
96	lula_saiu	2	61	2	all
97	lula_gosta	2	61	2	all
98	jair_lula	2	61	2	all
99	bolsonaro_lula	2	61	2	all
100	lula_ganha	2	61	2	all

- `padding = TRUE` faz com que o `tokens_remove` deixe espaços vazios no lugar de stopwords. Isso é interessante para manter a adjacência das palavras.

3.2.2 Matriz de Co-orrência de Características (Feature Co-occurrence Matrix, FCM)

Uma FCM é uma matriz que representa a frequência com que diferentes características (palavras, n-gramas, etc) coocorrem em um conjunto de documentos.

Acmatriz possui palavras como linhas e colunas, e cada célula contém a contagem de vezes que as palavras correspondentes coocorrem em um contexto específico (por exemplo, em um mesmo documento ou em uma janela de palavras).

```
fcm_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_symbols = TRUE, remove_numbers = TRUE) |>
```



```
tokens_remove(c(stopwords("portuguese"), "é", "porque", "aqui")) |>
dfm() |>
fcm()

fcm_discursos
```

Feature co-occurrence matrix of: 23,779 by 23,779 features.

	features						
features	discurso	íntegra	encaminhado	sr	deputado	acácio	favacho
discurso	110	203	212	626	408	2	2
íntegra	0	0	131	197	146	2	2
encaminhado	0	0	5	236	177	2	2
sr	0	0	0	3638	3774	5	5
deputado	0	0	0	0	3146	4	4
acácio	0	0	0	0	0	0	3
favacho	0	0	0	0	0	0	0
registro	0	0	0	0	0	0	0
taquigráfico	0	0	0	0	0	0	0
senhor	0	0	0	0	0	0	0

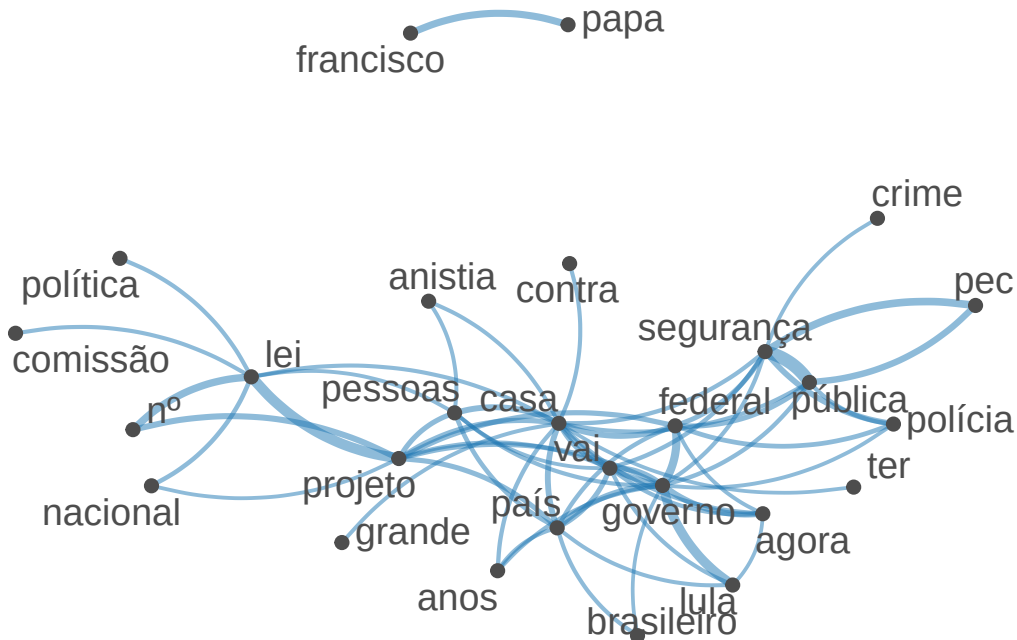
	features		
features	registro	taquigráfico	senhor
discurso	248	203	194
íntegra	147	130	118
encaminhado	150	131	120
sr	569	195	505
deputado	326	145	270
acácio	2	2	2
favacho	2	2	2
registro	57	147	136
taquigráfico	0	0	118
senhor	0	0	533

[reached max_nfeat ... 23,769 more features, reached max_nfeat ... 23,769 more features]

FCMs podem ser utilizadas para construção de *redes semânticas*, onde as palavras são representadas como nós e as coocorrências como arestas.

```
palavras_com_mais_coocorrencia <- fcm_discursos |>
colSums() |> # somar colunas
sort(decreasing = TRUE) |> # ordenar em ordem decrescente
head(50) |> # pegar as 50 palavras com mais coocorrências
names() # pegar os nomes das palavras (colunas)
```

```
fcm_discursos |>
  fcm_select(palavras_com_mais_coocorrencia) |>
  textplot_network(min_freq = 0.95, edge_alpha = 0.5)
```



Isso pode ajudar a identificar clusters de palavras relacionadas e a visualizar as relações entre diferentes termos em um corpus.

3.3 Collocations

Collocations são combinações de palavras que coocorrem com mais frequência do que seria esperado por acaso. Por exemplo, “distrito federal” é uma collocation comum em português, enquanto “distrito superior” não é.

Intuição:

- Em um corpus com 1000 palavras
- “distrito” aparece 50 vezes
- “federal” aparece 30 vezes
- “distrito federal” aparece 20 vezes
- A frequência esperada de “distrito federal” por acaso seria $(50/1000) * (30/1000) = 1.5$ vezes. Portanto, “distrito federal” é uma collocation, pois ocorre muito mais frequentemente do que o esperado por acaso.

3.3.1 Exemplo

```
collocations_discursos <- corpus_discursos |>
  tokens() |>
  tokens_remove(stopwords("portuguese"), padding = TRUE) |>
  textstat_collocations()

collocations_discursos |>
  head(100)
```

	collocation	count	count_nested	length	lambda	z
1	povo brasileiro	317	0	2	7.094949	74.87908
2	segurança pública	341	0	2	7.951203	73.03078
3	presidente lula	287	0	2	5.740204	69.40006
4	desta casa	336	0	2	6.789815	68.69073
5	nesta casa	264	0	2	6.369823	63.98644
6	neste momento	170	0	2	6.687064	60.17856
7	rio grande	174	0	2	6.386154	59.20216
8	lei nº	186	0	2	7.194025	58.26029
9	governo federal	203	0	2	4.866946	56.99339
10	muitas vezes	142	0	2	8.106255	55.50786
11	mil reais	137	0	2	6.296415	54.01904
12	polícia federal	153	0	2	5.842485	53.78756
13	congresso nacional	157	0	2	7.822798	51.41192
14	papa francisco	185	0	2	10.135354	50.74096
15	comunicação desta	114	0	2	7.346900	49.01191
16	políticas públicas	115	0	2	9.092849	48.53168
17	quero dizer	134	0	2	4.997685	48.17867
18	supremo tribunal	133	0	2	9.824673	47.41521
19	além disso	83	0	2	7.357538	46.72047
20	meio ambiente	103	0	2	8.443401	46.38697
21	governo lula	134	0	2	4.676415	46.35016
22	cada vez	95	0	2	5.774423	45.66492
23	todo mundo	97	0	2	5.648577	45.61196
24	tribunal federal	129	0	2	6.920726	45.14859
25	senhor presidente	113	0	2	5.929870	44.58059
26	extrema direita	77	0	2	8.661933	43.40683
27	polícia militar	72	0	2	7.055629	41.95126
28	salário mínimo	77	0	2	9.513562	41.60995
29	ano passado	64	0	2	7.736294	41.58084
30	população brasileira	71	0	2	6.118490	41.54126

31	deputado glauber	205	0	2	7.872754	41.18085
32	presidente hugo	145	0	2	7.229659	40.87213
33	pode ser	102	0	2	4.621680	40.46531
34	colegas parlamentares	62	0	2	6.544027	40.29669
35	deve ser	89	0	2	5.729375	40.13056
36	boa tarde	58	0	2	8.403893	39.26370
37	poder judiciário	56	0	2	6.474898	38.53665
38	vai ser	109	0	2	4.068184	38.22506
39	15 bilhões	56	0	2	7.477119	38.17092
40	espírito santo	76	0	2	10.408624	38.09059
41	2 anos	67	0	2	6.128967	38.05212
42	5 mil	58	0	2	7.112991	38.01413
43	falar sobre	71	0	2	5.289190	37.99832
44	6 bilhões	53	0	2	7.124557	37.97072
45	precisa ser	83	0	2	4.849300	37.64630
46	deste país	78	0	2	5.036228	37.56653
47	ministro alexandre	57	0	2	7.123284	37.47020
48	tão importante	62	0	2	5.706505	37.18533
49	polícia civil	53	0	2	6.718288	36.82482
50	glauber braga	111	0	2	10.522887	36.13865
51	é importante	175	0	2	3.266139	36.10948
52	neste país	81	0	2	4.445070	35.66245
53	redes sociais	66	0	2	9.859315	35.59447
54	senhores deputados	56	0	2	5.806192	35.50307
55	demais órgãos	39	0	2	7.711377	35.16803
56	frente parlamentar	42	0	2	6.888598	34.93247
57	sistema único	45	0	2	8.415360	34.80622
58	senhores parlamentares	46	0	2	6.192452	34.78971
59	quero agradecer	73	0	2	6.588346	34.66563
60	desenvolvimento sustentável	38	0	2	7.657122	33.44210
61	dinheiro público	45	0	2	5.780509	33.43958
62	sociedade brasileira	45	0	2	5.902238	33.40299
63	ministério público	41	0	2	6.155793	33.24163
64	1 ano	44	0	2	5.867420	33.18068
65	política pública	58	0	2	4.858917	33.01597
66	banco central	42	0	2	10.042825	32.87285
67	lei aldir	63	0	2	7.321572	32.70117
68	justiça social	47	0	2	5.456960	32.67941
69	presidente bolsonaro	71	0	2	4.314372	32.53561
70	ter sido	47	0	2	5.662171	32.44850
71	bom dia	52	0	2	5.330855	32.44725
72	colegas deputados	48	0	2	5.478509	32.41811
73	povos indígenas	42	0	2	9.676011	32.41243

74	jair bolsonaro	50	0	2	7.589680	32.30223
75	dia mundial	49	0	2	6.439371	32.29408
76	quero parabenizar	69	0	2	6.886609	32.05209
77	dia 8	47	0	2	5.571043	31.90361
78	gilberto silva	36	0	2	9.030900	31.83257
79	desenvolvimento econômico	33	0	2	7.448857	31.66985
80	boa noite	32	0	2	7.733865	31.45592
81	governo bolsonaro	64	0	2	4.324421	31.23625
82	igreja católica	38	0	2	9.468493	30.94283
83	nesse sentido	30	0	2	6.893765	30.77128
84	tribuna hoje	51	0	2	4.885489	30.71107
85	estados unidos	136	0	2	10.558781	30.67852
86	deus abençoe	34	0	2	8.219282	30.66900
87	poderia deixar	29	0	2	7.079134	30.54690
88	deputado federal	84	0	2	3.560410	30.33904
89	outro lado	31	0	2	6.242143	29.85701
90	primeiro lugar	27	0	2	6.981984	29.75911
91	seguinte discurso	42	0	2	9.225202	29.75177
92	1 milhão	40	0	2	8.642871	29.74764
93	constituição federal	46	0	2	5.113845	29.57232
94	tantos outros	31	0	2	7.302873	29.53508
95	poder público	37	0	2	5.450337	29.53091
96	democracia brasileira	35	0	2	5.783407	29.49473
97	é preciso	188	0	2	6.121146	29.45537
98	conscientização sobre	43	0	2	6.785642	29.33319
99	deveria ser	46	0	2	5.311634	29.10275
100	presidente charles	64	0	2	6.857871	29.09785

Além das collocations, a tabela gerada retorna:

- **count**: a contagem de ocorrências
- **count_nested**: a contagem de ocorrências com tamanho menor
- **length**: o tamanho da collocations em número de palavras
- **lambda**: indica o quão fortemente as palavras da colocação estão relacionadas
- **z**: estatística do valor **lambda**, corrigido de acordo com o erro padrão (quanto maior **z**, maior a confiança em **lambda**)

3.4 TF-IDF

Até agora, falamos sobre a frequência de palavras em um texto (TF, do inglês *term-frequency*).

Em certas análises, é importante considerar o quão comum ou rara uma palavra é em um corpus.

Por exemplo, a palavra “presidente” é mencionada por todos os discursos. Porém, palavras como “macroeconomia” e “transfobia” são particulares de determinados partidos e ideologias.

TF-IDF (Term Frequency-Inverse Document Frequency) é uma métrica de importância de termos que considera:

- TF (Term Frequency): A frequência de uma palavra (ou n-grama) em um documento.

Por exemplo, se a palavra “sonho” aparece 5 vezes em um documento com 100 palavras, a TF seria $5/100 = 0.05$.

- IDF (Inverse Document Frequency): Uma medida de quão comum ou rara uma palavra é em um corpus.

É calculada como $\log(N/n)$, onde N é o número total de documentos e n é o número de documentos que contêm a palavra.

Por exemplo, se temos 1000 documentos e a palavra “sonho” aparece em 100 deles, a IDF seria $\log(1000/100) = \log(10) = 1$.

O TF-IDF é calculado multiplicando a TF pela IDF. No exemplo acima, o TF-IDF para a palavra “sonho” seria $0.05 * 1 = 0.05$.

3.4.1 Exemplo

Vamos utilizar TF-IDF para comparar discursos de três parlamentares: Sóstenes Cavalcante (PL), Lindbergh Farias (PT) e Antonio Brito (PSD).

```
tokens_nomes <- docvars(corpus_discursos) |> pull(nome) |> tokens()

tfidf_discursos <- corpus_discursos |>
  corpus_subset(nome %in% c("Sóstenes Cavalcante", "Lindbergh Farias",
                           "Antonio Brito")) |>
  corpus_group(nome) |>
  tokens(remove_punct = T) |>
  tokens_remove(tokens_nomes) |>
  dfm() |>
  dfm_tfidf()

tfidf_discursos
```

Document-feature matrix of: 3 documents, 2,434 features (56.51% sparse) and 15 docvars.

```

      features
docs   o sr bloco      psd      ba inicialmente eu queria
Antonio Brito      0 0      0 2.8174601 0.3521825      0.1760913 0      0
Lindbergh Farias  0 0      0 0      0      0      0      0      0
Sóstenes Cavalcante 0 0      0 0.3521825 0.3521825      0.7043650 0      0
      features
docs   saudar v.exa
Antonio Brito      1.908485      0
Lindbergh Farias  0      0
Sóstenes Cavalcante 0      0
[ reached max_nfeat ... 2,424 more features ]

```

Para facilitar a análise, vamos transformar a DFM em tabela:

```

df_tfidf <- tfidf_discursos |>
  convert(to = "data.frame") |> # converter em tabela
  pivot_longer(                 # transformar colunas (palavras) em linhas
    cols = -doc_id,
    names_to = "termo",
    values_to = "tfidf"
  )

head(df_tfidf)

```

```

# A tibble: 6 x 3
  doc_id      termo      tfidf
  <chr>      <chr>      <dbl>
1 Antonio Brito o      0
2 Antonio Brito sr      0
3 Antonio Brito bloco    0
4 Antonio Brito psd      2.82
5 Antonio Brito ba      0.352
6 Antonio Brito inicialmente 0.176

```

Vamos extrair os 10 termos com maior TF-IDF para cada deputado:

```

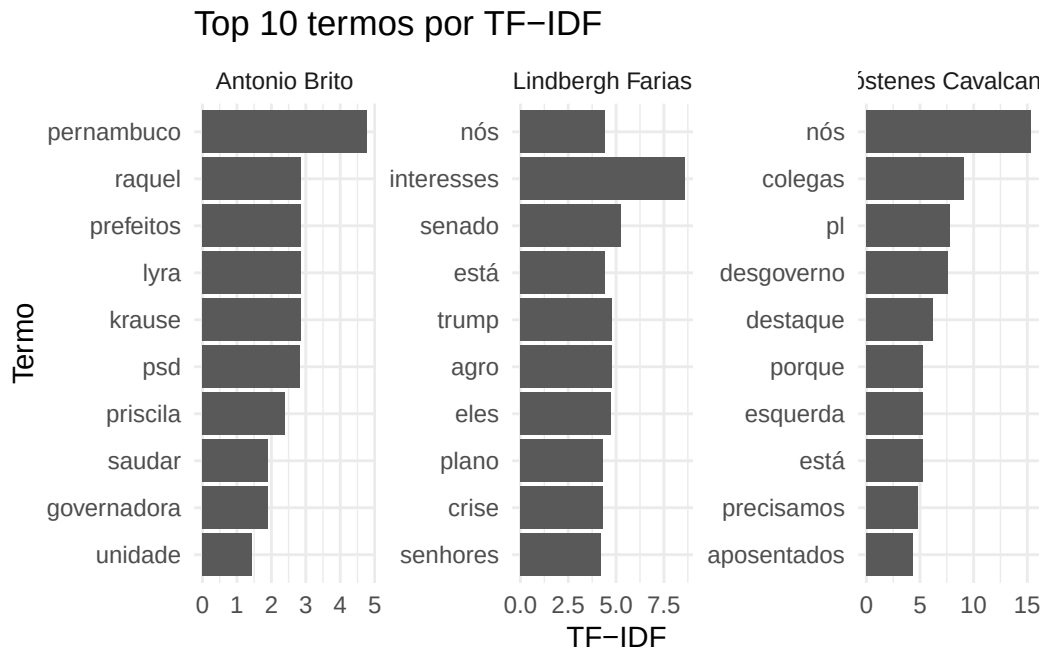
df_tfidf |>
  group_by(doc_id) |>
  slice_max(tfidf, n = 10)

```

```
# A tibble: 38 x 3
# Groups:   doc_id [3]
  doc_id      termo      tfidf
  <chr>      <chr>      <dbl>
1 Antonio Brito pernambuco  4.77
2 Antonio Brito krause      2.86
3 Antonio Brito prefeitos   2.86
4 Antonio Brito raquel      2.86
5 Antonio Brito lyra        2.86
6 Antonio Brito psd         2.82
7 Antonio Brito priscila    2.39
8 Antonio Brito saudar      1.91
9 Antonio Brito governadora 1.91
10 Antonio Brito unidade    1.43
# i 28 more rows
```

E plotar os termos mais importantes para cada deputado em um gráfico de barras:

```
df_tfidf |>
  group_by(doc_id) |>
  slice_max(tfidf, n = 10, with_ties = F) |>
  ggplot(aes(x = reorder(termo, tfidf), y = tfidf)) +
  geom_bar(stat = "identity") +
  facet_wrap(~ doc_id, scales = "free") +
  coord_flip() +
  labs(x = "Termo", y = "TF-IDF", title = "Top 10 termos por TF-IDF") +
  theme_minimal()
```

3.5 Diversidade léxica (lexical diversity)

A diversidade léxica é uma medida da variedade de palavras utilizadas em um texto.

Uma medida comum de diversidade léxica é o Type-Token Ratio (TTR), que é a razão entre o número de tipos (palavras únicas) e o número de tokens (total de palavras) em um texto.

```
corpus_discursos |>
  tokens() |>
  textstat_lexdiv() |> # calcula a diversidade léxica
  arrange(desc(TTR)) |> # ordena por Type-Token Ratio (TTR) em ordem decrescente
  head(20)
```

	document	TTR
1	1431_Tarcísio Motta_PSOL	0.9487179
2	827_Laura Carneiro_PSD	0.9473684
3	1057_Nilto Tatto_PT	0.9361702
4	649_Helena Lima_MDB	0.9275362
5	54_Alencar Santana_PT	0.9250000
6	436_Dr. Victor Linhalis_PODE	0.9189189
7	1109_Pauderney Avelino_UNIÃO	0.9189189

```

8      719_Jandira Feghali_PCdoB 0.9122807
9      1139_Pedro Campos_PSB 0.9117647
10     1058_Nilto Tatto_PT 0.9076923
11     826_Laura Carneiro_PSD 0.9069767
12     1326_Sargento Fahur_PL 0.8974359
13 1114_Pauderney Avelino_UNIÃO 0.8840580
14     131_Bandeira de Mello_PSB 0.8809524
15     646_Helder Salomão_PT 0.8809524
16    1430_Tarcísio Motta_PSOL 0.8809524
17     1407_Tadeu Veneri_PT 0.8666667
18     786_José Medeiros_PL 0.8648649
19     671_Hildo Rocha_MDB 0.8636364
20 1112_Pauderney Avelino_UNIÃO 0.8611111

```

O TTR é uma medida simples, mas pode ser influenciada pelo tamanho do texto. Textos mais longos tendem a ter um TTR mais baixo, pois é mais provável que palavras sejam repetidas.

Existem outras métricas de diversidade léxica que tentam corrigir essa influência do tamanho do texto, como o MTLT (Measure of Textual Lexical Diversity).

A função `textstat_lexdiv` possui várias outras métricas implementadas. Podemos encontrá-las na documentação do parâmetro “measure” em https://quanteda.io/reference/textstat_lexdiv.html.

3.6 Leiturabilidade (readability)

Assim como a diversidade léxica, a leiturabilidade é uma medida da complexidade de um texto. Existem várias fórmulas para calcular a leiturabilidade, como o Índice de Flesch, o Índice de Gunning Fog, o Índice de Coleman-Liau, entre outras.

O Índice de Flesch calcula um valor de 0 a 100 para a facilidade de leitura. A métrica considera dois fatores:

- o número de palavras por frase
- o número de sílabas por palavra

Na biblioteca `quanteda`, utilizamos a função `textstat_readability`, que por padrão retorna o Índice de Flesch:

```

corpus_discursos |>
  textstat_readability() |>
  arrange(desc(Flesch)) |>
  head(20)

```

	document	Flesch
1	1434_Tarcísio Motta_PSOL	58.14795
2	161_Bibo Nunes_PL	54.53661
3	1024_Max Lemos_PDT	51.86750
4	498_Enfermeira Rejane_PCdoB	50.32569
5	424_Dorinaldo Malafaia_PDT	49.26133
6	162_Bibo Nunes_PL	47.45642
7	157_Bibo Nunes_PL	46.86574
8	1451_Tenente Coronel Zucco_PL	46.18336
9	1525_Zucco_PL	46.18336
10	1153_Pompeo de Mattos_PDT	45.56941
11	1446_Tenente Coronel Zucco_PL	45.34269
12	1520_Zucco_PL	45.34269
13	595_Gilson Marques_NOVO	45.25167
14	779_José Medeiros_PL	45.25150
15	149_Bia Kicis_PL	44.72219
16	706_Ivan Valente_PSOL	44.50222
17	350_Dandara_PT	44.13721
18	588_Gilson Marques_NOVO	43.94362
19	158_Bibo Nunes_PL	43.92245
20	160_Bibo Nunes_PL	43.80917

Métricas de leituraabilidade podem ser úteis para comparar a complexidade de discursos de diferentes parlamentares, partidos ou temas. Por exemplo, podemos comparar a leituraabilidade de discursos sobre economia e saúde para verificar se há diferenças na complexidade dos textos.

Outras métricas de leituraabilidade disponíveis na função `textstat_readability` estão descritas na documentação da função: https://quanteda.io/reference/textstat_readability.html.

3.7 Palavras-chave (Keyness)

Em estudos quantitativos, *palavras-chaves* (*keyness*) do texto são aquelas quais a frequência é significativamente diferente entre grupos de documentos.

Por exemplo, quais palavras são mais associadas a discursos do PT em comparação com outros partidos?

Primeiro, separamos os discursos em grupos: discursos do PT (alvo) e discursos de outros partidos (referência).

Em seguida, a função `textstat_keyness` analisa as frequências de uma DFM em cada grupo e compara, utilizando um teste estatístico, se a frequência é maior no grupo alvo. Quanto maior a certeza do teste, maior a tendência de palavra-chave.

```
partidos <- docvars(corpus_discursos) |> pull(partido)

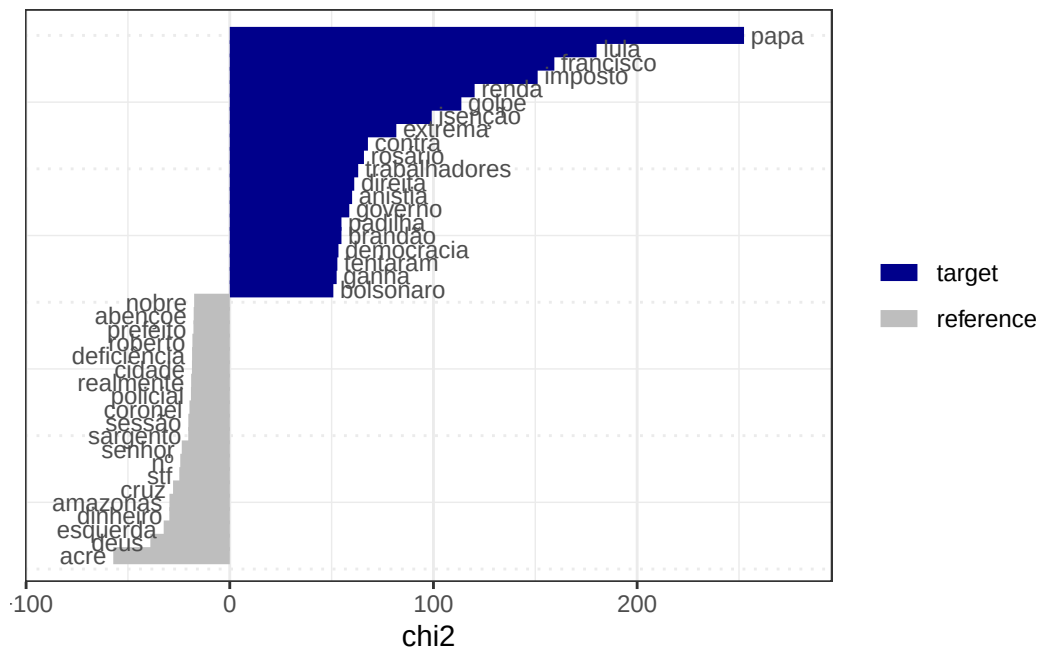
keyness_pt <- corpus_discursos |>
  tokens() |>
  tokens_tolower() |>
  tokens_remove(c(stopwords("portuguese"), partidos)) |>
  dfm() |>
  textstat_keyness(target = partidos == 'PT')

keyness_pt |> head(n = 20)
```

	feature	chi2	p	n_target	n_reference
1	papa	252.47581	0.000000e+00	168	98
2	lula	180.07901	0.000000e+00	336	457
3	francisco	159.35871	0.000000e+00	150	127
4	imposto	151.15490	0.000000e+00	90	45
5	renda	120.21576	0.000000e+00	126	117
6	golpe	113.70039	0.000000e+00	117	107
7	isenção	99.15517	0.000000e+00	51	20
8	extrema	81.76862	0.000000e+00	75	62
9	contra	67.82648	2.220446e-16	244	441
10	rosário	65.87742	4.440892e-16	22	2
11	trabalhadores	63.05585	1.998401e-15	105	134
12	direita	61.11905	5.329071e-15	86	99
13	anistia	60.06994	9.103829e-15	222	405
14	governo	58.68077	1.854072e-14	414	910
15	brandão	54.82755	1.315614e-13	20	3
16	padilha	54.82755	1.315614e-13	20	3
17	democracia	53.36180	2.774447e-13	133	208
18	tentaram	52.82306	3.650413e-13	37	23
19	ganha	52.46955	4.369838e-13	47	38
20	bolsonaro	50.85765	9.930945e-13	167	292

Podemos plotar as palavras-chave com maiores e menores keyness com a função `textplot_keyness`:

```
keyness_pt |>
  textplot_keyness(labelsizes = 3)
```



3.8 Para saber mais

Leia mais sobre TF-IDF e N-gramas nos capítulos 3 e 4 do livro Text Mining with R (Silge and Robinson 2017).

Veja como collocations são utilizadas para estudos sociais no artigos:

- A useful methodological synergy? Combining critical discourse analysis and corpus linguistics to examine discourses of refugees and asylum seekers in the UK press de Baker et al. (2008).
- Indicador de similaridade do discurso parlamentar: análise do comportamento das coalizões partidárias de Schwartz (2022).

3.9 Estudos guiados

3.9.1 Análise de temas com dicionários

```
library(tidyverse)
library(quantda)
library(quantda.textstats)

nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados e criar corpus
tabela_discursos <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_")) |>
  corpus(text_field = "transcricao")

# Criar tokens
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE) |>
  tokens_tolower()

# Criar um dicionário temático (ex: Saúde e Educação)
meu_dicionario <- dictionary(list(
  saude = c("saúde", "hospital", "hospitais", "médico", "médicos", "enfermeiro", "sus"),
  educacao = c("educação", "escola", "escolas", "professor", "professores", "ensino", "univer")
))

# Aplicar dicionário e criar DFM para contar as menções de cada tema
dfm_temas <- tokens_discursos |>
  tokens_lookup(dictionary = meu_dicionario) |>
  dfm()

# Contar a menção de cada termo
tabela_contagem_temas <- dfm_temas |>
  textstat_frequency()

# Imprimir tabela
print(tabela_contagem_temas)
```

	feature	frequency	rank	docfreq	group
1	saude	908	1	309	all
2	educacao	736	2	258	all

3.9.2 Gerando n-gramas (sequências de palavras)

```
library(tidyverse)
library(quanteda)
library(quanteda.textstats)

nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados e criar corpus
tabela_discursos <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_")) |>
  corpus(text_field = "transcricao")

# Gerar trigramas (sequências de 3 palavras contíguas)
tokens_trigramas <- corpus_discursos |>
  tokens(remove_punct = TRUE) |>
  tokens_tolower() |>
  tokens_ngrams(n = 3)

# Calcular a frequência dos trigramas mais comuns
dfm_trigramas <- dfm(tokens_trigramas)

# Criar tabela com os 10 trigramas mais frequentes
freq_trigramas <- dfm_trigramas |>
  textstat_frequency(n = 10)

# Imprimir tabela
print(freq_trigramas)
```

	feature	frequency	rank	docfreq	group
1	sem_revisão_do	1103	1	1102	all
2	revisão_do_orador	1103	1	1102	all
3	ordem_sem_revisão	742	3	741	all
4	pela_ordem_sem	728	4	727	all
5	do_orador_sr	448	5	448	all
6	orador_sr_presidente	448	5	448	all
7	projeto_de_lei	330	7	185	all
8	sem_revisão_da	262	8	261	all
9	revisão_da_oradora	262	8	261	all
10	câmara_dos_deputados	224	10	171	all

3.9.3 Skip-grams para capturar relações entre palavras não adjacentes

```
library(tidyverse)
library(quanteda)
library(quanteda.textstats)
library(stopwords)

nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados e criar corpus
tabela_discursos <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_")) |>
  corpus(text_field = "transcricao")

# Gerar skip-grams (bi-gramas que podem pular até 2 palavras entre si) contendo a palavra "f
tokens_skip <- corpus_discursos |>
  tokens(remove_punct = TRUE) |>
  tokens_tolower() |>
  tokens_remove(stopwords("portuguese")) |>
  tokens_ngrams(n = 2, skip = 0:2) |>
  tokens_select("*francisco*")

# Criar DFM com skip-grams
dfm_skipgrams <- tokens_skip |>
  dfm()

# Criar tabela com frequências de skipgramas
freq_skip <- dfm_skipgrams |>
  textstat_frequency(10)

print(freq_skip)
```

	feature	frequency	rank	docfreq	group
1	papa_francisco	196	1	62	all
2	francisco_papa	23	2	17	all
3	francisco_é	20	3	15	all
4	francisco_francisco	13	4	11	all
5	morte_francisco	10	5	8	all
6	francisco_assis	10	5	6	all
7	falecimento_francisco	9	7	8	all
8	é_francisco	9	7	9	all

9	francisco_deixou	9	7	6	all
10	homenagem_francisco	8	10	8	all

3.9.4 Redes de co-ocorrência de palavras (FCM)

```
library(tidyverse)
library(quanteda)
library(quanteda.textplots)
library(stopwords)

nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados e criar corpus
corpus_discursos <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_")) |>
  corpus(text_field = "transcricao")

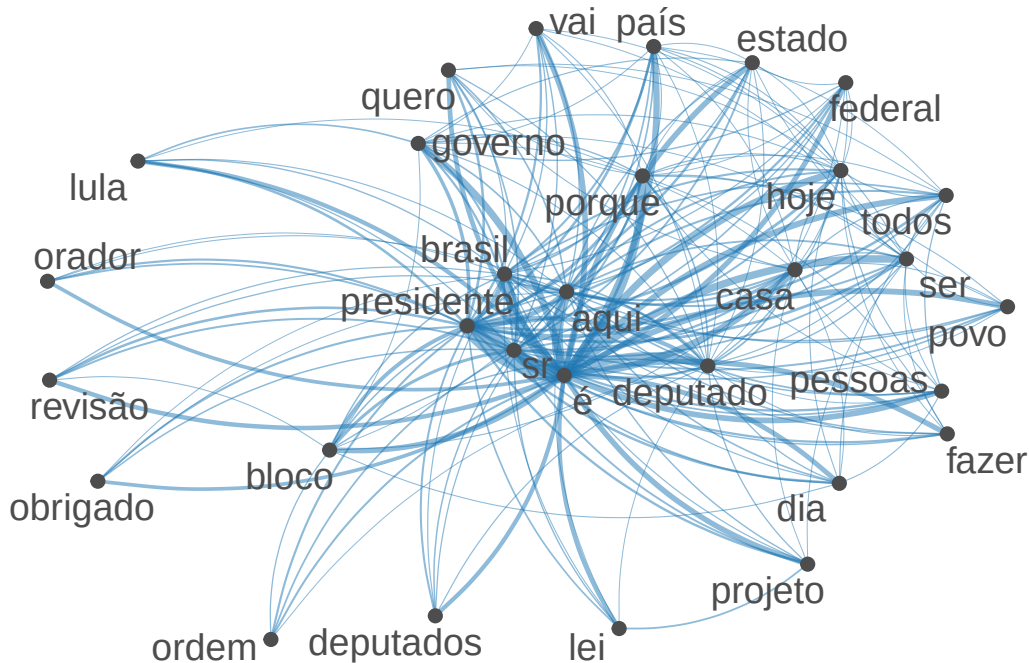
# Criar tokens removendo pontuação e stopwords
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE) |>
  tokens_tolower() |>
  tokens_remove(stopwords("portuguese"))

# Criar DFM
dfm_discursos <- tokens_discursos |>
  dfm()

# Selecionar as 30 palavras mais frequentes para visualizar a rede
palavras_mais_citadas <- dfm_discursos |>
  textstat_frequency(30) |>
  pull(feature)

# Criar FCM (Feature Co-occurrence Matrix)
fcm_discursos <- dfm_discursos |>
  dfm_select(palavras_mais_citadas) |>
  fcm()

# Desenhar rede semântica
textplot_network(fcm_discursos)
```



3.9.5 Identificando Collocations (expressões típicas)

```
library(tidyverse)
library(quanteda)
library(quanteda.textstats)

nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados e criar corpus
corpus_discursos <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_")) |>
  corpus(text_field = "transcricao")

# Criar tokens (filtrando stopwords)
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE) |>
  tokens_tolower() |>
  tokens_remove(stopwords("portuguese"), padding = TRUE)

# Criar tabela de collocations (expressões de duas palavras que ocorrem juntas frequentemente)
collocs <- tokens_discursos |>
```

```
textstat_collocations(size = 2, min_count = 15) |>
head(20) # filtrar 20 primeiras linhas

# Visualizar as principais collocations estatisticamente significativas
print(collocs)
```

	collocation	count	count_nested	length	lambda	z
1	sr presidente	1606	0	2	5.542681	133.13009
2	segurança pública	341	0	2	7.689422	77.82226
3	orador sr	448	0	2	4.951940	77.16461
4	bloco pl	397	0	2	6.687782	76.72034
5	povo brasileiro	317	0	2	6.847499	76.23346
6	desta casa	337	0	2	6.981374	70.66551
7	bloco pt	353	0	2	6.894202	69.95841
8	nesta casa	264	0	2	6.547219	65.95347
9	neste momento	170	0	2	6.862600	61.87973
10	rio grande	174	0	2	6.527853	60.88333
11	presidente lula	287	0	2	4.536320	59.80079
12	lei nº	186	0	2	7.144808	58.77372
13	governo federal	203	0	2	4.850408	57.61622
14	obrigado presidente	266	0	2	4.596799	56.86728
15	muitas vezes	142	0	2	8.287803	56.82921
16	papa francisco	185	0	2	9.516792	56.39894
17	mil reais	137	0	2	6.431336	55.64574
18	polícia federal	153	0	2	5.883158	55.47569
19	congresso nacional	157	0	2	7.726140	55.22283
20	srs deputados	212	0	2	7.871286	54.27577

3.9.6 Calculando TF-IDF para destacar termos distintivos

```
library(tidyverse)
library(quantda)

nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados e criar corpus
corpus_discursos <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_")) |>
  corpus(text_field = "transcricao")
```

```
# Criar tokens (removendo stopwords)
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE) |>
  tokens_tolower() |>
  tokens_remove(stopwords("portuguese"))

# Calcular TF-IDF agrupando os discursos por partido
dfm_tfidf <- tokens_discursos |>
  dfm() |>
  dfm_group(groups = partido) |>
  dfm_tfidf()

# Extrair termos com maior peso TF-IDF para um partido específico (ex: PT)
dfm_tfidf |>
  convert(to = "data.frame") |>
  filter(doc_id == "PT") |>
  pivot_longer(-doc_id, names_to = "termo", values_to = "tfidf") |>
  arrange(desc(tfidf)) |>
  head(10)
```

```
# A tibble: 10 x 3
  doc_id termo      tfidf
  <chr>   <chr>    <dbl>
1 PT     pt        99.4
2 PT     golpe     49.0
3 PT     glauber    45.3
4 PT     anistia     39.1
5 PT     imposto    33.1
6 PT     lula        30.8
7 PT     papa        24.5
8 PT     bolsonaristas 22.5
9 PT     querem      22.4
10 PT    braga       22.1
```

3.9.7 Medindo a diversidade léxica (vocabulário)

```
library(tidyverse)
library(quanteda)
library(quanteda.textstats)
```

```

nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados e criar corpus
corpus_discursos <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_")) |>
  corpus(text_field = "transcricao")

# Criar tabela com TTR (Type-Token Ratio) para cada documento
ttr_discursos <- corpus_discursos |>
  tokens() |>
  textstat_lexdiv(measure = "TTR")

# Criar tabela com 10 maiores TTRs
top_10_ttr_discursos <- ttr_discursos |>
  arrange(desc(TTR)) |> # ordenar por TTR decrescente (utilizar arrange(TTR) crescente)
  head(10)              # primeiras 10 linhas

# Imprimir os 10 discursos com maior TTR
print(top_10_ttr_discursos)

```

	document	TTR
1	1431_Tarcísio Motta_PSOL	0.9487179
2	827_Laura Carneiro_PSD	0.9473684
3	1057_Nilto Tatto_PT	0.9361702
4	649_Helena Lima_MDB	0.9275362
5	54_Alencar Santana_PT	0.9250000
6	436_Dr. Victor Linhalis_PODE	0.9189189
7	1109_Pauderney Avelino_UNIÃO	0.9189189
8	719_Jandira Feghali_PCdoB	0.9122807
9	1139_Pedro Campos_PSB	0.9117647
10	1058_Nilto Tatto_PT	0.9076923

3.9.8 Análise de leituraabilidade (complexidade do texto)

```

library(tidyverse)
library(quanteda)
library(quanteda.textstats)

nome_arquivo <- "discursos_camara_abril_2025.csv"

```

```
# Carregar dados e criar corpus
corpus_discursos <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_")) |>
  corpus(text_field = "transcricao")

# Calcular o índice Flesch de facilidade de leitura
leiturabilidade <- corpus_discursos |>
  textstat_readability(measure = "Flesch")

# Criar tabela com os 10 maiores índices de Flesch
top_10_leiturabilidade <- leituraabilidade |>
  arrange(desc(Flesch)) |>
  head(10)

# Imprimir tabela
print(top_10_leiturabilidade)
```

	document	Flesch
1	1434_Tarcísio Motta_PSOL	58.14795
2	161_Bibo Nunes_PL	54.53661
3	1024_Max Lemos_PDT	51.86750
4	498_Enfermeira Rejane_PCdoB	50.32569
5	424_Dorinaldo Malafaia_PDT	49.26133
6	162_Bibo Nunes_PL	47.45642
7	157_Bibo Nunes_PL	46.86574
8	1451_Tenente Coronel Zucco_PL	46.18336
9	1525_Zucco_PL	46.18336
10	1153_Pompeo de Mattos_PDT	45.56941

3.9.9 Identificando keyness entre grupos

```
library(tidyverse)
library(quanteda)
library(quanteda.textstats)

nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados e criar corpus
corpus_discursos <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_")) |>
```

```

corpus(text_field = "transcricao")

# Criar tokens (removendo stopwords)
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE) |>
  tokens_tolower() |>
  tokens_remove(stopwords("portuguese"))

# Criar DFM agrupada por uma variável (ex: partido)
dfm_partidos <- tokens_discursos |>
  dfm() |>
  dfm_group(groups = partido)

# Comparar o vocabulário de um partido contra todos os outros (Keyness)
# Exemplo comparando o PL contra os demais
keyness_pl <- dfm_partidos |>
  textstat_keyness(target = "PL")

# Criar tabela com as 20 palavras com maior estatística de keyness do PL
top_20_keyness_pl <- keyness_pl |>
  arrange(desc(chi2)) |> # ordenar pela estatística chi-quadrado
  head(20)               # 20 primeiras linhas

# Listar palavras significativamente mais frequentes no grupo alvo
print(top_20_keyness_pl)

```

	feature	chi2	p	n_target	n_reference
1	pl	1119.29289	0.000000e+00	500	134
2	esquerda	195.88254	0.000000e+00	146	93
3	polícia	185.83239	0.000000e+00	220	216
4	stf	180.34922	0.000000e+00	106	49
5	zucco	150.99558	0.000000e+00	51	4
6	policiais	147.26364	0.000000e+00	84	37
7	monteiro	138.08000	0.000000e+00	51	7
8	fahur	110.99378	0.000000e+00	45	9
9	sargento	105.61020	0.000000e+00	64	31
10	policial	99.05324	0.000000e+00	64	34
11	capitão	95.26546	0.000000e+00	30	1
12	segurança	92.27695	0.000000e+00	241	372
13	pai	88.20171	0.000000e+00	68	45
14	anistia	88.03191	0.000000e+00	243	384
15	oposição	86.19807	0.000000e+00	80	64

16	alberto	85.95111	0.000000e+00	36	8
17	minoria	83.60135	0.000000e+00	33	6
18	descondenado	81.90058	0.000000e+00	26	1
19	desgoverno	74.48401	0.000000e+00	41	17
20	assinaturas	69.20840	1.110223e-16	48	28

3.10 Atividade prática

Analise os discursos vistos em aula utilizando:

- Crie um dicionário de termos com temas de seu interesse e analise a frequência nos discursos da câmara
- Busque collocations com 3 ou 4 palavras
- TF-IDF, analisando as diferenças entre um partido de direita e um partido de esquerda
- Compare palavras-chave (keyness) de diferentes gêneros

4 Análise Sintática e Modelos de Tópicos

i Nesta aula

- Análise sintática
- Modelos de tópicos

Link para o Google Colab: <https://drive.google.com/file/d/1RlbTbNNC2JLMvYxenrzCH1Vyw3AR70oy/view?usp=sharing>

```
if (!"tidyverse" %in% installed.packages()) {  
  install.packages("tidyverse")  
}  
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
v dplyr      1.2.0      v readr      2.2.0  
v forcats    1.0.1      v stringr    1.6.0  
v ggplot2     4.0.2      v tibble     3.3.1  
v lubridate  1.9.5      v tidyr      1.3.2  
v purrr       1.2.1
```

```
-- Conflicts ----- tidyverse_conflicts() --  
x dplyr::filter() masks stats::filter()  
x dplyr::lag()     masks stats::lag()  
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
if (!"quanteda" %in% installed.packages()) {  
  install.packages("quanteda")  
}  
library(quanteda)
```

Package version: 4.3.1

Unicode version: 15.0

ICU version: 72.1

Parallel computing: 12 of 12 threads used.

See <https://quanteda.io> for tutorials and examples.

```

if (!"quanteda.textstats" %in% installed.packages()) {
  install.packages("quanteda.textstats")
}
library(quanteda.textstats)

if (!"quanteda.textplots" %in% installed.packages()) {
  install.packages("quanteda.textplots")
}
library(quanteda.textplots)

if (!"stopwords" %in% installed.packages()) {
  install.packages("stopwords")
}
library(stopwords)

if (!"spacyr" %in% installed.packages()) {
  install.packages("spacyr")
}
library(spacyr)

if (!"seededlda" %in% installed.packages()) {
  install.packages("seededlda")
}
library(seededlda)

```

Anexando pacote: 'seededlda'

O seguinte objeto é mascarado por 'package:quanteda':

```
info_tbb
```

O seguinte objeto é mascarado por 'package:stats':

```
terms
```

```

dados_discursos <- read_csv("discursos_camara_abril_2025.csv") |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_"))

```

Rows: 1527 Columns: 21

-- Column specification -----

```

Delimiter: ","
chr  (16): nome, uf, partido, uri, nome_civil, sexo, uf_nascimento, municipi...
dbl  (1): id
lgl  (1): data_falecimento
dtm  (1): hora_inicio
date (2): data_nascimento, data_situacao

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

```

```

corpus_discursos <- dados_discursos |>
  corpus(text_field = "transcricao")

```

4.1 Análise sintática: classes gramaticais

Classes gramaticais, classes de palavras ou partes do discurso (em inglês *parts of speech*) é a forma como categorizamos palavras com base em sua função. Por exemplo: substantivos, verbos, adjetivos, advérbios, pronomes, etc.

O (determinante)	gato (substantivo)	subiu (verbo)	no (preposição)	telhado (substantivo)
----------------------------	------------------------------	-------------------------	---------------------------	---------------------------------

A tarefa de atribuir uma classe para cada palavra de um texto é chamada de **rotulagem de sequência**. Existem diversos algoritmos para rotulagem de classes gramaticais.

Com classes anotadas, podemos fazer análises mais aprofundadas, como:

- investigar os principais verbos, substantivos e adjetivos utilizados por determinado grupo
- comparar contagens de cada classe para métricas de complexidade gramatical
- identificar nominalizações: transformação de verbos em substantivos para ocultar atores (ex: “a polícia repreendeu os manifestantes de forma violenta” vs “a repressão violenta contra os manifestantes”).

Para construir e avaliar modelos para rotulagens de sequência, pesquisadores de linguística computacional criaram diversas bases de anotação e esquemas de rotulação. Os mais utilizados são: Penn Treebank e Universal Dependencies.

O padrão de rótulos proposto por Marcus, Santorini, and Marcinkiewicz (1993) foi utilizado para criar a base anotada **Penn Treebank**:

Anotação (tag)	Descrição	Anotação (tag)	Descrição
CC	Coordinating conjunction	PRP\$	Possessive pronoun
CD	Cardinal number	RB	Adverb
DT	Determiner	RBR	Adverb, comparative
EX	Existential <i>there</i>	RBS	Adverb, superlative
FW	Foreign word	RP	Particle
IN	Preposition or subordinating conjunction	SYM	Symbol
JJ	Adjective	TO	<i>to</i>
JJR	Adjective, comparative	UH	Interjection
JJS	Adjective, superlative	VB	Verb, base form
LS	List item marker	VBD	Verb, past tense
MD	Modal	VBG	Verb, gerund or present participle
NN	Noun, singular or mass	VBN	Verb, past participle
NNS	Noun, plural	VBP	Verb, non-3rd person singular present
NNP	Proper noun, singular	VBZ	Verb, 3rd person singular present
NNPS	Proper noun, plural	WDT	Wh-determiner
PDT	Predeterminer	WP	Wh-pronoun
POS	Possessive ending	WP\$	Possessive wh-pronoun
PRP	Personal pronoun	WRB	Wh-adverb

Nivre et al. (2020) propõe o **Universal Dependencies v2**, uma base anotada com notações gramaticais universais:

Anotação	Nome (inglês)	Nome (por- tuguês)	Função	Exemplo
ADJ	adjective	Adjetivo	Atribuem características aos substantivos	<i>feliz, grande, azul, rápido, inteligente, cansado</i>
ADP	adposition	Adposição/ Preposição	Ligam palavras estabelecendo relação de sentido entre elas	<i>de, em, para, com, por, sem, sob</i>

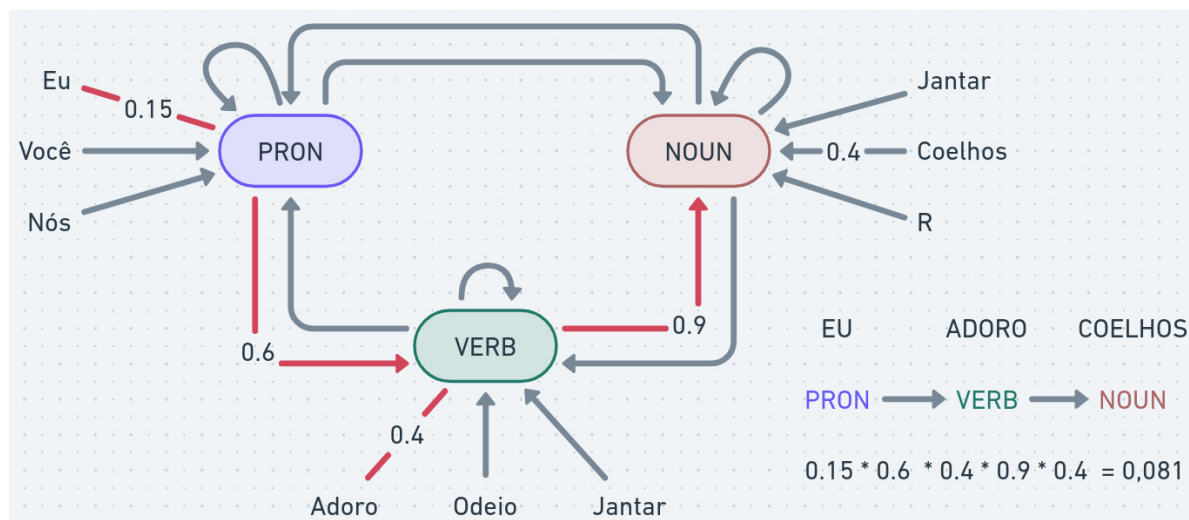
Anotação	Nome (inglês)	Nome (português)	Função	Exemplo
ADV	adverb	Advérbio	Modificam verbos, adjetivos ou outros advérbios	<i>não, muito, aqui, rapidamente, ontem, sempre, talvez</i>
AUX	auxiliary	Verbo auxiliar	Complementar verbos: “tenho feito”, “está chovendo”, “foi construído”, “vou estudar”	ter, haver, ser, estar, ir
CCONJ	coordinating conjunction	Conjunção coordenativa	Ligam orações ou termos de mesma função sintática	e, ou, mas, porém, contudo, todavia
DET	determiner	Determinante	Acompanham o substantivo para especificá-lo, incluindo artigos e pronomes adjetivos	o, a, um, umas, este, meu, alguns, qualquer
INTJ	interjection	Interjeição	Expressam emoções repentinas	ah!, uau!, eita!, oxalá!, nossa!, ai!
NOUN	noun	Substantivo	Nomeiam seres, objetos, sentimentos ou conceitos	livro, amor, cachorro, ideia, tempo, água
NUM	numeral	Numeral	Indicam quantidade exata, ordem ou fração	um, dois, mil, primeiro, segundo, dobro, metade
PART	particle	Partícula	Palavras invariáveis que não se encaixam perfeitamente nas outras classes, geralmente com função de realce ou apassivação	apassivadora: “vende-se”. realce: “ele é que sabe”
PRON	pronoun	Pronome	Substituem o substantivo	eu, você, eles, quem, que (pronome relativo), alguém, nada
PROPN	proper noun	Substantivo próprio	Nomeiam entidades específicas, sempre com letra maiúscula	Brasil, Maria, Amazonas, São Paulo, Google

Anotação	Nome (inglês)	Nome (português)	Função	Exemplo
PUNCT	punctuation	Pontuação	Sinais gráficos usados na escrita	, . ? ! ; : () " ”
SCONJ	subordinating conjunction	Conjunção subordinativa	Introduzem orações subordinadas, indicando dependência	que, se, porque, embora, quando, conforme
SYM	symbol	Símbolo	Caracteres ou símbolos não alfabéticos	%, \$, €, +, @, =
VERB	verb	Verbo	Indicam ação, estado ou fenômeno da natureza; atuando como verbo principal da frase	correr, falar, comer, pensar, agir, chover
X	other	Outro	Palavras não classificáveis, como erros de digitação, URLs, palavras estrangeiras	www, http, software, feedback

Modelos ocultos de Markov (HMM, em inglês, *Hidden Markov Models*) são modelos estatísticos comuns para tarefas de rotulagem de sequência.

A ideia central do HMM é definir estados e transições probabilísticas.

Vejamos um exemplo com três classes de palavras:



No exemplo, a palavra “eu” tem 15% de chance de ser um pronome e a próxima palavra tem 60% de chance de ser verbo e 40% de chance de ser “adoro”.

As probabilidades são acumuladas e o “caminho mais provável” é escolhido. A escolha de melhor caminho é feita, por exemplo, com o algoritmo de Viterbi.

HMMs não são a única abordagem para este tipo de problema:

- **Gramáticas formais:** historicamente, gramáticas foram propostas para rotular frases e derivar árvores sintáticas. Gramáticas livres de contexto contêm uma série de regras determinísticas para derivar frases em diversas línguas. A complexidade das regras e questões como ambiguidade fizeram com que o uso de gramáticas seja mais restrito.
- **Modelos estatísticos:** técnicas como HMMs, quando acrescidas de atributos adicionais (formas das palavras, dados morfológicos etc.) que são conhecidas como campos aleatórios condicionais (CRF, do inglês *conditional random fields*), possuem bom desempenho (96% de acurácia, o mesmo desempenho de anotadores humanos).
- **Modelos de linguagem:** o uso de redes neurais para linguística computacional e processamento de linguagem natural é bastante difundido. Estes modelos se destacam principalmente em tarefas que exigem conhecimento sobre significados (semântica) e contexto externo. Veremos mais detalhes sobre estes modelos nas próximas aulas.

4.1.1 Exemplo de análise sintática com Spacy

Spacy é uma biblioteca da linguagem python que foi adaptada pela biblioteca **spacyr** em R.

Spacy utiliza diversos componentes para analisar linguagem natural, como tokenizers, parsers, modelos de embedding etc. Estes componentes executam em sequência (*pipeline*) e são configuráveis.

Spacy utiliza diversos componentes para analisar linguagem natural, como tokenizers, parsers, modelos de embedding etc. Estes componentes executam em sequência (*pipeline*) e são configuráveis.

A configuração padrão que vamos utilizar é o *pipeline* para língua portuguesa, treinada a partir de textos de notícias: `pt_core_news_sm`.

O código abaixo baixa e inicializa os componentes:

```
spacy_install()
```

```
Warning in spacy_install(): Skipping installation. Use `force` to force
installation or update. Or use `spacy_download_langmodel()` if you just want to
install a model.
```

```
spacy_download_langmodel("pt_core_news_sm")
```

Warning in spacy_download_langmodel("pt_core_news_sm"): Skipping installation.
Use `force` to force installation or update.

```
spacy_initialize("pt_core_news_sm")
```

successfully initialized (spaCy Version: 3.8.11, language model: pt_core_news_sm)

Para mais detalhes sobre a pipeline: <https://spacy.io/models/pt>

Com a biblioteca carregada, podemos processar os textos com a função `spacy_parse`:

```
texto <- "O gato subiu no telhado."
```

```
spacy_parse(texto)
```

Warning in spacy_parse.character(texto): lemmatization may not work properly in
model 'pt_core_news_sm'

	doc_id	sentence_id	token_id	token	lemma	pos	entity
1	text1	1	1	O	o	DET	
2	text1	1	2	gato	gato	NOUN	
3	text1	1	3	subiu	subir	VERB	
4	text1	1	4	no	em o	ADP	
5	text1	1	5	telhado	telhado	NOUN	
6	text1	1	6	.	.	PUNCT	

A função `spacy_parse` também processa objetos `corpus` da biblioteca `quanteda`:

```
frases_ulysses <- c(
  "A Nação quer mudar, a Nação deve mudar, a Nação vai mudar.",
  "O homem público é o cidadão de tempo inteiro, de quem as circunstâncias exigem o sacrifício."
)

corpus_ulysses <- corpus(frases_ulysses)

spacy_parse(corpus_ulysses)
```


Warning in spacy_parse.character(frases_ulysses): lemmatization may not work properly in model 'pt_core_news_sm'

	doc_id	sentence_id	token_id	token	lemma	pos	entity
1	text1	1	1	A	o	DET	
2	text1	1	2	Nação	Nação	PROPN	MISC_B
3	text1	1	3	quer	querer	VERB	
4	text1	1	4	mudar	mudar	VERB	
5	text1	1	5	,	,	PUNCT	
6	text1	1	6	a	o	DET	
7	text1	1	7	Nação	Nação	PROPN	MISC_B
8	text1	1	8	deve	dever	VERB	
9	text1	1	9	mudar	mudar	VERB	
10	text1	1	10	,	,	PUNCT	
11	text1	1	11	a	o	DET	
12	text1	1	12	Nação	Nação	PROPN	MISC_B
13	text1	1	13	vai	ir	AUX	
14	text1	1	14	mudar	mudar	VERB	
15	text1	1	15	.	.	PUNCT	
16	text2	1	1	O	o	DET	
17	text2	1	2	homem	homem	NOUN	
18	text2	1	3	público	público	ADJ	
19	text2	1	4	é	ser	AUX	
20	text2	1	5	o	o	DET	
21	text2	1	6	cidadão	cidadão	NOUN	
22	text2	1	7	de	de	ADP	
23	text2	1	8	tempo	tempo	NOUN	
24	text2	1	9	inteiro	inteiro	ADJ	
25	text2	1	10	,	,	PUNCT	
26	text2	1	11	de	de	ADP	
27	text2	1	12	quem	quem	PRON	
28	text2	1	13	as	o	DET	
29	text2	1	14	circunstâncias	circunstância	NOUN	
30	text2	1	15	exigem	exigir	VERB	
31	text2	1	16	o	o	DET	
32	text2	1	17	sacrifício	sacrifício	NOUN	
33	text2	1	18	da	de o	ADP	
34	text2	1	19	liberdade	liberdade	NOUN	
35	text2	1	20	pessoal	pessoal	ADJ	
36	text2	1	21	,	,	PUNCT	
37	text2	1	22	mas	mas	CCONJ	
38	text2	1	23	a	a	ADP	

39	text2	1	24	quem	quem	PRON	
40	text2	1	25	o	o	DET	
41	text2	1	26	destino	destino	NOUN	
42	text2	1	27	oferece	oferecer	VERB	
43	text2	1	28	a	o	DET	
44	text2	1	29	mais	mais	ADV	
45	text2	1	30	confortadora	confortadorar	VERB	
46	text2	1	31	das	de o	ADP	
47	text2	1	32	recompensas	recompensa	NOUN	
48	text2	1	33	:	:	PUNCT	
49	text2	1	34	a	o	DET	
50	text2	1	35	de	de	SCONJ	
51	text2	1	36	servir	servir	VERB	
52	text2	1	37	à	a o	ADP	
53	text2	1	38	Nação	Nação	PROPN	MISC_B
54	text2	1	39	em	em	ADP	
55	text2	1	40	sua	seu	DET	
56	text2	1	41	grandeza	grandeza	NOUN	
57	text2	1	42	e	e	CCONJ	
58	text2	1	43	projeção	projeção	NOUN	
59	text2	1	44	na	em o	ADP	
60	text2	1	45	eternidade	eternidade	NOUN	
61	text2	1	46	.	.	PUNCT	

Por padrão, o resultado retorna:

- `doc_id`: o identificador do documento
- `sentence_id`: o identificador da frase
- `token_id`: o identificador do token
- `token`: o texto do token (palavra, pontuação, símbolo)
- `lemma`: o lema do token (sua versão canônica)
- `pos`: part-of-speech ou classe gramatical
- `entity`: entidade nomeada identificada (pessoas, lugares, organizações, etc)

Vamos agora identificar estatísticas dos discursos de abril de 2025 da Câmara dos Deputados:

```
amostra_discursos <- corpus_sample(corpus_discursos, 100) # amostra de 100 discursos (apenas
dt_sintaxe_discursos <- spacy_parse(amostra_discursos, entity = FALSE) # processa discurso d
```

Warning in spacy_parse.character(amostra_discursos, entity = FALSE):
lemmatization may not work properly in model 'pt_core_news_sm'

```
head(dt_sintaxe_discursos)
```

	doc_id	sentence_id	token_id	token	lemma	pos
1	446_Duarte_PSB	1	1	DISCURSO	DISCURSO	NOUN
2	446_Duarte_PSB	2	1	NA	NA	ADP
3	446_Duarte_PSB	2	2	ÍNTEGRA	ÍNTEGRA	NOUN
4	446_Duarte_PSB	3	1	ENCAMINHADO	ENCAMINHADO	NOUN
5	446_Duarte_PSB	3	2	PELO	PELO	ADP
6	446_Duarte_PSB	3	3	SR	SR	PROPN

Contagem de classes gramaticais:

```
dt_sintaxe_discursos |>  
  count(pos, sort = TRUE) # contagem de classes gramaticais
```

	pos	n
1	NOUN	8411
2	PUNCT	6964
3	ADP	6331
4	VERB	4821
5	DET	4570
6	PROPN	3984
7	PRON	2406
8	ADV	2331
9	ADJ	2247
10	AUX	1655
11	SCONJ	1308
12	CCONJ	1249
13	NUM	714
14	SYM	75
15	X	28
16	INTJ	18

Verbos mais utilizados:

```
dt_sintaxe_discursos |>  
  filter(pos == "VERB") |>  
  count(lemma, sort = TRUE) |>  
  head(10)
```

	lemma	n
1	ter	276
2	fazer	188
3	querer	182
4	poder	112
5	dizer	108
6	haver	101
7	falar	78
8	precisar	66
9	ser	63
10	dar	47

A função `as.tokens` permite converter a tabela novamente em objeto tokens da biblioteca `quanteda` concatenando a informação de classe gramatical (opcionalmente utilizando lemas no lugar do token original):

```
tokens_sintaxe_discursos <- as.tokens(dt_sintaxe_discursos, include_pos = "pos", use_lemma =
tokens_sintaxe_discursos |>
  tokens_select("*/VERB")
```

Tokens consisting of 100 documents.

446_Duarte_PSB :

[1]	"venho/VERB"	"comemorar/VERB"	"celebrar/VERB"	"transformar/VERB"
[5]	"constrói/VERB"	"projetar/VERB"	"basear/VERB"	"refleter/VERB"
[9]	"superar/VERB"	"impor/VERB"	"integrar/VERB"	"atraemr/VERB"
[... and 41 more]				

744_João Daniel_PT :

[1]	"querer/VERB"	"levar/VERB"	"acompanhar/VERB"	"ocorrer/VERB"
[5]	"saber/VERB"	"haver/VERB"	"merecer/VERB"	"analisar/VERB"
[9]	"respeitar/VERB"	"garantir/VERB"	"vir/VERB"	"recebemos/VERB"
[... and 2 more]				

88_Allan Garcês_PP :

[1]	"orientar/VERB"	"Vemos/VERB"	"chorar/VERB"	"poder/VERB"
[5]	"chorar/VERB"	"poder/VERB"	"chorar/VERB"	"querer/VERB"
[9]	"obstruir/VERB"	"fazer/VERB"	"Vemos/VERB"	"querer/VERB"
[... and 3 more]				

494_Enfermeira Ana Paula_PODE :

[1]	"receber/VERB"	"elegir/VERB"	"tínhar/VERB"
-----	----------------	---------------	---------------

```

[4] "avançar/VERB"      "haver/VERB"      "sinto/VERB"
[7] "receber/VERB"      "querer/VERB"      "parabenizá-los/VERB"
[10] "representado/VERB" "continuar/VERB"   "estarer/VERB"

26_Afonso Hamm_PP :
[1] "querer/VERB"      "manifestar/VERB" "ter/VERB"      "tratar/VERB"
[5] "estar/VERB"      "passar/VERB"      "assentado/VERB" "comprometer/VERB"
[9] "precisar/VERB"    "estabeleci/VERB" "honrar/VERB"    "fazer/VERB"
[ ... and 15 more ]

543_Fernanda Melchionna_PSOL :
[1] "cair/VERB"      "ador/VERB"      "dizer/VERB"      "defender/VERB"
[5] "ver/VERB"      "provocar/VERB"   "prometer/VERB"   "anunciar/VERB"
[9] "lamber/VERB"    "fazer/VERB"      "defender/VERB"   "atacar/VERB"
[ ... and 41 more ]

[ reached max_ndoc ... 94 more documents ]

```

Por exemplo, para contar os substantivos próprios mais utilizados no PT e PL:

```

tokens_sintaxe_discursos <- as.tokens(dt_sintaxe_discursos, include_pos = "pos")

docvars(tokens_sintaxe_discursos) <- docvars(amostra_discursos) # recupera metadados do corpus

tokens_sintaxe_discursos |>
  tokens_subset(partido %in% c("PL", "PT")) |>
  tokens_select("*/PROPN") |>
  dfm() |>
  textstat_frequency(10, groups = partido)

```

	feature	frequency	rank	docfreq	group
1	pl/propn	32	1	15	PL
2	governo/propn	24	2	6	PL
3	sr/propn	23	3	19	PL
4	bloco/propn	21	4	17	PL
5	presidente/propn	21	4	9	PL
6	deputados/propn	16	6	8	PL
7	motta/propn	15	7	5	PL
8	lula/propn	15	7	5	PL
9	brasil/propn	14	9	7	PL
10	hugo/propn	14	9	5	PL
11	brasil/propn	49	1	17	PT

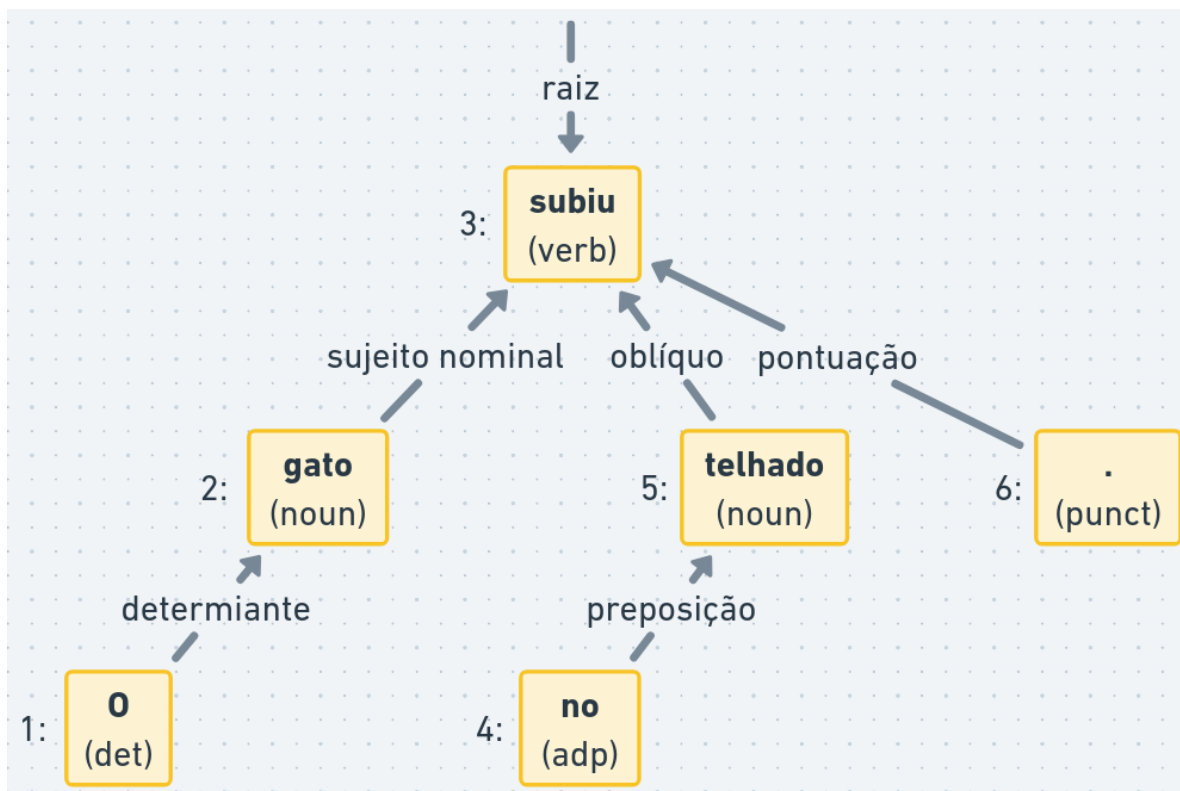
12	governo/propn	35	2	12	PT
13	papa/propn	29	3	2	PT
14	lula/propn	25	4	11	PT
15	pt/propn	24	5	22	PT
16	bolsonaro/propn	23	6	8	PT
17	bloco/propn	22	7	22	PT
18	casa/propn	21	8	11	PT
19	"/propn	21	8	7	PT
20	sr/propn	20	10	20	PT

4.2 Análise sintática: relações de dependência

Além das classes gramaticais, também podemos gerar **relações de dependência** entre as palavras:

- as relações de dependência determinam a função de uma palavra, representada pelo **tipo de relação de dependência**
- a função é relativa a uma palavra “pai” dentro da frase, ligando o dependente à dependência
- cada frase possui uma palavra raiz (geralmente o verbo principal)

Por exemplo:



As relações de dependência permitem análises mais aprofundadas, como:

- qualificação: adjetivos relacionados a um objeto
- identificação de atores e ações
- atribuir agência: usos de voz ativa e passiva ou estruturas impessoais

Os dois últimos pontos fazem com que as relações de dependência formem uma hierarquia ou árvore. Estas regras não são universais, algumas relações e línguas são melhor representadas como redes de dependências ou grafos, onde qualquer palavra pode depender de outra.

As hierarquias das relações de dependências são similares, mas não equivalem às árvores sintáticas. Árvores sintáticas geram relações entre constituintes, que podem ser palavras ou sub-frases (ex: frase nominal, frase verbal etc).

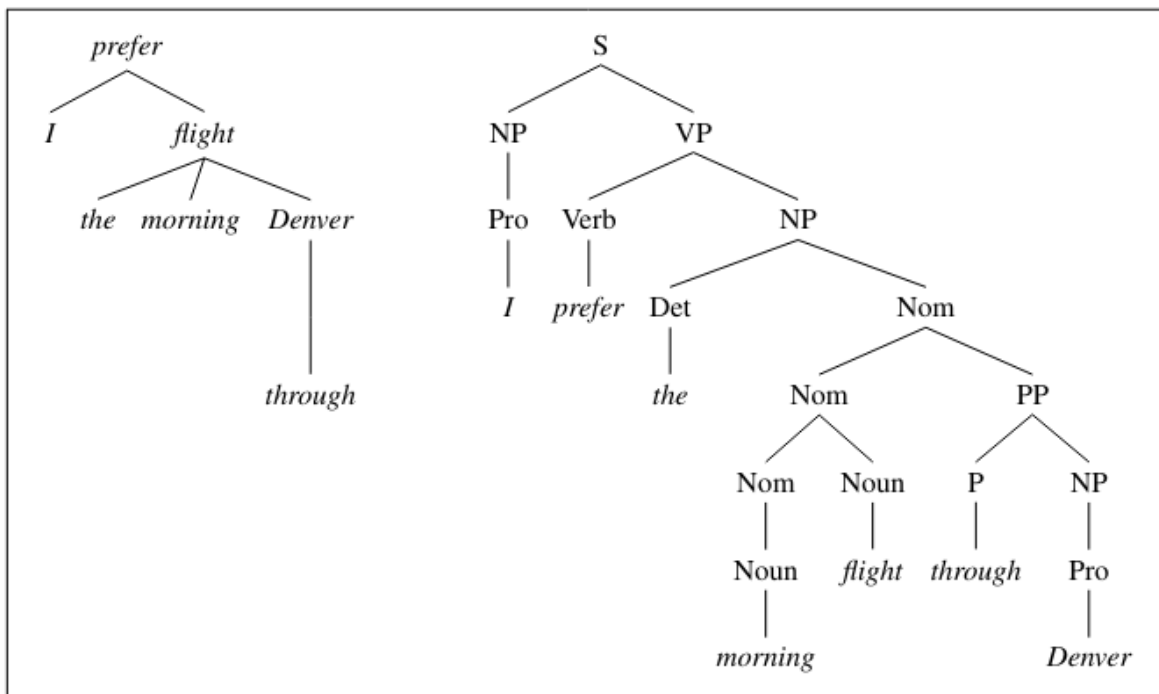


Figure 19.1 Dependency and constituent analyses for *I prefer the morning flight through Denver.*

Há duas vantagens de relações de dependência sobre árvores gramaticais:

- relações de dependência são mais diretas, por exemplo, a palavra do sujeito é conectada diretamente ao verbo
- gramáticas impõem ordem nas relações e algumas línguas são mais flexíveis quanto à ordem das palavras

Alguns exemplos de tipos de relações de dependência do padrão Universal Dependencies v2 são:

Tag (dep_rel)	Nome (Inglês)	Nome (Português)	Exemplo Prático (O termo em negrito recebe a tag)
root	Root	Raiz (Termo principal)	O plenário aprovou a medida provisória.
nsubj	Nominal subject	Sujeito nominal	O ministro discursou hoje.
nsubj:pass	Passive nominal subject	Sujeito nominal passivo	A lei foi sancionada ontem.
obj	Object	Objeto direto	O partido criticou a reforma .
iobj	Indirect object	Objeto indireto	O governo concedeu perdão aos devedores.
obl	Oblique nominal	Modificador oblíquo (prep.)	O debate ocorreu na comissão .

amod	Adjectival modifier	Modificador adjetival	O país enfrenta uma crise econômica .
nmod	Nominal modifier	Modificador nominal	É um projeto de lei .
advmod	Adverbial modifier	Modificador adverbial	O deputado votou rapidamente .
appos	Appositional modifier	Aposto	Silva, o relator , pediu a palavra.
cc	Coordinating conjunction	Conjunção coordenativa	Senadores e deputados debateram.
conj	Conjunct	Elemento coordenado	Senadores e deputados debateram.
case	Case marking	Marca de caso (Preposição)	Protestaram contra o corte.
mark	Marker	Marcador (Subordinação)	Prometeu que reduziria impostos.
acl	Clausal modifier of noun	Oração adjetiva/modificadora	O parlamentar, que estava ausente , justificou-se.
xcomp	Open clausal complement	Complemento oracional aberto	A oposição tentou obstruir a pauta.
ccomp	Clausal complement	Complemento oracional	O relatório indica que haverá déficit .
det	Determiner	Determinante (Artigo/Pronome)	Os vetos foram mantidos.
nummod	Numeric modifier	Modificador numérico	O partido elegeu quinze prefeitos.
punct	Punctuation	Pontuação	Senhor presidente, peço a palavra.

A lista completa de relações no padrão Universal Dependencies v2 está disponível em <https://universaldependencies.org/u/dep/index.html>.

4.2.1 Exemplo de relações de dependência com Spacy

Podemos gerar relações de dependência com a biblioteca `spacyr` utilizando o parâmetro `dependency = TRUE` na função `spacy_parse`:

```
texto <- "O gato subiu no telhado."

spacy_parse(texto, dependency = TRUE)
```

Warning in `spacy_parse.character(texto, dependency = TRUE)`: lemmatization may not work properly in model 'pt_core_news_sm'

	doc_id	sentence_id	token_id	token	lemma	pos	head_token_id	dep_rel
1	text1	1	1	o	o	DET	2	det
2	text1	1	2	gato	gato	NOUN	3	nsubj
3	text1	1	3	subiu	subir	VERB	3	ROOT
4	text1	1	4	no	em o	ADP	5	case
5	text1	1	5	telhado	telhado	NOUN	3	obl
6	text1	1	6	.	.	PUNCT	3	punct

entity

1
2
3
4
5
6

Os dados retornados agora incluem:

- `head_token_id`: aponta para o token “pai” na árvore de dependências
- `dep_rel`: o tipo da relação de dependência

```
textos_exemplo <- c(
  doc1 = "O governo cortou os gastos públicos ontem.",
  doc2 = "Os gastos públicos foram cortados pela inflação.",
  doc3 = "A oposição propôs uma nova lei orçamentária."
)

sintaxe_exemplo <- spacy_parse(textos_exemplo, pos = TRUE, dependency = TRUE, nounphrase = T
```

Warning in `spacy_parse.character(textos_exemplo, pos = TRUE, dependency = TRUE,`
: lemmatization may not work properly in model 'pt_core_news_sm'

sintaxe_exemplo

	doc_id	sentence_id	token_id	token	lemma	pos	head_token_id
1	doc1	1	1	o	o	DET	2
2	doc1	1	2	governo	governo	NOUN	3
3	doc1	1	3	cortou	cortar	VERB	3

4	doc1	1	4	os	o	DET	5
5	doc1	1	5	gastos	gasto	NOUN	3
6	doc1	1	6	públicos	público	ADJ	5
7	doc1	1	7	ontem	ontem	ADV	5
8	doc1	1	8	.	.	PUNCT	3
9	doc2	1	1	Os	o	DET	2
10	doc2	1	2	gastos	gasto	NOUN	5
11	doc2	1	3	públicos	público	ADJ	2
12	doc2	1	4	foram	ser	AUX	5
13	doc2	1	5	cortados	cortar	VERB	5
14	doc2	1	6	pela	por o	ADP	7
15	doc2	1	7	inflação	inflação	NOUN	5
16	doc2	1	8	.	.	PUNCT	5
17	doc3	1	1	A	o	DET	2
18	doc3	1	2	oposição	oposição	NOUN	3
19	doc3	1	3	propôs	propor	VERB	3
20	doc3	1	4	uma	um	DET	6
21	doc3	1	5	nova	novo	ADJ	6
22	doc3	1	6	lei	lei	NOUN	3
23	doc3	1	7	orçamentária	orçamentário	ADJ	6
24	doc3	1	8	.	.	PUNCT	3

	dep_rel	entity	nounphrase	whitespace
1	det		beg	TRUE
2	nsubj		end_root	TRUE
3	ROOT			TRUE
4	det		beg	TRUE
5	obj		mid_root	TRUE
6	amod		end	TRUE
7	advmod			FALSE
8	punct			FALSE
9	det		beg	TRUE
10	nsubj:pass		mid_root	TRUE
11	amod		end	TRUE
12	aux:pass			TRUE
13	ROOT			TRUE
14	case			TRUE
15	obl:agent		beg_root	FALSE
16	punct			FALSE
17	det		beg	TRUE
18	nsubj		end_root	TRUE
19	ROOT			TRUE
20	det		beg	TRUE
21	amod		mid	TRUE

22	obj	mid_root	TRUE
23	amod	end	FALSE
24	punct		FALSE

Para identificar atores podemos filtrar por tipo de relação “sujeito nominal” `nsubj`:

```
atores <- sintaxe_exemplo |>
  filter(dep_rel == "nsubj") |> # sujeitos nominais
  select(doc_id, sentence_id, ator = token, verbo_id = head_token_id)
```

atores

	doc_id	sentence_id	ator	verbo_id
1	doc1	1	governo	3
2	doc3	1	oposição	3

Para associar às ações, precisamos obter o token relativo à `head_token_id` que aponta para o verbo do qual o sujeito nominal depende.

Utilizamos a função `left_join` do tidyverse para unir novamente as colunas da tabela original sem filtro (`sintaxe_exemplo`) de acordo com os critérios: mesmo documento, mesma frase, com o token identificado pela coluna `verbo_id`:

```
acoes_atores <- atores |>
  left_join(sintaxe_exemplo, by = c( # juntando de volta para pegar o lema do verbo
    "doc_id" = "doc_id",
    "sentence_id" = "sentence_id",
    "verbo_id" = "token_id"
  ))
```

acoes_atores

	doc_id	sentence_id	ator	verbo_id	token	lemma	pos	head_token_id	dep_rel
1	doc1	1	governo	3	cortou	cortar	VERB	3	ROOT
2	doc3	1	oposição	3	propôs	propor	VERB	3	ROOT
			entity		nounphrase		whitespace		
1									TRUE
2									TRUE

Selecionando as colunas de interesse:

```
acoes_atores |>
  select(doc_id, sentence_id, ator, verbo_acao = lemma) |>
  mutate(ator = tolower(ator))
```

	doc_id	sentence_id	ator	verbo_acao
1	doc1	1	governo	cortar
2	doc3	1	oposição	propor

Vejamos outro exemplo de como detectar frases com linguagem passiva:

```
sintaxe_exemplo |>
  filter(dep_rel == "nsubj:pass")
```

	doc_id	sentence_id	token_id	token	lemma	pos	head_token_id	dep_rel	entity
1	doc2	1	2	gastos	gasto	NOUN	5	nsubj:pass	
				nounphrase	whitespace				
1	mid_root			TRUE					

4.3 Modelos de tópicos: Latent Dirichlet Allocation (LDA)

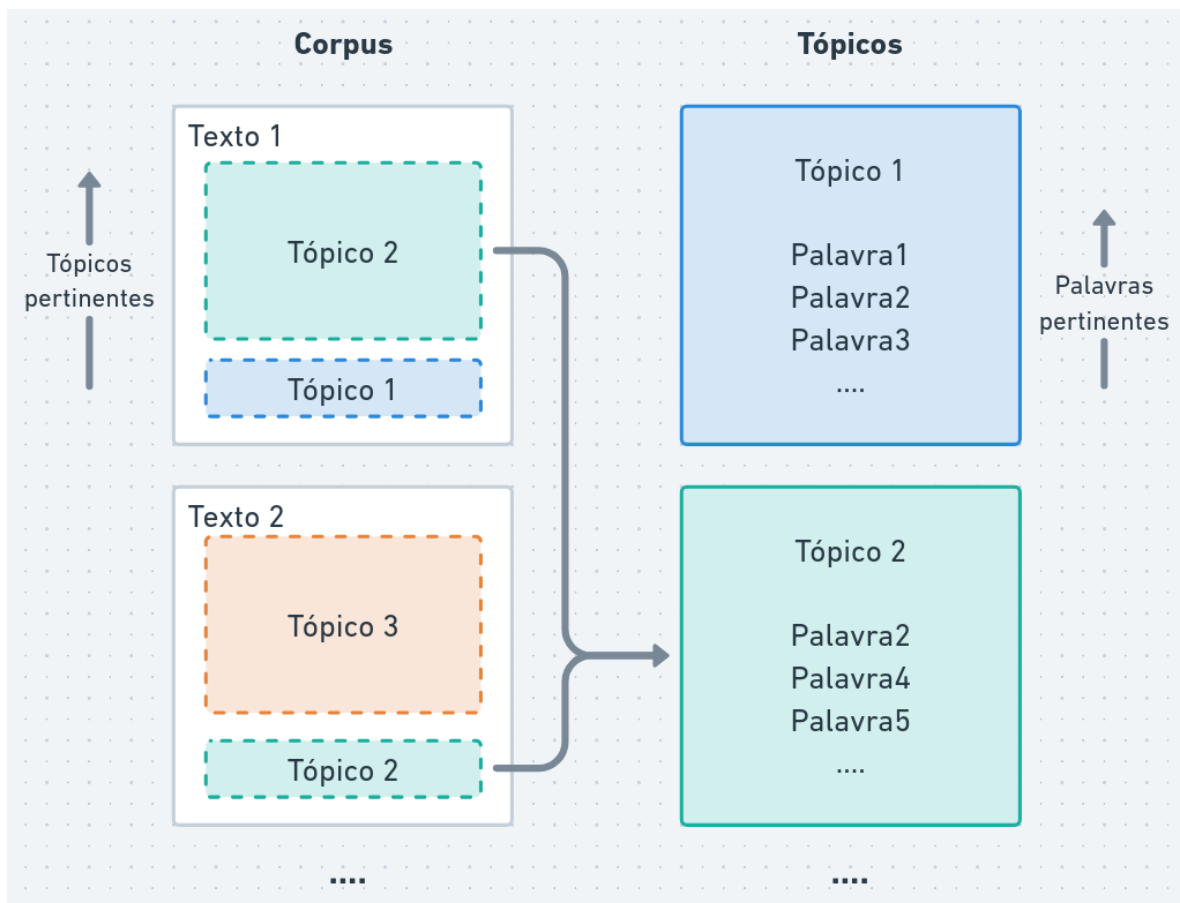
Em diversos cenários estamos analisando um conjunto de documentos e queremos identificar os temas ou tópicos principais presentes nesses documentos. Por exemplo:

- quais temas foram discutidos no plenário da câmara em 2025?
- quais os temas mais discutidos por um partido político?
- quais os temas mais discutidos por um deputado?

A **análise de tópicos** é uma técnica de modelagem de texto que permite descobrir automaticamente os tópicos subjacentes em um conjunto de documentos.

A análise de tópicos é considerada uma técnica de **aprendizado não-supervisionado**. Os dados são apresentados e o modelo tenta encontrar padrões e estruturas subjacentes sem rótulos ou categorias pré-definidas.

O objetivo é identificar grupos de palavras que frequentemente coocorrem em documentos, sugerindo a presença de um tópico.



Latent Dirichlet Allocation (LDA) é um dos métodos mais populares para análise de tópicos. Ele assume que:

- cada documento é uma mistura de vários tópicos
- cada tópico é uma distribuição de palavras

4.3.1 Exemplo de análise de tópicos com LDA

Primeiramente, vamos construir uma DFM com o corpus de interesse.

Para melhores resultados, vamos nos concentrar em palavras:

- muito faladas em termos absolutos: 20% dos termos mais frequentes
- concentradas em poucos discursos: termos que aparecem em até 10% dos discursos

```
tm_dfm <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_symbols = TRUE) |>
  tokens_remove(stopwords("portuguese")) |>
  dfm() |>
  dfm_trim(
    min_termfreq = 0.8, # descarta os 80% de termos menos frequentes
    termfreq_type = "quantile",
    max_docfreq = 0.1, # descarta palavras que aparecem em mais de 10% dos documentos
    docfreq_type = "prop"
  )

tm_dfm
```

Document-feature matrix of: 1,527 documents, 5,063 features (98.39% sparse) and 20 docvars.

docs	features	íntegra	encaminhado	taquigráfico	senhor	nobres
1_Acácio Favacho_MDB		1	1	1	1	1
2_Acácio Favacho_MDB		0	0	0	0	0
3_Acácio Favacho_MDB		1	1	1	1	0
4_Adail Filho_REPUBLICANOS		0	0	0	0	0
5_Adilson Barroso_PL		0	0	0	0	0
6_Adilson Barroso_PL		0	0	0	0	0

docs	features	caros	sistema	comunicações	boa	tarde
1_Acácio Favacho_MDB		1	1	1	1	1
2_Acácio Favacho_MDB		0	0	0	0	0
3_Acácio Favacho_MDB		1	1	1	1	1
4_Adail Filho_REPUBLICANOS		0	0	0	0	0
5_Adilson Barroso_PL		0	1	0	1	0
6_Adilson Barroso_PL		0	0	0	0	0

[reached max_ndoc ... 1,521 more documents, reached max_nfeat ... 5,053 more features]

Para construir o modelo utilizamos `textmodel_lda`:

```
set.seed(42) # semente para gerador de números aleatórios

numero_de_topicos <- 10

tmod_discursos <- textmodel_lda(tm_dfm, numero_de_topicos)
```

A função `terms` retorna a matriz com as palavras mais relevantes de cada tópico:

```
terms(tmod_discursos) |>
  as.data.frame()
```

	topic1	topic2	topic3	topic4	topic5	topic6
1	desenvolvimento	glauber	trabalhadores	francisco	golpe	pec
2	acre	inss	renda	papa	supremo	policiais
3	comunidades	aposentados	unidos	deus	esquerda	militar
4	ambiental	conselho	imposto	mulheres	direita	senhor
5	amazonas	ética	economia	autismo	8	policial
6	ambiente	braga	salário	igreja	stf	deus
7	indígenas	psol	mínimo	cruz	judiciário	pai
8	sustentável	mandato	5	autistas	ditadura	civil
9	áreas	cassação	trump	autista	querem	organizado
10	amazônia	cpi	países	amor	crimes	coronel

	topic7	topic8	topic9	topic10
1	município	votar	nº	contas
2	região	fundo	sistema	ministério
3	prefeito	hugo	acesso	gestão
4	sul	pauta	profissionais	falta
5	bahia	texto	direitos	órgãos
6	hospital	urgência	crianças	orçamento
7	vereadores	matéria	aprovação	r
8	governador	votação	2024	obras
9	vereador	voto	públicos	queremos
10	maranhão	vota	2025	senhor

A função `topics` traz o tópico mais relevante para cada documento:

```
topics(tmod_discursos) |>
  as.data.frame() |>
  head(30)
```

	topics(tmod_discursos)
1_Acácio Favacho_MDB	topic1
2_Acácio Favacho_MDB	topic4
3_Acácio Favacho_MDB	topic1
4_Adail Filho_REPUBLICANOS	topic1
5_Adilson Barroso_PL	topic5
6_Adilson Barroso_PL	topic8
7_Adilson Barroso_PL	topic3
8_Adriana Ventura_NOVO	topic8

9_Adriana Ventura_NOVO	topic3
10_Adriana Ventura_NOVO	topic5
11_Adriana Ventura_NOVO	topic4
12_Adriana Ventura_NOVO	topic9
13_Adriana Ventura_NOVO	topic5
14_Adriana Ventura_NOVO	topic9
15_Adriana Ventura_NOVO	topic3
16_Adriana Ventura_NOVO	topic1
17_Adriana Ventura_NOVO	topic10
18_Adriana Ventura_NOVO	topic5
19_Adriana Ventura_NOVO	topic9
20_Adriana Ventura_NOVO	topic5
21_Adriana Ventura_NOVO	topic5
22_Adriana Ventura_NOVO	topic5
23_Afonso Hamm_PP	topic7
24_Afonso Hamm_PP	topic7
25_Afonso Hamm_PP	topic7
26_Afonso Hamm_PP	topic7
27_Afonso Hamm_PP	topic7
28_Afonso Hamm_PP	topic1
29_Afonso Motta_PDT	topic8
30_Afonso Motta_PDT	topic8

Podemos também investigar em mais detalhes os valores do modelo conforme duas matrizes de parâmetros:

- **theta**: probabilidades de cada tópico por documento (linhas = documentos, colunas = tópicos)
- **phi**: probabilidades de cada tópico por palavra (linhas = tópicos, colunas = palavras)

```
tmod_discursos$theta |>
  as.data.frame() |>
  head()
```

	topic1	topic2	topic3	topic4
1_Acácio Favacho_MDB	0.503289474	0.009868421	0.167763158	0.095394737
2_Acácio Favacho_MDB	0.029801325	0.003311258	0.009933775	0.493377483
3_Acácio Favacho_MDB	0.340361446	0.027108434	0.087349398	0.009036145
4_Adail Filho_REPUBLICANOS	0.312500000	0.006944444	0.048611111	0.034722222
5_Adilson Barroso_PL	0.011627907	0.135658915	0.042635659	0.003875969
6_Adilson Barroso_PL	0.003816794	0.011450382	0.141221374	0.019083969
	topic5	topic6	topic7	topic8

1_Acácio Favacho_MDB	0.003289474	0.003289474	0.023026316	0.023026316
2_Acácio Favacho_MDB	0.003311258	0.043046358	0.003311258	0.062913907
3_Acácio Favacho_MDB	0.009036145	0.003012048	0.268072289	0.003012048
4_Adail Filho_REPUBLICANOS	0.062500000	0.062500000	0.076388889	0.104166667
5_Adilson Barroso_PL	0.515503876	0.127906977	0.050387597	0.011627907
6_Adilson Barroso_PL	0.156488550	0.026717557	0.003816794	0.568702290
	topic9	topic10		
1_Acácio Favacho_MDB	0.148026316	0.02302632		
2_Acácio Favacho_MDB	0.274834437	0.07615894		
3_Acácio Favacho_MDB	0.147590361	0.10542169		
4_Adail Filho_REPUBLICANOS	0.159722222	0.13194444		
5_Adilson Barroso_PL	0.011627907	0.08914729		
6_Adilson Barroso_PL	0.003816794	0.06488550		

Por exemplo, para relacionar os discursos mais pertinentes ao tópico 4:

```
tmod_discursos$theta |>
  as.data.frame() |>
  arrange(desc(topic4)) |> # ordena documentos mais relevantes do tópico 4
  select(topic4) |>
  head(10)
```

	topic4
281_Chris Tonietto_PL	0.9495114
842_Lenir de Assis_PT	0.8962264
887_Luiz Carlos Hauly_PODE	0.8960784
903_Luiz Gastão_PSD	0.8750000
414_Diego Garcia_REPUBLICANOS	0.8575758
282_Chris Tonietto_PL	0.8483607
1298_Rubens Pereira Júnior_PT	0.8479381
894_Luiz Couto_PT	0.8458333
291_Clodoaldo Magalhães_PV	0.8369565
1138_Pedro Campos_PSB	0.8129771

Da mesma forma, podemos utilizar `phi` para investigar os tópicos mais relevantes para as palavras “católica” e “católico”:

```
tmod_discursos$phi |>
  as_tibble(rownames = "Tópico") |>
  select(Tópico, católica, católico)
```

```
# A tibble: 10 x 3
  Tópico      católica católico
  <chr>      <dbl>      <dbl>
1 topic1  0.00000626 0.00000626
2 topic2  0.00000724 0.00000724
3 topic3  0.00000660 0.00000660
4 topic4  0.00307    0.000811
5 topic5  0.00000426 0.00000426
6 topic6  0.00000675 0.00000675
7 topic7  0.00000532 0.00000532
8 topic8  0.00000653 0.00000653
9 topic9  0.00000482 0.00000482
10 topic10 0.00000740 0.00000740
```

4.4 Modelos de tópicos: análise de tópicos guiada

A análise de tópicos com LDA descobre tópicos de forma automática e não supervisionada com base nas frequências de palavras nos documentos.

No entanto, às vezes, precisamos orientar a análise para focar em temas específicos ou para obter tópicos mais interpretáveis.

A **análise guiada de tópicos** (em inglês, *guided topic analysis* ou *seeded topic analysis*) é uma abordagem que permite incorporar conhecimento prévio ou restrições na modelagem de tópicos.

Neste método, precisamos fornecer um conjunto de palavras-chave “sementes” para cada tópico potencial.

O modelo usa estas palavras-chave para orientar a descoberta de tópicos mais alinhados com os temas de interesse.

4.4.1 Exemplo de análise de tópicos guiada com `seededlda`

Para especificar as sementes, construímos um objeto dicionário da biblioteca `quanteda`:

```
topicos <- list(
  "golpe" = c("anistia", "golp*", "ditadura", "stf", "janeiro"),
  "papa" = c("papa", "francisco", "igreja", "católica"),
  "glauber" = c("glauber", "cassação", "cpi", "ética", "mandato")
)
```

```
dicionario_topicos <- dictionary(topicos)
dicionario_topicos
```

Dictionary object with 3 key entries.

```
- [golpe]:
  - anistia, golp*, ditadura, stf, janeiro
- [papa]:
  - papa, francisco, igreja, católica
- [glauber]:
  - glauber, cassação, cpi, ética, mandato
```

As sementes são passadas como segundo argumento da função `textmodel_seededlda`:

```
set.seed(42)
stmod_discursos <- textmodel_seededlda(tm_dfm, dicionario_topicos)

terms(stmod_discursos)
```

	golpe	papa	glauber
[1,]	"golpe"	"francisco"	"glauber"
[2,]	"stf"	"papa"	"mandato"
[3,]	"pec"	"igreja"	"nº"
[4,]	"esquerda"	"deus"	"desenvolvimento"
[5,]	"querem"	"município"	"renda"
[6,]	"supremo"	"mulheres"	"economia"
[7,]	"votar"	"prefeito"	"fundo"
[8,]	"judiciário"	"autismo"	"acre"
[9,]	"inss"	"bahia"	"ética"
[10,]	"tribunal"	"direitos"	"acesso"

De forma similar ao LDA, utilizamos `topics` para determinar o tópico de cada documento:

```
topics(stmod_discursos) |>
  as.data.frame() |>
  head(20)
```

	topics(stmod_discursos)
1_Acácio Favacho_MDB	glauber
2_Acácio Favacho_MDB	papa
3_Acácio Favacho_MDB	glauber

4_Adail Filho_REPUBLICANOS	glauber
5_Adilson Barroso_PL	golpe
6_Adilson Barroso_PL	golpe
7_Adilson Barroso_PL	golpe
8_Adriana Ventura_NOVO	golpe
9_Adriana Ventura_NOVO	golpe
10_Adriana Ventura_NOVO	golpe
11_Adriana Ventura_NOVO	golpe
12_Adriana Ventura_NOVO	papa
13_Adriana Ventura_NOVO	golpe
14_Adriana Ventura_NOVO	glauber
15_Adriana Ventura_NOVO	golpe
16_Adriana Ventura_NOVO	glauber
17_Adriana Ventura_NOVO	glauber
18_Adriana Ventura_NOVO	golpe
19_Adriana Ventura_NOVO	glauber
20_Adriana Ventura_NOVO	golpe

4.5 Para saber mais

Para saber mais sobre análise sintática, gramáticas e relações de dependência, veja os capítulos 17, 18 e 19 do livro *Speech and Language Processing* (Jurafsky and Martin 2026).

Sobre análise de tópicos e LDA, leia o capítulo 6 do livro (Silge and Robinson 2017).

Para uma discussão sobre análise de tópicos guiada em estudos sociais, veja o artigo de Schweinberger (2025).

4.6 Estudos guiados

4.6.1 Inicialização do Spacy para Português

```
library(spacyr)

# Instalar spacy (se necessário) e baixar modelo em português
# spacy_install()
# spacy_download_langmodel("pt_core_news_sm")

# Inicializar o spacy com o modelo de língua portuguesa
spacy_initialize(model = "pt_core_news_sm")
```

spaCy is already initialized

NULL

```
print(reticulate::py_config())
```

```
python:          /home/lucasmp/.virtualenvs/r-spacyr/bin/python
libpython:       /home/lucasmp/.pyenv/versions/3.10.12/lib/libpython3.10.so
pythonhome:      /home/lucasmp/.virtualenvs/r-spacyr:/home/lucasmp/.virtualenvs/r-
spacyr
version:         3.10.12 (main, Aug 23 2023, 07:28:31) [GCC 12.2.0]
numpy:           /home/lucasmp/.virtualenvs/r-spacyr/lib/python3.10/site-
packages/numpy
numpy_version:   2.2.6
```

NOTE: Python version was forced by use_python() function

4.6.2 Análise de classes gramaticais (POS Tagging)

```
library(tidyverse)
library(quantda)
library(spacyr)

spacy_initialize(model = "pt_core_news_sm")
```

spaCy is already initialized

NULL

```
nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados e criar corpus (amostra para rapidez)
corpus_amostra <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  head(50) |> # limitar a 50 discursos para este exemplo
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_")) |>
  corpus(text_field = "transcricao")

# Realizar análise sintática (extraíndo classes gramaticais e lemas)
# Resultado é uma tabela, com um token por linha, o lemma e sua classe gramatical (pos)
analise_pos <- spacy_parse(corpus_amostra, pos = TRUE, lemma = TRUE, entity = FALSE)
```

Warning in spacy_parse.character(corpus_amostra, pos = TRUE, lemma = TRUE, :
lemmatization may not work properly in model 'pt_core_news_sm'

```
# Filtrar e contar os substantivos próprios (PROPN) mais comuns
top_propn <- analise_pos |>
  filter(pos == "PROPN") |>      # filtra linhas de substantivos próprios
  count(token, sort = TRUE) |>  # conta cada token e ordena (maior para menor)
  head(20)                      # filtra 20 primeiras linhas

print(top_propn)
```

	token	n
1	Governo	52
2	Brasil	49
3	SR	41
4	Bloco	39
5	Presidente	36
6	Casa	24
7	"	22
8	Obrigada	20
9	PL	20
10	Lula	19
11	SP	19
12	Federal	17
13	\u0097	17
14	RS	16
15	Sul	16
16	ADRIANA	15
17	AFONSO	15
18	NOVO	15
19	Nacional	15
20	SRA	15

4.6.3 Extraíndo Relações de Dependência (Atores e Ações)

```
library(tidyverse)
library(quanteda)
library(spacyr)

spacy_initialize(model = "pt_core_news_sm")
```

spaCy is already initialized

NULL

```
nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados e criar corpus
corpus_amostra <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  head(50) |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_")) |>
  corpus(text_field = "transcricao")

# Parse com relações de dependência
analise_dep <- spacy_parse(corpus_amostra, dependency = TRUE, pos = TRUE)
```

Warning in spacy_parse.character(corpus_amostra, dependency = TRUE, pos = TRUE): lemmatization may not work properly in model 'pt_core_news_sm'

```
# Criar tabela com verbos
acoes <- analise_dep |>
  filter(dep_rel == "ROOT", pos == "VERB") |>
  select(doc_id, sentence_id, token_id, verbo = lemma)

atores <- analise_dep |>
  filter(dep_rel == "nsubj", pos %in% c("NOUN", "PROPN")) |>
  select(doc_id, sentence_id, ator = lemma, head_token_id)

# Criar tabela unindo sujeitos (atores) e seus respectivos verbos (ações)
# unir (inner_join) verbos e atores do mesmo documento (doc_id), frase (sentence_id) e onde
atores_acoes <- atores |>
  inner_join(acoes, by = c("doc_id", "sentence_id", "head_token_id" = "token_id"))

# Criar tabela com 10 verbos principais mais frequentes
top_acoes <- acoes |>
  count(verbo, sort = TRUE) |>
  head(10)

# Criar tabela com 10 substantivos principais mais frequentes
top_atores <- atores |>
  count(ator, sort = TRUE) |>
  head(10)
```



```
# Criar tabela com 10 combinações ação-ator mais comuns
top_acoes_atores <- atores_acoes |>
  count(ator, verbo, sort = TRUE) |>
  head(10)

# Exibir os primeiros pares ator-ação encontrados
print("Principais ações:")
```

```
[1] "Principais ações:"
```

```
print(top_acoes)
```

	verbo	n
1	querer	46
2	ter	40
3	achar	20
4	falar	20
5	saber	19
6	fazer	18
7	precisar	18
8	poder	16
9	dar	15
10	haver	14

```
print("Principais atores:")
```

```
[1] "Principais atores:"
```

```
print(top_atores)
```

	ator	n
1	gente	29
2	presidente	19
3	Governo	17
4	Brasil	15
5	Presidente	11
6	País	9
7	mundo	9

```
8     educação  8
9     projeto   7
10    Deputado  6
```

```
print("Principais ações-atores:")
```

```
[1] "Principais ações-atores:"
```

```
print(top_acoes_atores)
```

	ator	verbo	n
1	gente	saber	4
2	presidente	querer	3
3	Brasil	precisar	2
4	agro	comprometer	2
5	dado	mostrar	2
6	dança	pedir	2
7	mundo	achar	2
8	mundo	saber	2
9	BR-163	faltar	1
10	Brasil	crescer	1

4.6.4 Modelagem de Tópicos (LDA)

```
library(tidyverse)
library(quanteda)
library(seededlda)
library(stopwords)

nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados e criar corpus
corpus_discursos <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_")) |>
  corpus(text_field = "transcricao")

# Criar tokens
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE, remove_symbols = TRUE) |>
```

```

tokens_remove(stopwords("portuguese"))

# Criar DFM e aplicar filtros de frequência (80% das palavras mais frequentes, máximo de 50%
dfm_discursos <- tokens_discursos |>
  dfm() |>
  dfm_trim(
    min_termfreq = 0.8, termfreq_type = "quantile",
    max_docfreq = 0.1, docfreq_type = "prop"
  )

# Treinar modelo LDA com 5 tópicos
set.seed(123) # semente para reprodutibilidade
modelo_lda <- textmodel_lda(dfm_discursos, k = 5)

# Criar tabela com os 10 principais termos de cada tópico
termos_topicos <- modelo_lda |>
  terms(10) |>
  as.data.frame()

# Imprimir termos
print(termos_topicos)

```

	topic1	topic2	topic3	topic4	topic5
1	direitos	desenvolvimento	matéria	glauber	francisco
2	crianças	renda	voto	esquerda	papa
3	autismo	acre	pauta	golpe	deus
4	sistema	região	fundo	querem	município
5	pec	economia	votar	supremo	prefeito
6	inclusão	trabalhadores	urgência	inss	bahia
7	nº	ambiental	nº	direita	santa
8	combate	imposto	hugo	aposentados	igreja
9	profissionais	infraestrutura	votação	stf	parabéns
10	públicas	indígenas	texto	ética	governador

```

# Atribuir o tópico mais provável a cada discurso
topicos_10_docs <- modelo_lda |>
  topics() |>
  as.data.frame() |>
  head(10)

# Imprimir tópicos dos discursos
print(topicos_10_docs)

```

	topics(modelo_lda)
1_Acácio Favacho_MDB	topic2
2_Acácio Favacho_MDB	topic1
3_Acácio Favacho_MDB	topic2
4_Adail Filho_REPUBLICANOS	topic2
5_Adilson Barroso_PL	topic4
6_Adilson Barroso_PL	topic3
7_Adilson Barroso_PL	topic4
8_Adriana Ventura_NOVO	topic3
9_Adriana Ventura_NOVO	topic5
10_Adriana Ventura_NOVO	topic4

4.6.5 Modelagem de Tópicos Guiada (Seeded LDA)

```
library(tidyverse)
library(quanteda)
library(seededlda)
library(stopwords)

nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados e criar corpus
corpus_discursos <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_")) |>
  corpus(text_field = "transcricao")

# Criar tokens
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE, remove_symbols = TRUE) |>
  tokens_remove(stopwords("portuguese"))

# Criar DFM
dfm_discursos <- tokens_discursos |>
  dfm() |>
  dfm_trim(
    min_termfreq = 0.8, termfreq_type = "quantile",
    max_docfreq = 0.1, docfreq_type = "prop"
  )

# Definir sementes para tópicos específicos
sementes <- dictionary(list(
```

```

economia = c("inflação", "juros", "economia", "mercado", "fiscal"),
seguranca = c("polícia", "crime", "segurança", "violência", "presídio"),
saude = c("saúde", "hospital", "sus", "médico", "vacina")
))

# Treinar Seeded LDA
set.seed(123) # semente para reprodutibilidade
modelo_seeded <- textmodel_seededlda(dfm_discursos, dictionary = sementes)

# Criar tabela com os 10 principais termos de cada tópico (incluindo os descobertos pelo modelo)
termos_topicos <- modelo_seeded |>
  terms(10) |>
  as.data.frame()

# Imprimir termos de cada tópico
print(termos_topicos)

```

	economia	seguranca	saude
1	economia	francisco	pec
2	nº	violência	glauber
3	desenvolvimento	papa	esquerda
4	renda	mulheres	golpe
5	fiscal	deus	querem
6	acre	município	supremo
7	fundo	prefeito	judiciário
8	sul	autismo	votar
9	mercado	parabéns	inss
10	acesso	bahia	pauta

```

# Ver o tópico mais provável para os primeiros 10 documentos
topicos_seeded_10_docs <- topics(modelo_seeded) |>
  as.data.frame() |>
  head(10)

# Imprimir tópicos dos primeiros 10 discursos
print(topicos_seeded_10_docs)

```

	topics(modelo_seeded)
1_Acácio Favacho_MDB	economia
2_Acácio Favacho_MDB	seguranca
3_Acácio Favacho_MDB	economia

4_Adail Filho_REPUBLICANOS	economia
5_Adilson Barroso_PL	saude
6_Adilson Barroso_PL	saude
7_Adilson Barroso_PL	saude
8_Adriana Ventura_NOVO	saude
9_Adriana Ventura_NOVO	saude
10_Adriana Ventura_NOVO	saude

4.7 Atividade prática

Utilizando o corpus de discursos de abril de 2025 da câmara:

- Utilize o procedimento para identificar atores e ações no corpus dos discursos da câmara e identifique os principais verbos relacionados ao governo.
- Realize a análise de tópicos LDA com 4 e 15 tópicos
- Crie um dicionário com palavras-chave de outros temas e realize a análise de tópicos guiada

5 Análise de tópicos estruturada e modelos de escala

5.1 Análise de tópicos estruturada

Na LDA assumimos que os tópicos advêm de uma mistura de palavras, mas não levamos em consideração outras informações que podem influenciar ou relevar determinados tópicos.

Outra extensão dos modelos de tópicos LDA são os Modelos de Tópicos Estruturados (STM, do inglês **structured topic models**). Este método incorpora informações adicionais (metadados, **docvars**) do texto para orientar a análise.

Tip

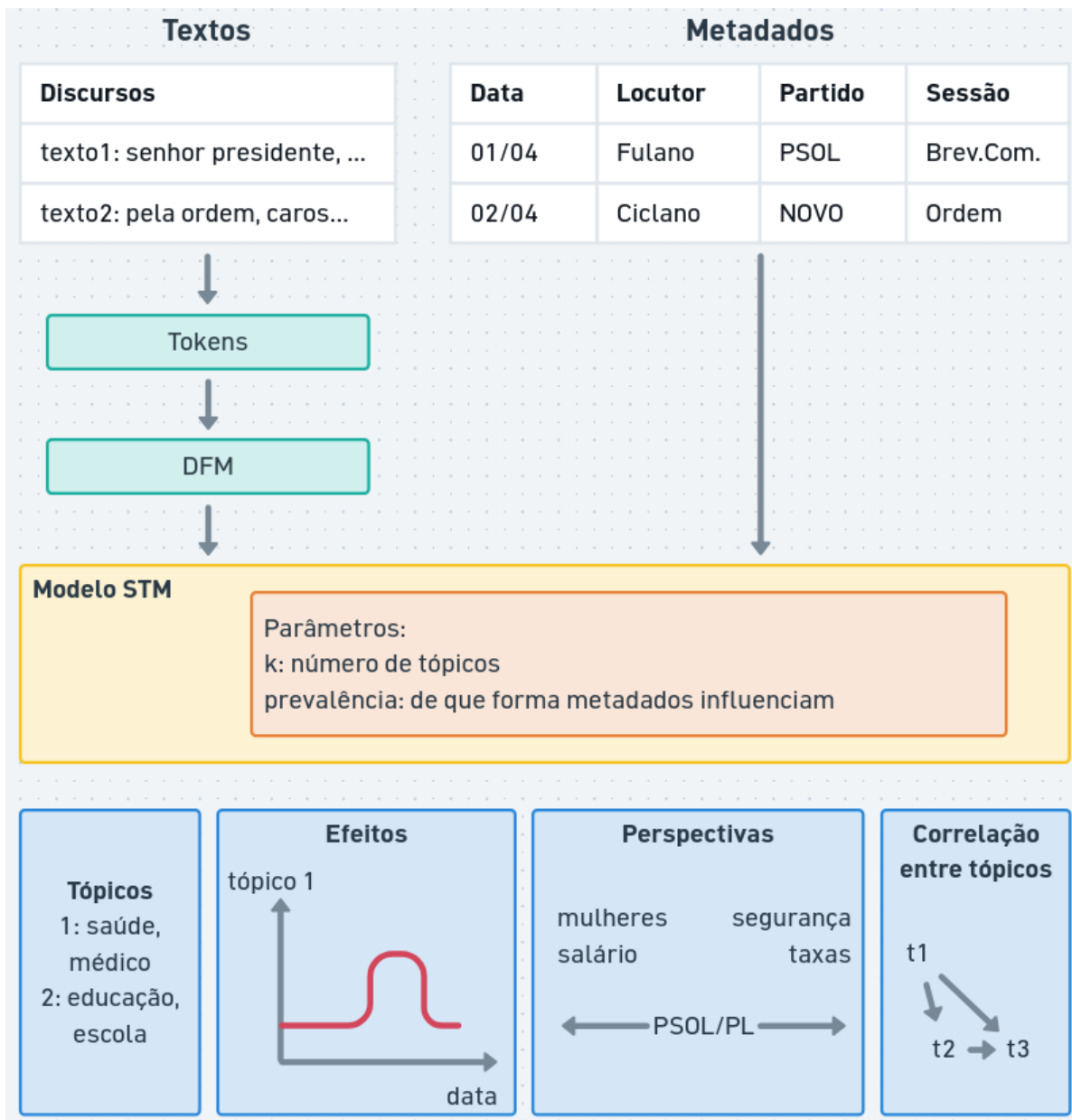
Quais outras variáveis que influenciam os tópicos dos discursos políticos?

Algumas estruturas que podem influenciar análise de discurso político seriam, por exemplo:

- Data do discurso: Tópicos podem variar ao longo do tempo, e a data que o discurso foi proferido pode ajudar a identificar tendências ou mudanças nos temas discutidos.
- Locutor: diferentes políticos podem ter interesses ou perspectivas diferentes, e isso pode influenciar os temas debatidos.
- Partido político: os partidos podem ter plataformas ou agendas diferentes, e isso pode influenciar os temas discutidos nos discursos dos seus membros.

Modelos de tópicos estruturados permitem:

- incorporar informações qualitativas na análise
- examinar o **efeito estimado** de uma ou mais variáveis nos tópicos: como uma variável influencia nos tópicos (ex: como o tema “saúde” é abordado ao longo do tempo)
- examinar **perspectivas**: como palavras mudam de acordo com uma variável ou entre dois tópicos (ex: como partidos de esquerda e direita tratam o tema “segurança”)
- modelar **correlação entre tópicos** (em LDA, os tópicos geralmente são independentes)



Algumas fragilidades do STM, e também do LDA, são:

- utilizam a abordagem “bag-of-words” (saco de palavras), ou seja, não há informação sobre a ordem, sintaxe ou semântica das palavras, apenas quantas vezes elas foram utilizadas em cada discurso
- as escolhas de pré-processamento, número de tópicos, variáveis dependentes e interpretação dos termos é experimental e sujeitas à interpretação

Assim como em outros modelos, o resultado mostra uma visão ampla dos dados, cabe ao pesquisador investigar a fundo os discursos relevantes.

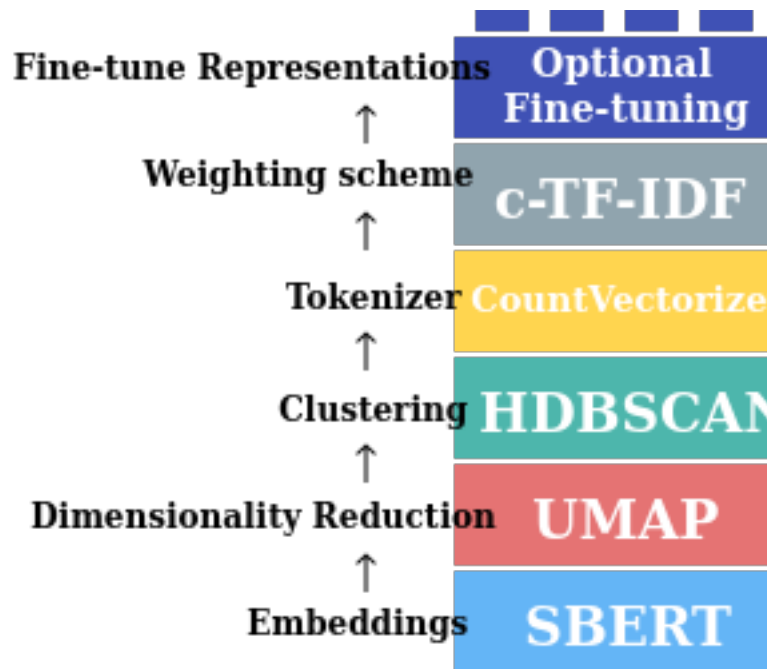
5.2 BERTopic

O **BERTopic** é um modelo de tópicos baseado em embeddings, que utiliza técnicas de aprendizado profundo para gerar representações vetoriais dos textos (*embeddings*).

Embeddings podem capturar relações semânticas entre palavras e documentos, o que pode levar a uma melhor identificação de tópicos.

Seu funcionamento é baseado em três etapas principais:

1. Geração de embeddings: os textos são convertidos em vetores de alta dimensão utilizando modelos pré-treinados, como BERT ou outros modelos de linguagem.
2. Redução de dimensionalidade: os embeddings são reduzidos para um espaço de menor dimensão utilizando técnicas como (UMAP), facilitando a visualização e a identificação de tópicos.
3. Agrupamento: os vetores reduzidos são agrupados utilizando algoritmos de clustering, como HDBSCAN, para identificar os tópicos presentes nos textos.



As capacidades do BERTopic incluem:

- *Tópicos guiados*: permite incorporar palavras-chave ou rótulos para orientar a identificação de tópicos.
- *Tópicos supervisionados*: permite incorporar rótulos de tópicos pré-definidos para orientar a análise. Por exemplo: partidos, onde o modelo constrói palavras pertinentes de acordo com os discursos.
- *Tópicos dinâmicos*: permite analisar a evolução dos tópicos ao longo do tempo, identificando como os temas mudam e se desenvolvem.
- *Tópicos hierárquicos*: permite identificar sub-tópicos dentro de tópicos maiores, criando uma estrutura hierárquica de temas.
- *Tópicos multimodais*: permite incorporar diferentes tipos de dados (texto, imagens, etc) para identificar tópicos que abrangem múltiplas modalidades.

Desvantagens de modelos de tópicos baseados em embeddings incluem:

- menor interpretabilidade: os tópicos gerados podem ser mais difíceis de interpretar, especialmente se os embeddings não capturam bem as nuances semânticas dos textos.
- maior complexidade computacional: requer mais recursos computacionais para gerar embeddings e realizar clustering.
- configuração mais complexa: a escolha do modelo de linguagem, técnicas de redução de dimensionalidade e algoritmos de clustering pode ser mais complexa e exigir mais experimentação.
- geralmente requerem mais dados para treinar adequadamente, o que pode ser um desafio em conjuntos de dados menores.

5.3 Escalonamento de texto

Escalonamento de texto (text scaling) é a tarefa de derivar posições para textos ou autores ao longo de um contínuo ideológico ou temático.



Exemplos de aplicações em análise de conteúdo:

- estimativa de posição ideológica: de partidos/políticos em um espectro de “progressista/esquerda” e “conservador/direita”
- medir polarização política ao longo do tempo: como o conjunto de discursos são posicionados entre mais ou menos polarizados

- analisar coesão de bancada: como os integrantes de um partido estão alinhados ideologicamente dentro de um partido
- mudanças de agenda: como os discursos de um político ou grupo se alteram ao longo do tempo
- enquadramento de assuntos: como um assunto é tratado por diferentes discursos (ex: reforma tributária é tratada como “carga tributária” ou “justiça fiscal”)

Existem diversos métodos de estimar posicionamentos a partir de textos.

5.3.1 Wordscore

O modelo **wordscore** é um método de escalonamento **supervisionado** de texto que atribui uma pontuação a cada palavra com base em sua frequência em um conjunto de textos de referência (seed).

O conjunto de referência possui uma pontuação (score) definido *a priori* pelo pesquisador. O modelo então estima a pontuação de outros textos não-pontuados (virgin), com base nas palavras contidas.

As pontuações de cada palavra são estimadas com probabilidade condicional Bayesiana simples. Cada texto é estimado com a média ponderada da pontuação de cada palavra.

5.3.2 Wordfish

Modelos **wordfish** são **não-supervisionados**, ou seja, não necessitam de um conjunto de referência. Eles estimam a posição ideológica dos textos com base na frequência das palavras, assumindo que as palavras mais frequentes em um texto são mais representativas do seu conteúdo.

O modelo estatístico do wordfish é baseado em um modelo de Poisson mais sofisticado, onde a frequência de cada palavra em um texto é modelada como uma função da posição ideológica do texto e da importância da palavra.

5.3.3 Análise de correspondência

A **análise de correspondência** (CA, do inglês *correspondence analysis*) é uma técnica para redução de dimensionalidade que funciona com variáveis categóricas (tabela de contingência).

No caso de dados textuais, podemos aplicar a análise de correspondência a uma matriz de frequência de palavras, onde as linhas representam os textos e as colunas representam as palavras (transposta da DFM).

A análise de correspondência pode ser utilizada para identificar padrões de coocorrência entre palavras e textos, e para visualizar a relação entre eles em um espaço bidimensional.

5.4 Similaridade textual

Uma tarefa comum em análise de conteúdo é medir a **similaridade entre textos**, ou seja, o quão semelhantes são dois ou mais textos em termos de conteúdo.

Existem diversas métricas para medir a similaridade textual, como:

- *Correspondência de palavras*: textos similares utilizam as mesmas palavras.

$$C(A, B) = |A \cap B|$$

- *Distância de Jaccard*: mede a similaridade entre dois conjuntos de palavras, calculando a razão entre a interseção e a união dos conjuntos.

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

- *Distância euclidiana*: mede distância entre dois vetores de palavras, calculando a raiz quadrada da soma das diferenças ao quadrado.

$$E(A, B) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

- *Similaridade de cosseno*: mede a similaridade entre dois vetores de palavras, calculando o cosseno do ângulo entre eles. Quanto mais próximo de 1, mais semelhantes são os textos.

$$S(A, B) = \cos(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

A **similaridade de cosseno** é muito utilizada em análise de conteúdo, especialmente quando os textos são representados como vetores de palavras (por exemplo, usando contagens, TF-IDF ou embeddings). Ela é útil para identificar textos que são semanticamente semelhantes, mesmo que não compartilhem muitas palavras em comum.

O resultado da similaridade textual pode ser utilizado para diversas análises, como:

- Agrupamento de textos: textos similares podem ser agrupados para identificar temas ou tópicos comuns.
- Análise de redes: textos similares podem ser conectados em uma rede para visualizar as relações entre eles.
- Busca de documentos: textos similares podem ser utilizados para encontrar documentos relevantes em um grande conjunto de dados.

6 Estudos guiados

6.1 STM

O STM permite incorporar metadados. Aqui, usaremos o `partido` como uma variável que influencia a prevalência dos tópicos (o quanto cada partido fala de cada tema).

```
library(tidyverse)
library(quanteda)
library(stm)
library(stopwords)

nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados e criar corpus
corpus_discursos <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  filter(partido %in% c("PT", "PL", "PSOL", "REPUBLICANOS")) |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_")) |>
  corpus(text_field = "transcricao")

# Criar tokens
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE, remove_symbols = TRUE) |>
  tokens_remove(stopwords("portuguese"))

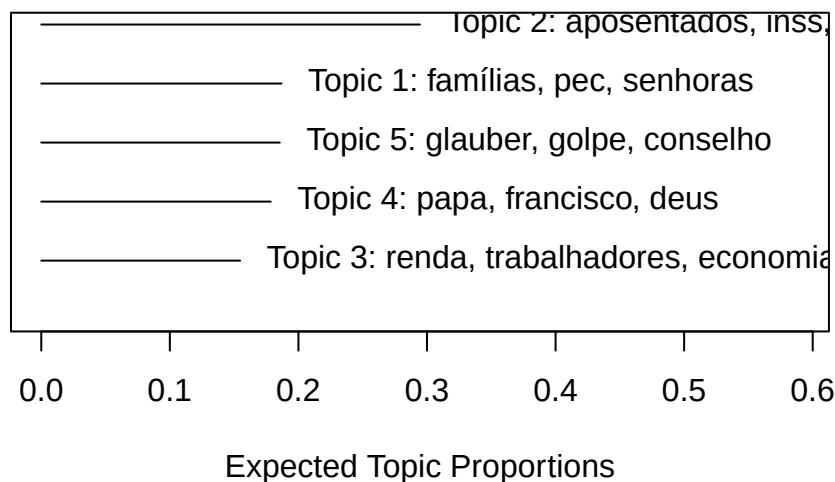
# Criar DFM, removendo palavras muito frequentes e muito raras
dfm_discursos <- tokens_discursos |>
  dfm() |>
  dfm_trim(
    min_termfreq = 0.8, termfreq_type = "quantile",
    max_docfreq = 0.1, docfreq_type = "prop"
  )

# Converter DFM do quanteda para o formato esperado pelo pacote STM
stm_data <- convert(dfm_discursos, to = "stm")
```

```
# Treinar o modelo STM
# K = 5 tópicos para este exemplo
# prevalence =~ partido: indica que a escolha do tópico depende do partido
modelo_stm <- stm(
  documents = stm_data$documents,
  vocab = stm_data$vocab,
  data = stm_data$meta,
  K = 5,
  prevalence = ~partido + fase_evento,
  verbose = FALSE
)

# Visualizar resumo dos tópicos e sua frequência relativa
plot(modelo_stm, type = "summary", main = "Principais Tópicos (Modelo STM)")
```

Principais Tópicos (Modelo STM)



Continuando com o mesmo modelo, podemos investigar os textos mais pertinentes a cada tópico:

```
library(stm)
```

```
stm v1.3.8 successfully loaded. See ?stm for help.
Papers, resources, and other materials at structuraltopicmodel.com
```

```
# Imprimir os 2 textos mais relevantes do tópico 5
findThoughts(
  modelo_stm,
  texts = as.character(corpus_discursos), # vetor de textos
  topics = c(4), # tópicos
  n = 2 # número de textos
)
```

Topic 4:

A SRA. CHRIS TONIETTO (Bloco/PL - RJ. Sem revisão da oradora.) - Muito obrigada, Sr. Presidente. Quero cumprimentar também a indígena pataxó que está aqui. Como muito bem disse o Presidente.

O SR. DIEGO GARCIA (Bloco/REPUBLICANOS - PR. Sem revisão do orador.) - Bom dia a todos. O Brasil vai até o fim, porque o final vai ser com vitória. Eu me lembro, no início da minha caminhada política.

Estimando efeitos da variável dependente categórica (partidos) sobre um determinado tópico. O gráfico mostra proporção (e sua variância) que um tópico é coberto em cada partido.

```
library(quanteda)
```

Package version: 4.3.1

Unicode version: 15.0

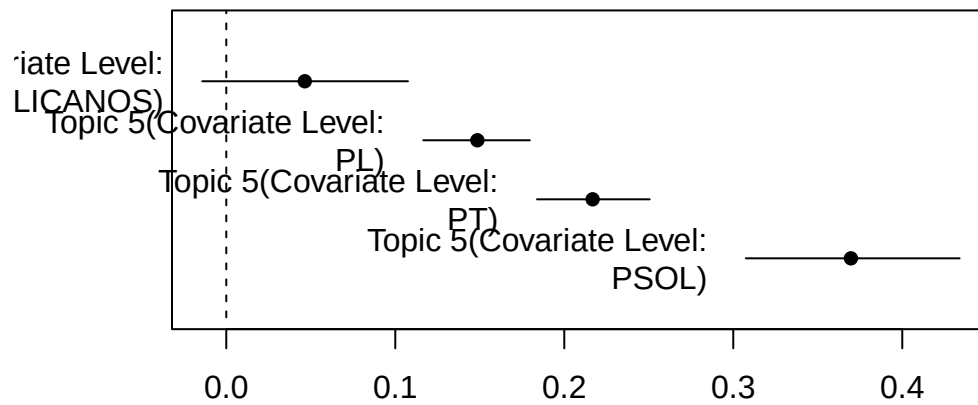
ICU version: 72.1

Parallel computing: 12 of 12 threads used.

See <https://quanteda.io> for tutorials and examples.

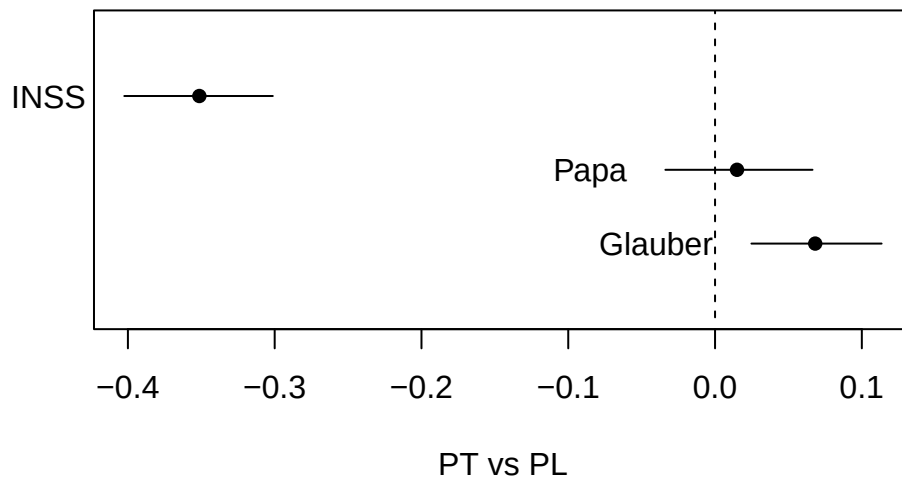
```
ee_partido <- estimateEffect(
  ~partido,
  modelo_stm,
  documents = stm_data,
  metadata = docvars(corpus_discursos)
)

plot.estimateEffect(
  ee_partido,
  covariate = "partido",
  method = "pointestimate",
  topics = 5
)
```



Também podemos analisar a diferença entre dois polos (ex PT vs PL) em relação a cada tópico. Intervalos próximos de 0 (zero) são temas citados por ambos.

```
plot.estimateEffect(
  ee_partido,
  covariate = "partido",
  method = "difference",
  cov.value1 = "PT",
  cov.value2 = "PL",
  topics = c(2, 4, 5),
  labeltype = "custom",
  custom.labels = c("INSS", "Papa", "Glauber"),
  xlab = "PT vs PL",
)
```

6.2 BERTopic

O estudo guiado está disponível no Google Colab em: <https://colab.research.google.com/drive/1ZhOSzNBMEZWP5MVLnginDWyPEYSnLbQl?usp=sharing>

6.3 Escalonamento não-supervisionado (Wordfish)

O Wordfish estima as posições ideológicas dos documentos apenas com base na frequência das palavras, sem precisar de rótulos prévios.

```
library(tidyverse)
library(quanteda)
library(quanteda.textmodels)
library(quanteda.textplots)
library(stopwords)

nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados e criar corpus
corpus_discursos <- read_csv(nome_arquivo, show_col_types = FALSE) |>
```

```

filter(partido %in% c("PT", "PL", "PSOL", "REPUBLICANOS")) |>
mutate(doc_id = str_c(row_number(), nome, partido, sep = "_")) |>
corpus(text_field = "transcricao")

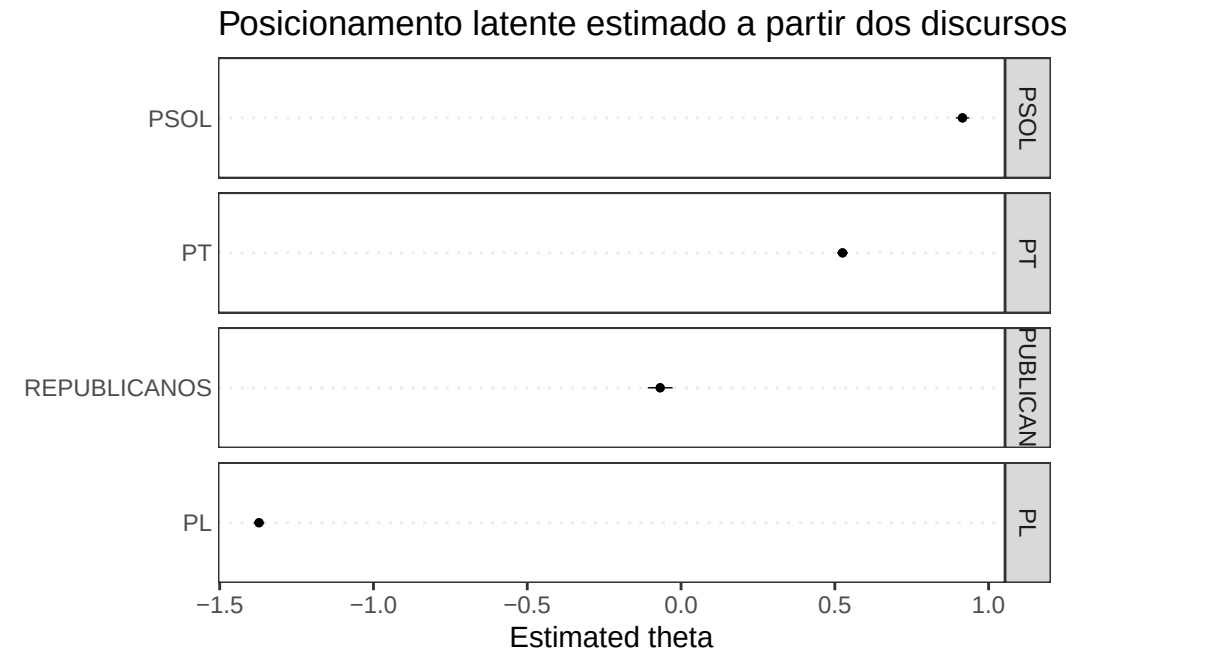
# Criar tokens
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE, remove_symbols = TRUE) |>
  tokens_remove(stopwords("portuguese"))

# Criar DFM, removendo palavras muito frequentes e muito raras
dfm_discursos <- tokens_discursos |>
  dfm() |>
  dfm_trim(
    min_termfreq = 0.8, termfreq_type = "quantile",
    max_docfreq = 0.1, docfreq_type = "prop"
  ) |>
  dfm_group(partido)

# Treinar modelo Wordfish
set.seed(42)
modelo_wf <- textmodel_wordfish(dfm_discursos)

# Visualizar o posicionamento médio dos partidos na dimensão estimada
textplot_scale1d(modelo_wf, groups = dfm_discursos$partido) +
  labs(title = "Posicionamento latente estimado a partir dos discursos")

```



6.4 Escalonamento supervisionado (Wordscores)

No Wordscores, definimos pontuações para documentos de referência (ex: PT como esquerda e PL como direita) e o modelo estima a posição dos demais.

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.0      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.2      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.1

-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(quanteda)
library(quanteda.textmodels)
```

```

library(quantda.textplots)
library(stopwords)

nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados
dados_discursos <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_"))

# Criar corpus
corpus_discursos <- dados_discursos |>
  corpus(text_field = "transcricao")

# Criar tokens
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE, remove_symbols = TRUE) |>
  tokens_remove(stopwords("portuguese"))

# Criar DFM, removendo palavras muito frequentes e muito raras
dfm_discursos <- tokens_discursos |>
  dfm() |>
  dfm_trim(
    min_termfreq = 0.8, termfreq_type = "quantile",
    max_docfreq = 0.1, docfreq_type = "prop"
  ) |>
  dfm_group(partido)

# Criar vetor com referências
mapeamento <- c(
  "PSOL" = -1,
  "PT" = -1,
  "MDB" = 0,
  "PP" = 0.5,
  "NOVO" = 1,
  "PL" = 1
)
scores <- mapeamento[docid(dfm_discursos)]

# Treinar o modelo Wordscores
modelo_ws <- textmodel_wordscores(dfm_discursos, y = scores)

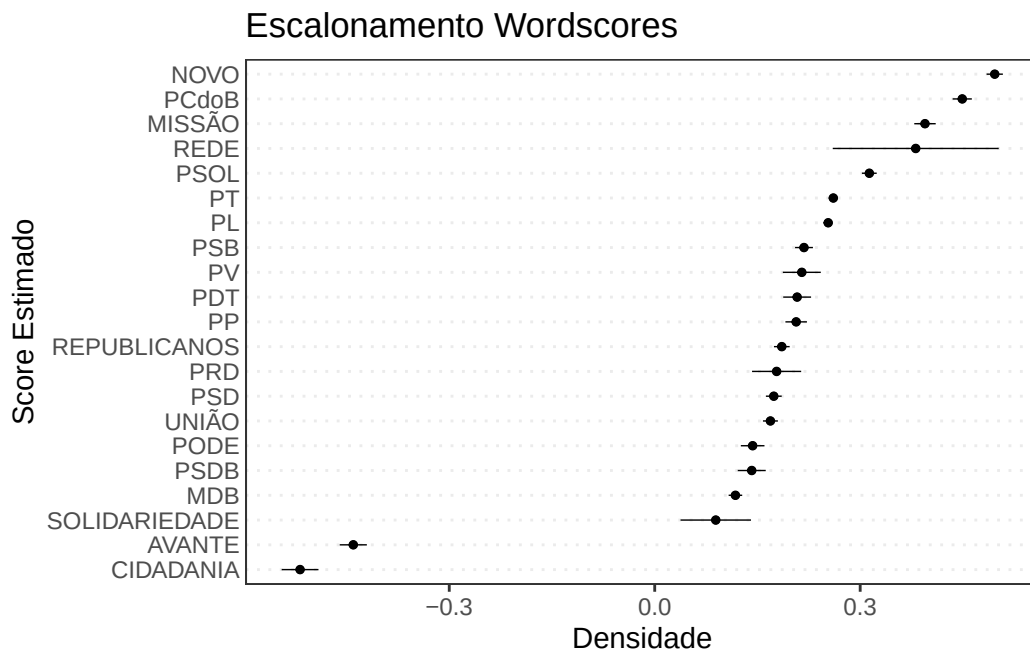
# Predizer posições para todos os discursos

```

```
pred_ws <- predict(modelo_ws, se.fit = TRUE)
```

Warning: 1037 features in newdata not used in prediction.

```
# Visualizar resultados agrupados por partido
textplot_scale1d(pred_ws) +
  labs(title = "Escalonamento Wordscores",
       x = "Score Estimado", y = "Densidade")
```



6.5 Análise de Correspondência (CA)

A CA é uma técnica de redução de dimensionalidade que permite visualizar a “distância” entre categorias (ex partidos) e palavras em um espaço comum.

```
library(tidyverse)
library(quanteda)
library(quanteda.textmodels)
library(quanteda.textplots)
library(stopwords)
```

```

nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados
dados_discursos <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_"))

# Criar corpus
corpus_discursos <- dados_discursos |>
  corpus(text_field = "transcricao")

# Criar tokens
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE, remove_symbols = TRUE) |>
  tokens_remove(stopwords("portuguese"))

# Criar DFM, removendo palavras muito frequentes e muito raras
dfm_discursos <- tokens_discursos |>
  dfm() |>
  dfm_trim(
    min_termfreq = 0.8, termfreq_type = "quantile",
    max_docfreq = 0.1, docfreq_type = "prop"
  ) |>
  dfm_group(partido)

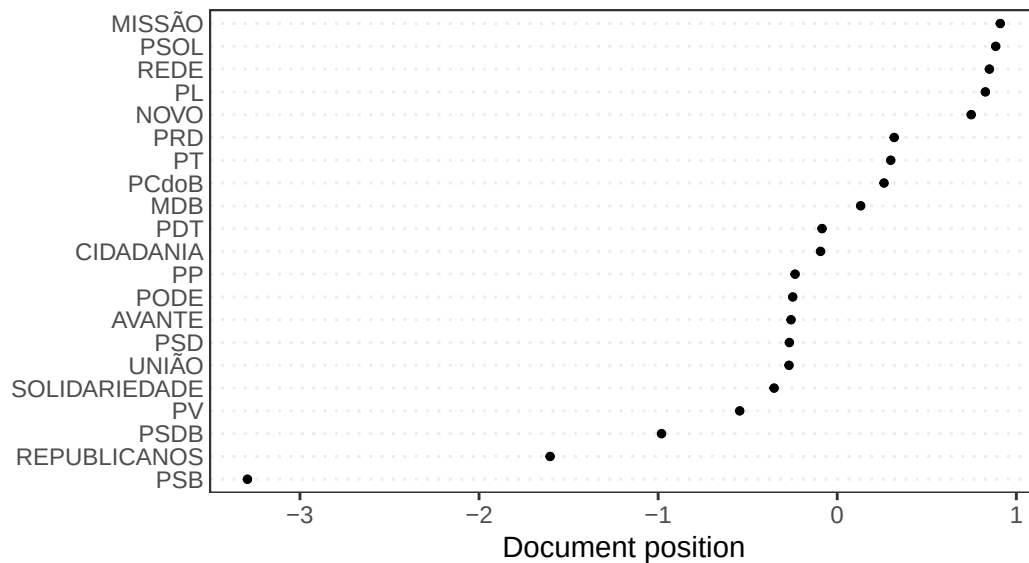
# Executar Análise de Correspondência
modelo_ca <- textmodel_ca(dfm_discursos)

# Visualizar a primeira dimensão da CA
textplot_scale1d(modelo_ca) +
  labs(title = "Análise de Correspondência: 1ª Dimensão",
       subtitle = "Agrupamento por partido")

```

Análise de Correspondência: 1ª Dimensão

Agrupamento por partido

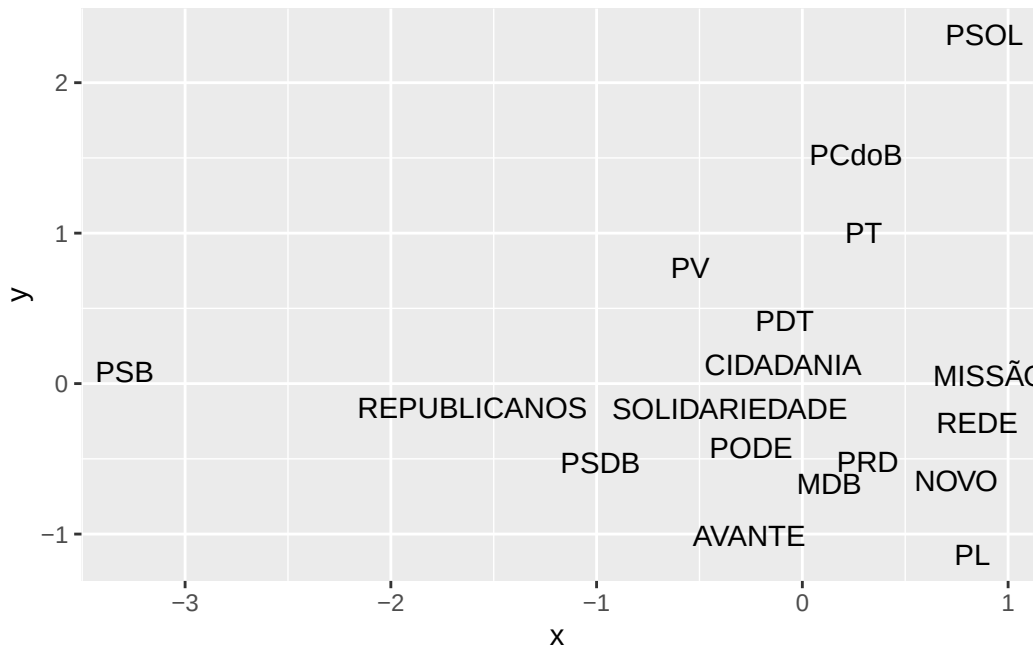


CA permite extrair outras dimensões do modelo:

```
coefs_ca_2d <- data_frame(  
  partido = modelo_ca$rownames,  
  x = coef(modelo_ca, doc_dim = 1)$coef_document,  
  y = coef(modelo_ca, doc_dim = 2)$coef_document  
)
```

Warning: `data\_frame()` was deprecated in tibble 1.1.0.
i Please use `tibble()` instead.

```
ggplot(coefs_ca_2d) +  
  geom_text(aes(x = x, y = y, label = partido), position = "nudge",  
            check_overlap = TRUE)
```



6.6 Similaridade Textual

A similaridade de cosseno mede o quão parecido é o vocabulário utilizado por diferentes grupos.

```
library(tidyverse)
library(quanteda)
library(quanteda.textstats)
library(quanteda.textplots)
library(stopwords)

nome_arquivo <- "discursos_camara_abril_2025.csv"

# Carregar dados
dados_discursos <- read_csv(nome_arquivo, show_col_types = FALSE) |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_"))

# Criar corpus
corpus_discursos <- dados_discursos |>
  corpus(text_field = "transcricao")
```



```

# Criar tokens
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE, remove_symbols = TRUE) |>
  tokens_remove(stopwords("portuguese"))

# Criar DFM e calcular TF-IDF
dfm_discursos <- tokens_discursos |>
  dfm() |>
  dfm_group(partido) |>
  dfm_tfidf()

# Calcular similaridade entre os textos cada partido
simil_cosseno <- textstat_simil(dfm_discursos, method = "cosine")

# Exibir como uma matriz de similaridade
matriz_simil <- as.matrix(simil_cosseno)

matriz_simil

```

	AVANTE	CIDADANIA	MDB	MISSÃO	NOVO
AVANTE	1.000000000	0.02787337	0.07380372	0.019631335	0.04485795
CIDADANIA	0.027873371	1.00000000	0.04369623	0.022123361	0.04573593
MDB	0.073803716	0.04369623	1.00000000	0.076356894	0.13200163
MISSÃO	0.019631335	0.02212336	0.07635689	1.000000000	0.13640773
NOVO	0.044857953	0.04573593	0.13200163	0.136407732	1.00000000
PCdoB	0.044107047	0.04904823	0.09325166	0.064075783	0.09736563
PDT	0.030919866	0.03348857	0.07546434	0.044773153	0.05749956
PL	0.128066709	0.07693393	0.24724439	0.178480068	0.30444040
PODE	0.068523881	0.07280090	0.14239923	0.048788652	0.11101765
PP	0.063266160	0.06679444	0.11308537	0.069248009	0.10791410
PRD	0.039376348	0.02456326	0.04924585	0.032951604	0.06735844
PSB	0.041043911	0.04041095	0.08618435	0.042934493	0.08249229
PSD	0.077358918	0.06454202	0.15889577	0.065271085	0.11145946
PSDB	0.040073915	0.04277501	0.08300785	0.030581116	0.05185333
PSOL	0.048683520	0.05073260	0.13940012	0.155322271	0.14754334
PT	0.107756605	0.08901398	0.23754635	0.180877637	0.24790576
PV	0.035220240	0.02268635	0.06747704	0.036841007	0.05868708
REDE	0.005847773	0.01415298	0.02173596	0.003466535	0.03218736
REPUBLICANOS	0.090222228	0.07820126	0.14204029	0.080380908	0.14666687
SOLIDARIEDADE	0.027678226	0.03535256	0.03662918	0.013309865	0.03384708
UNIÃO	0.088273252	0.08756866	0.16160667	0.091587775	0.15571983
	PCdoB	PDT	PL	PODE	PP

AVANTE	0.04410705	0.03091987	0.12806671	0.06852388	0.06326616
CIDADANIA	0.04904823	0.03348857	0.07693393	0.07280090	0.06679444
MDB	0.09325166	0.07546434	0.24724439	0.14239923	0.11308537
MISSÃO	0.06407578	0.04477315	0.17848007	0.04878865	0.06924801
NOVO	0.09736563	0.05749956	0.30444040	0.11101765	0.10791410
PCdoB	1.00000000	0.05717851	0.16735640	0.08683122	0.10017920
PDT	0.05717851	1.00000000	0.10435286	0.07130165	0.07351838
PL	0.16735640	0.10435286	1.00000000	0.18226007	0.20028036
PODE	0.08683122	0.07130165	0.18226007	1.00000000	0.11097406
PP	0.10017920	0.07351838	0.20028036	0.11097406	1.00000000
PRD	0.04604957	0.02746630	0.11480949	0.05936028	0.04458288
PSB	0.06144504	0.04166940	0.12017737	0.10256463	0.08499375
PSD	0.11796589	0.06798925	0.23182487	0.12403827	0.14909714
PSDB	0.05009437	0.03774048	0.10567578	0.06392002	0.07953645
PSOL	0.15806005	0.06738889	0.27411020	0.09810549	0.11574245
PT	0.23966729	0.11990024	0.46061379	0.19319766	0.20816439
PV	0.05988489	0.04268525	0.10240486	0.06193345	0.08366903
REDE	0.02124193	0.01007144	0.04120137	0.01826950	0.01207763
REPUBLICANOS	0.10975239	0.07339669	0.24397837	0.14417155	0.15155496
SOLIDARIEDADE	0.03179445	0.02067598	0.06110712	0.04106118	0.04353620
UNIÃO	0.11510133	0.07723010	0.30377991	0.14272272	0.15609655
	PRD	PSB	PSD	PSDB	PSOL
AVANTE	0.039376348	0.04104391	0.07735892	0.040073915	0.04868352
CIDADANIA	0.024563258	0.04041095	0.06454202	0.042775011	0.05073260
MDB	0.049245848	0.08618435	0.15889577	0.083007850	0.13940012
MISSÃO	0.032951604	0.04293449	0.06527108	0.030581116	0.15532227
NOVO	0.067358445	0.08249229	0.11145946	0.051853327	0.14754334
PCdoB	0.046049571	0.06144504	0.11796589	0.050094372	0.15806005
PDT	0.027466302	0.04166940	0.06798925	0.037740482	0.06738889
PL	0.114809488	0.12017737	0.23182487	0.105675779	0.27411020
PODE	0.059360284	0.10256463	0.12403827	0.063920017	0.09810549
PP	0.044582882	0.08499375	0.14909714	0.079536452	0.11574245
PRD	1.000000000	0.03003230	0.06276192	0.031295001	0.05632476
PSB	0.030032305	1.00000000	0.10049483	0.053813205	0.07369789
PSD	0.062761916	0.10049483	1.00000000	0.090908162	0.11961952
PSDB	0.031295001	0.05381321	0.09090816	1.000000000	0.05022647
PSOL	0.056324758	0.07369789	0.11961952	0.050226470	1.00000000
PT	0.106773354	0.14793576	0.23107618	0.121368474	0.37676418
PV	0.028869606	0.06556678	0.09436734	0.050131474	0.07788873
REDE	0.003269991	0.01455187	0.01639078	0.006021034	0.02203964
REPUBLICANOS	0.076520497	0.57069448	0.18142563	0.095010128	0.13027068
SOLIDARIEDADE	0.055407839	0.02664754	0.04177005	0.027435881	0.03396576
UNIÃO	0.108378480	0.13750687	0.20228906	0.104466945	0.16301463

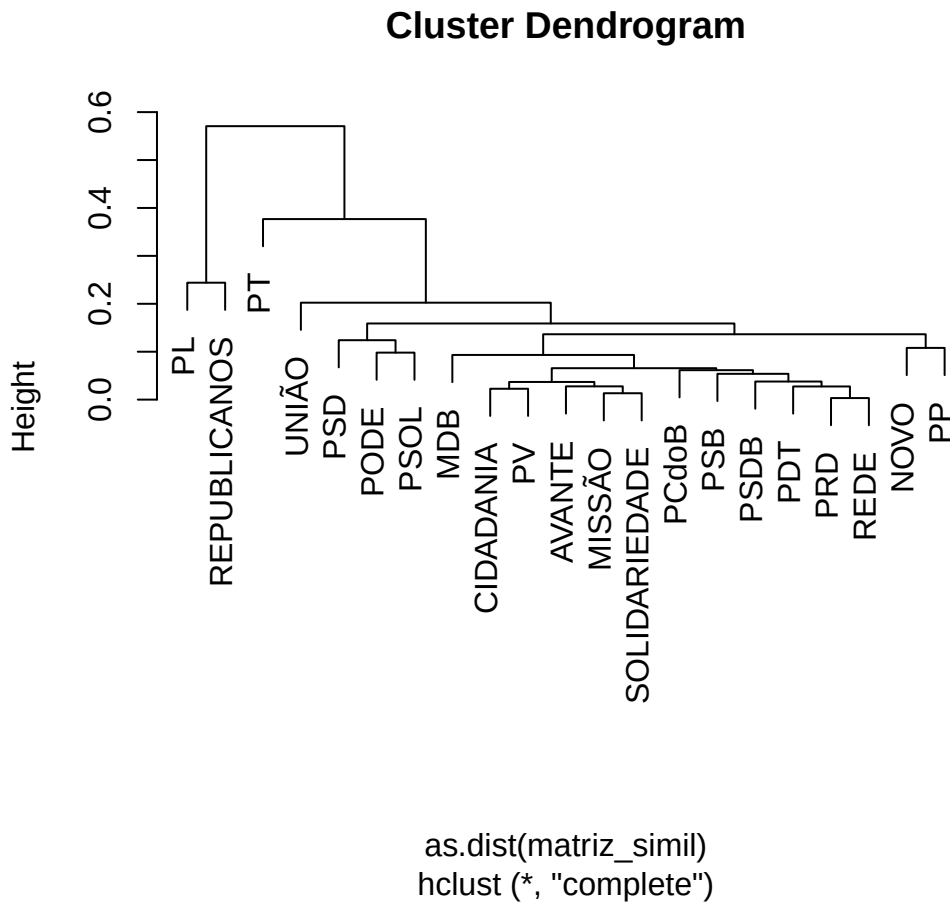
	PT	PV	REDE	REPUBLICANOS	SOLIDARIEDADE
AVANTE	0.10775661	0.03522024	0.005847773	0.09022223	0.027678226
CIDADANIA	0.08901398	0.02268635	0.014152977	0.07820126	0.035352559
MDB	0.23754635	0.06747704	0.021735959	0.14204029	0.036629178
MISSÃO	0.18087764	0.03684101	0.003466535	0.08038091	0.013309865
NOVO	0.24790576	0.05868708	0.032187359	0.14666687	0.033847078
PCdoB	0.23966729	0.05988489	0.021241927	0.10975239	0.031794454
PDT	0.11990024	0.04268525	0.010071442	0.07339669	0.020675978
PL	0.46061379	0.10240486	0.041201367	0.24397837	0.061107123
PODE	0.19319766	0.06193345	0.018269497	0.14417155	0.041061181
PP	0.20816439	0.08366903	0.012077631	0.15155496	0.043536198
PRD	0.10677335	0.02886961	0.003269991	0.07652050	0.055407839
PSB	0.14793576	0.06556678	0.014551869	0.57069448	0.026647540
PSD	0.23107618	0.09436734	0.016390780	0.18142563	0.041770048
PSDB	0.12136847	0.05013147	0.006021034	0.09501013	0.027435881
PSOL	0.37676418	0.07788873	0.022039636	0.13027068	0.033965756
PT	1.00000000	0.14851140	0.031137059	0.26134259	0.068081275
PV	0.14851140	1.00000000	0.030925006	0.09838080	0.022042157
REDE	0.03113706	0.03092501	1.000000000	0.01436216	0.008185284
REPUBLICANOS	0.26134259	0.09838080	0.014362164	1.00000000	0.049210784
SOLIDARIEDADE	0.06808127	0.02204216	0.008185284	0.04921078	1.000000000
UNIÃO	0.28740643	0.08940217	0.017001647	0.22259318	0.070512379
UNIÃO					
AVANTE	0.08827325				
CIDADANIA	0.08756866				
MDB	0.16160667				
MISSÃO	0.09158777				
NOVO	0.15571983				
PCdoB	0.11510133				
PDT	0.07723010				
PL	0.30377991				
PODE	0.14272272				
PP	0.15609655				
PRD	0.10837848				
PSB	0.13750687				
PSD	0.20228906				
PSDB	0.10446695				
PSOL	0.16301463				
PT	0.28740643				
PV	0.08940217				
REDE	0.01700165				
REPUBLICANOS	0.22259318				
SOLIDARIEDADE	0.07051238				

UNIÃO 1.00000000

A matriz de distâncias pode ser apresentada como um dendograma:

```
# Construir agrupamentos
agrupamentos <- hclust(as.dist(matriz_simil))

# Desenhar dendograma a partir dos agrupamentos
plot(agrupamentos)
```



6.7 Atividade prática

Utilizando o corpus de discursos de abril de 2025 da câmara:

- Treine um modelo STM para analisar a influência do estado dos parlamentares (coluna “uf”) e seu gênero (coluna “sexo”)
- Utilize diferentes distâncias de “textstat_dist” e “textstat_simil” da biblioteca quanteda para comparar discursos de diferentes estados. Existem relações entre distâncias geográficas e distâncias textuais?

Glossário

Introdução

ADP Análise de Discurso Político

PLN (NLP) Processamento de Linguagem Natural (Natural Language Processing)

Aula 1

R linguagem de programação utilizada para análise de dados

RStudio ambiente de desenvolvimento integrado (IDE) para R

Tidyverse coleção de pacotes R para ciência de dados

Variável nome para um valor armazenado em memória, que pode ser alterado

Condicional estrutura de código que executa ou não um bloco, de acordo com uma expressão lógica

Função um trecho de código que possui um nome, geralmente para uso de operações comuns

|> operador pipe utilizado para encadear chamadas de funções (o resultado é passado como argumento para a próxima função)

Vetor coleção ou sequência de um ou mais valores do mesmo tipo

Data Frame estrutura de dado para armazenar tabelas com linhas e colunas

Biblioteca coleção de funções, pode ser instalada e compartilhada em repositórios na Internet

String cadeia de caracteres, tipo de dado utilizado para representar texto

Referências

- Baker, Paul, Costas Gabrielatos, Majid KhosraviNik, Michał Krzyżanowski, Tony McEnery, and Ruth Wodak. 2008. “A Useful Methodological Synergy? Combining Critical Discourse Analysis and Corpus Linguistics to Examine Discourses of Refugees and Asylum Seekers in the UK Press.” *Discourse & Society* 19 (3): 273–306. <https://doi.org/10.1177/0957926508088962>.
- Benoit, Kenneth, Kohei Watanabe, Haiyan Wang, Paul Nulty, Adam Obeng, Stefan Müller, and Akitaka Matsuo. 2018. “Quanteda: An r Package for the Quantitative Analysis of Textual Data.” *Journal of Open Source Software* 3 (30): 774. <https://doi.org/10.21105/joss.00774>.
- Caseli, H. M., and M. G. V. Nunes, eds. 2024. *Processamento de Linguagem Natural: Conceitos, Técnicas e Aplicações Em Português*. 2nd ed. BPLN. <https://brasileiraspln.com/livro-pln/2a-edicao/>.
- Jurafsky, Daniel, and James H. Martin. 2026. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*. 3rd ed. <https://web.stanford.edu/~jurafsky/slp3/>.
- Marcus, Mitchell P., Beatrice Santorini, and Mary Ann Marcinkiewicz. 1993. “Building a Large Annotated Corpus of English: The Penn Treebank.” Edited by Julia Hirschberg. *Computational Linguistics* 19 (2): 313–330. <https://aclanthology.org/J93-2004/>.
- Moffitt, Benjamin. 2016. *The Global Rise of Populism: Performance, Political Style, and Representation*. 1st ed. Stanford University Press. <https://doi.org/10.2307/j.ctvqsd8>.
- Nivre, Joakim, Marie-Catherine de Marneffe, Filip Ginter, Jan Hajič, Christopher D. Manning, Sampo Pyysalo, Sebastian Schuster, Francis Tyers, and Daniel Zeman. 2020. “Universal Dependencies v2: An Evergrowing Multilingual Treebank Collection.” In, edited by Nicoletta Calzolari, Frédéric Béchet, Philippe Blache, Khalid Choukri, Christopher Cieri, Thierry Declerck, Sara Goggi, et al., 40344043. Marseille, France: European Language Resources Association. <https://aclanthology.org/2020.lrec-1.497/>.
- O’Neill, Lachlan, Nandini Anantharama, Wray Buntine, and Simon D. Angus. 2021. “Quantitative Discourse Analysis at Scale - AI, NLP and the Transformer Revolution.” *SoDa Laboratories Working Paper Series*, SoDa Laboratories Working Paper Series, December. <https://ideas.repec.org/p/ajr/sodwps/2021-12.html>.
- Schwartz, Fabiano Peruzzo. 2022. “INDICADOR DE SIMILARIDADE DO DISCURSO PARLAMENTAR: ANÁLISE DO COMPORTAMENTO DAS COALIZÕES PARTIDÁRIAS.” *E-Legis - Revista Eletrônica Do Programa de Pós-Graduação Da Câmara Dos Deputados* 15 (37): 1–20. <https://doi.org/10.51206/elegis.v15i37.715>.
- Schweinberger, Martin. 2025. “Seeded Topic Modeling as a More Appropriate Alternative to Unsupervised Standard Topic Models.” *Discourse Studies* 27 (4): 675–678. <https://doi.org/10.1177/14614456241293895>.

- Silge, Julia, and David Robinson. 2017. *Text mining with R: a tidy approach*. Beijing, China: O'Reilly.
- Wickham, Hadley. 2023. *R for Data Science*. 2nd ed. Sebastopol: O'Reilly Media, Incorporated.

A Coletando Dados do legislativo

A.1 Coletando dados da API da Câmara dos Deputados

A Câmara dos Deputados disponibiliza dados de discursos por meio de um sistema automatizado (API).

A documentação da API (versão 2) está disponível em: <https://dadosabertos.camara.leg.br/>

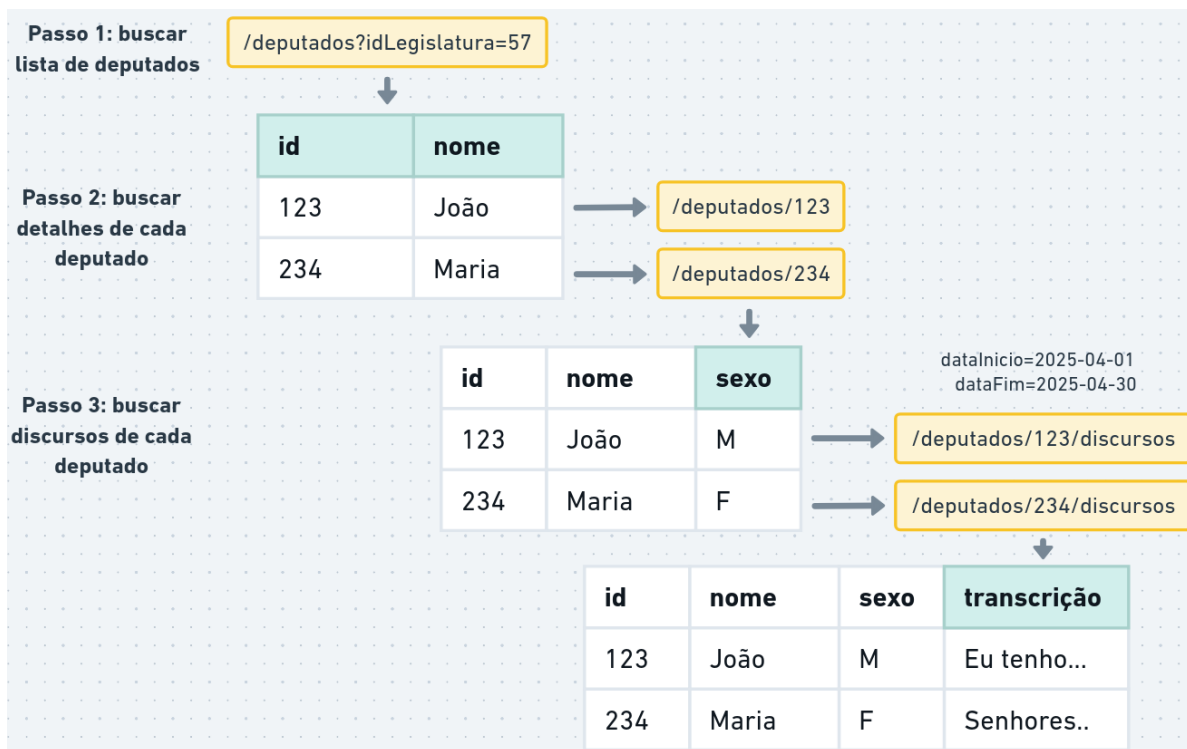
Para baixar e organizar esses dados, utilizamos pacotes da linguagem R:

- `httr2` para fazer requisições na API
- `tidyverse` para organizar e tratar os dados

```
if (!"tidyverse" %in% installed.packages()) {  
  install.packages("tidyverse")  
}  
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
v dplyr      1.2.0      v readr      2.2.0  
v forcats    1.0.1      v stringr    1.6.0  
v ggplot2    4.0.2      v tibble     3.3.1  
v lubridate  1.9.5      v tidyr      1.3.2  
v purrr      1.2.1  
-- Conflicts ----- tidyverse_conflicts() --  
x dplyr::filter() masks stats::filter()  
x dplyr::lag()     masks stats::lag()  
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
if (!"httr2" %in% installed.packages()) {  
  install.packages("httr2")  
}  
library(httr2)
```



Primeiro, vamos montar uma requisição (função `request`) para obter dados dos deputados na legislatura 57 (atual):

```
url_api <- "https://dadosabertos.camara.leg.br/"

requisicao_deputados <- request(url_api) |>
  req_url_path("api", "v2", "deputados") |>
  req_url_query(
    idLegislatura = 57, # número da legislatura
    nome = "",         # parte do nome
    siglaUF = "",      # sigla UF eleito
    siglaPartido = "", # sigla do partido
    siglaSexo = "",    # sexo (M ou F)
  )

print(requisicao_deputados)
```

<httr2_request>

GET https://dadosabertos.camara.leg.br/api/v2/deputados?idLegislatura=57&nome=&siglaUF=&siglaPartido=&siglaSexo=

Body: empty

Executamos a requisição (`req_perform`) para obter a resposta do serviço de dados abertos:

```
resp_deputados <- req_perform(requisicao_deputados)

print(resp_deputados)
```

```
<httr2_response>
GET https://dadosabertos.camara.leg.br/api/v2/deputados?idLegislatura=57&nome=&siglaUF=&siglaPartido=
Status: 200 OK
Content-Type: application/json
Body: In memory (248328 bytes)
```

Os dados retornados estão no formato JSON, que é representado no R como uma lista aninhada (lista com elementos de lista).

```
dados_resposta <- resp_body_json(resp_deputados)
dados_resposta <- dados_resposta$dados # acessa elemento "dados" da lista

print(length(dados_resposta)) # número de elementos
```

```
[1] 722
```

```
print(dados_resposta[[1]]) # primeiro elemento
```

```
$id
```

```
[1] 220593
```

```
$uri
```

```
[1] "https://dadosabertos.camara.leg.br/api/v2/deputados/220593"
```

```
$nome
```

```
[1] "Abilio Brunini"
```

```
$siglaPartido
```

```
[1] "PL"
```

```
$uriPartido
```

```
[1] "https://dadosabertos.camara.leg.br/api/v2/partidos/37906"
```

```
$siglaUf
```

```
[1] "MT"
```

```
$idLegislatura
```

```
[1] 57
```

```
$urlFoto
```

```
[1] "https://www.camara.leg.br/internet/deputado/bandep/220593.jpg"
```

```
$email
```

```
NULL
```

- `$dados` acessa o elemento “dados” da lista, `[[1]]` acessa o primeiro elemento da lista.

Transformamos a lista em dataframe com o código comentado abaixo:

```
df_deputados_partido <- dados_resposta |>
  tibble() |>                                # transformar em dataframe
  unnest_wider("dados_resposta") |>         # desaninhar em colunas
  select(                                    # seleciona colunas relevantes
    id,
    nome,
    uf = siglaUf,
    partido = siglaPartido,
    uri
  )

head(df_deputados_partido) # head mostra as primeiras linhas do data frame
```

```
# A tibble: 6 x 5
```

	id	nome	uf	partido	uri
	<int>	<chr>	<chr>	<chr>	<chr>
1	220593	Abilio Brunini	MT	PL	https://dadosabertos.camara.leg.br/~
2	204379	Acácio Favacho	AP	MDB	https://dadosabertos.camara.leg.br/~
3	220714	Adail Filho	AM	REPUBLICANOS	https://dadosabertos.camara.leg.br/~
4	221328	Adilson Barroso	SP	PL	https://dadosabertos.camara.leg.br/~
5	204560	Adolfo Viana	BA	PSDB	https://dadosabertos.camara.leg.br/~
6	204528	Adriana Ventura	SP	NOVO	https://dadosabertos.camara.leg.br/~

Note que existem repetições, quando um deputado pertenceu a mais de um partido:

```
deputados_multiplos_partidos <- df_deputados_partido |>
  group_by(nome) |> # agrupar por nome
  filter(n() > 1)   # filtrar grupos com mais de um elemento

head(deputados_multiplos_partidos)
```

```
# A tibble: 6 x 5
# Groups:   nome [3]
      id nome          uf partido uri
  <int> <chr>         <chr> <chr> <chr>
1 220542 Alexandre Guimarães TO REPUBLICANOS https://dadosabertos.camara.leg~
2 220542 Alexandre Guimarães TO MDB          https://dadosabertos.camara.leg~
3 178881 Aluisio Mendes      MA PSC          https://dadosabertos.camara.leg~
4 178881 Aluisio Mendes      MA REPUBLICANOS https://dadosabertos.camara.leg~
5 220612 Bebeto              RJ PTB          https://dadosabertos.camara.leg~
6 220612 Bebeto              RJ PP           https://dadosabertos.camara.leg~
```

A coluna uri contém o endereço para buscarmos detalhes sobre o deputado, como sexo, data de nascimento, situação atual, etc.

Para cada deputado, vamos fazer a requisição que retorna os detalhes. Para isso, vamos mapear cada elemento da coluna uri em uma requisição:

```
df_deputados_detalhes <- df_deputados_partido |>
  mutate(req_detalhes = map(uri, request)) |> # montar requisições
  mutate(resp_detalhes = req_perform_parallel(req_detalhes)) # buscar detalhes e armazenar a
```

```
[working] (699 + 0) -> 10 -> 13 | =>----- 2%
[working] (658 + 0) -> 10 -> 54 | ==>----- 7%
[working] (614 + 0) -> 10 -> 98 | ===>----- 14%
[working] (570 + 0) -> 10 -> 142 | =====>----- 20%
[working] (524 + 0) -> 10 -> 188 | =====>----- 26%
[working] (486 + 0) -> 10 -> 226 | =====>----- 31%
[working] (447 + 0) -> 9 -> 266 | =====>----- 37%
```

[working] (405 + 0) -> 10 -> 307 | =====>----- 43%

[working] (360 + 0) -> 10 -> 352 | =====>----- 49%

[working] (321 + 0) -> 10 -> 391 | =====>----- 54%

[working] (279 + 0) -> 10 -> 433 | =====>----- 60%

[working] (234 + 0) -> 10 -> 478 | =====>----- 66%

[working] (192 + 0) -> 10 -> 520 | =====>----- 72%

[working] (147 + 0) -> 10 -> 565 | =====>----- 78%

[working] (106 + 0) -> 10 -> 606 | =====>----- 84%

[working] (65 + 0) -> 10 -> 647 | =====>---- 90%

Waiting 29s for rate limit =>-----

Waiting 29s for rate limit ===>-----

Waiting 29s for rate limit =====>-----

Waiting 29s for rate limit =====>-----

Waiting 29s for rate limit =====>-----

Waiting 29s for rate limit =====>-----

Waiting 29s for rate limit =====>-----

Waiting 29s for rate limit =====>-----

Waiting 29s for rate limit =====>-----

Waiting 29s for rate limit =====>--

Waiting 29s for rate limit =====>

```
[working] (65 + 0) -> 10 -> 647 | =====>--- 90%
[working] (24 + 1) -> 11 -> 687 | =====>- 95%
[working] (0 + 1) -> 0 -> 722 | =====> 100%
```

```
head(df_deputados_detalhes)
```

```
# A tibble: 6 x 7
```

	id	nome	uf	partido	uri	req_detalhes	resp_detalhes
	<int>	<chr>	<chr>	<chr>	<chr>	<list>	<list>
1	220593	Abilio Brunini	MT	PL	https://~	<httr2_rq>	<httr2_rs>
2	204379	Acácio Favacho	AP	MDB	https://~	<httr2_rq>	<httr2_rs>
3	220714	Adail Filho	AM	REPUBLICANOS	https://~	<httr2_rq>	<httr2_rs>
4	221328	Adilson Barroso	SP	PL	https://~	<httr2_rq>	<httr2_rs>
5	204560	Adolfo Viana	BA	PSDB	https://~	<httr2_rq>	<httr2_rs>
6	204528	Adriana Ventura	SP	NOVO	https://~	<httr2_rq>	<httr2_rs>

- `req_perform_parallel` executa múltiplas requisições em paralelo (cada requisição da coluna `req_detalhes`).

Vamos extrair os detalhes da coluna `resp_detalhes` e organizar os dados na tabela.

```
df_deputados <- df_deputados_detalhes |>
mutate(detalhes = map(resp_detalhes, resp_body_json)) |> # ler dados da resposta
unnest_wider(detalhes) |> unnest_wider("dados", names_sep = "_") |> # desaninhar coluna "dados"
unnest_wider("dados_ultimoStatus", names_sep = "_") |> # desaninhar ultimoStatus
select( # selecionar colunas
  # colunas antigas
  colnames(df_deputados_partido),
  # novas colunas
  nome_civil = dados_nomeCivil,
  sexo = dados_sexo,
  cpf = dados_cpf,
  data_nascimento = dados_dataNascimento,
  data_falecimento = dados_dataFalecimento,
  uf_nascimento = dados_ufNascimento,
  municipio_nascimento = dados_municipioNascimento,
  escolaridade = dados_escolaridade,
  situacao = dados_ultimoStatus_situacao,
  data_situacao = dados_ultimoStatus_data,
  ultimo_partido = dados_ultimoStatus_siglaPartido
```

```

) |>
filter(partido == ultimo_partido) |> # considerar apenas último partido
distinct()                          # filtrar dados duplicados

head(df_deputados)

```

A tibble: 6 x 16

```

      id nome      uf partido uri nome_civil sexo cpf data_nascimento
  <int> <chr>    <chr> <chr> <chr> <chr>    <chr> <chr> <chr>
1 220593 Abilio Brun~ MT PL http~ ABILIO JA~ M 9977~ 1984-01-31
2 204379 Acácio Fava~ AP MDB http~ ACÁCIO DA~ M 7428~ 1983-09-28
3 220714 Adail Filho AM REPUB~ http~ ADAIL JOS~ M 7726~ 1992-02-16
4 221328 Adilson Bar~ SP PL http~ ADILSON B~ M 0558~ 1964-06-14
5 204560 Adolfo Viana BA PSDB http~ ADOLFO VI~ M 8012~ 1981-02-02
6 204528 Adriana Ven~ SP NOVO http~ ADRIANA M~ F 1251~ 1969-03-06
# i 7 more variables: data_falecimento <chr>, uf_nascimento <chr>,
# municipio_nascimento <chr>, escolaridade <chr>, situacao <chr>,
# data_situacao <chr>, ultimo_partido <chr>

```

Para obter os discursos, vamos definir uma função que monta a requisição para um deputado (por número identificador):

```

req_discursos_deputado <- function(id_deputado) {
  req <- request(url_api) |>
  req_url_path("api", "v2", "deputados", id_deputado, "discursos") |>
  req_url_query(
    dataInicio = "2025-04-01",
    dataFim = "2025-04-30",
    itens=100
  )
  return(req)
}

print(req_discursos_deputado(160575))

```

<httr2_request>

```

GET https://dadosabertos.camara.leg.br/api/v2/deputados/160575/discursos?dataInicio=2025-04-01&dataFim=2025-04-30&itens=100
Body: empty

```


Da mesma forma que obtemos os dados dos deputados, vamos fazer requisições para obter discursos de cada deputado:

```
df_discursos <- df_deputados |>
  mutate(req_discursos = map(id, req_discursos_deputado)) |>      # montar a requisição chamada
  mutate(resp_discursos = req_perform_parallel(req_discursos)) |>  # executar requisições em paralelo
  mutate(dados_discursos = map(resp_discursos, resp_body_json))    # ler dados das respostas
```

```
[working] (627 + 0) -> 10 -> 8 | >----- 1%
[working] (625 + 0) -> 10 -> 10 | >----- 2%
[working] (609 + 0) -> 10 -> 26 | =>----- 4%
[working] (582 + 0) -> 10 -> 53 | ==>----- 8%
[working] (565 + 0) -> 9 -> 71 | ===>----- 11%
[working] (524 + 0) -> 10 -> 111 | =====>----- 17%
[working] (499 + 0) -> 10 -> 136 | =====>----- 21%
[working] (463 + 0) -> 10 -> 172 | =====>----- 27%
[working] (434 + 0) -> 10 -> 201 | =====>----- 31%
[working] (405 + 0) -> 10 -> 230 | =====>----- 36%
[working] (376 + 0) -> 10 -> 259 | =====>----- 40%
[working] (353 + 0) -> 10 -> 282 | =====>----- 44%
[working] (346 + 0) -> 10 -> 289 | =====>----- 45%
[working] (326 + 0) -> 10 -> 309 | =====>----- 48%
[working] (293 + 0) -> 10 -> 342 | =====>----- 53%
```

```

[working] (269 + 0) -> 10 -> 366 | =====>----- 57%

[working] (238 + 0) -> 10 -> 397 | =====>----- 62%

[working] (219 + 0) -> 10 -> 416 | =====>----- 64%

[working] (190 + 0) -> 10 -> 445 | =====>----- 69%

[working] (157 + 0) -> 10 -> 478 | =====>----- 74%

[working] (124 + 0) -> 10 -> 511 | =====>----- 79%

[working] (100 + 0) -> 10 -> 535 | =====>----- 83%

[working] (71 + 0) -> 10 -> 564 | =====>----- 87%

[working] (48 + 0) -> 10 -> 587 | =====>----- 91%

[working] (17 + 0) -> 10 -> 618 | =====>- 96%

[working] (0 + 0) -> 5 -> 640 | =====> 99%

[working] (0 + 0) -> 0 -> 645 | =====> 100%

```

```
print(df_discurso$dados_discurso[[2]])
```

```

$dados
$dados[[1]]
$dados[[1]]$dataHoraInicio
[1] "2025-04-29T13:55"

$dados[[1]]$dataHoraFim
NULL

$dados[[1]]$uriEvento
[1] ""

$dados[[1]]$faseEvento

```

\$dados[[1]]\$faseEvento\$titulo
[1] "Encerramento"

\$dados[[1]]\$faseEvento\$dataHoraInicio
NULL

\$dados[[1]]\$faseEvento\$dataHoraFim
NULL

\$dados[[1]]\$tipoDiscurso
[1] "DISCURSO ENCAMINHADO"

\$dados[[1]]\$urlTexto
[1] "https://imagem.camara.gov.br/dc_20b.asp?largura=&altura=&tipoForm=diarios&selCodColecao"

\$dados[[1]]\$urlAudio
NULL

\$dados[[1]]\$urlVideo
NULL

\$dados[[1]]\$keywords
[1] "Deputado federal, Missão oficial, China, Balanço, Amapá, Fonte alternativa de energia, Desenvol"

\$dados[[1]]\$sumario
[1] "O Deputado relatou sua participação em missão oficial à China, realizada entre 17 e 27 de"

\$dados[[1]]\$transcricao
[1] "DISCURSO NA ÍNTEGRA ENCAMINHADO PELO SR. DEPUTADO ACÁCIO FAVACHO (SEM REGISTRO TAQUIGRÁFICO)"

\$dados[[2]]
\$dados[[2]]\$dataHoraInicio
[1] "2025-04-02T10:32"

\$dados[[2]]\$dataHoraFim
NULL

\$dados[[2]]\$uriEvento
[1] ""

\$dados[[2]]\$faseEvento

\$dados[[2]]\$faseEvento\$titulo

[1] "Homenagem"

\$dados[[2]]\$faseEvento\$dataHoraInicio

NULL

\$dados[[2]]\$faseEvento\$dataHoraFim

NULL

\$dados[[2]]\$tipoDiscurso

[1] "HOMENAGEM"

\$dados[[2]]\$urlTexto

[1] "https://imagem.camara.gov.br/dc_20b.asp?largura=&altura=&tipoForm=diarios&selCodColecao"

\$dados[[2]]\$urlAudio

NULL

\$dados[[2]]\$urlVideo

NULL

\$dados[[2]]\$keywords

[1] "Dia Nacional de Conscientização sobre o Autismo,Homenagem,Sessão solene"

\$dados[[2]]\$sumario

[1] "O Deputado discursou na sessão solene em homenagem ao Dia Mundial de Conscientização sol"

\$dados[[2]]\$transcricao

[1] "O SR. ACÁCIO FAVACHO (Bloco/MDB - AP. Sem revisão do orador.) - Presidente, bom dia. Ob
o para Brasília para dar a ele um tratamento adequado. E, no dia de hoje, ela apresentou ess

\$dados[[3]]

\$dados[[3]]\$dataHoraInicio

[1] "2025-04-01T13:55"

\$dados[[3]]\$dataHoraFim

NULL

\$dados[[3]]\$uriEvento

[1] ""

```

$dados[[3]]$faseEvento
$dados[[3]]$faseEvento$titulo
[1] "Encerramento"

$dados[[3]]$faseEvento$dataHoraInicio
NULL

$dados[[3]]$faseEvento$dataHoraFim
NULL

$dados[[3]]$tipoDiscurso
[1] "DISCURSO ENCAMINHADO"

$dados[[3]]$urlTexto
[1] "https://imagem.camara.gov.br/dc_20b.asp?largura=&altura=&tipoForm=diarios&selCodColecao"

$dados[[3]]$urlAudio
NULL

$dados[[3]]$urlVideo
NULL

$dados[[3]]$keywords
[1] "Macapá (AP),Habitação,Política habitacional,Programa Minha Casa, Minha Vida (PMCMV),Min"

$dados[[3]]$sumario
[1] "O Deputado celebrou os avanços do Programa Casa Macapá, vinculado ao Programa Minha Casa"

$dados[[3]]$transcricao
[1] "DISCURSO NA ÍNTEGRA ENCAMINHADO PELO SR. DEPUTADO ACÁCIO FAVACHO (SEM REGISTRO TAQUIGRÁF"

$links
$links[[1]]
$links[[1]]$rel
[1] "self"

$links[[1]]$href
[1] "https://dadosabertos.camara.leg.br/api/v2/deputados/204379/discursos?dataInicio=2025-04-01&dataFim=2025-04-30&itens=100"

```

```

$links[[2]]
$links[[2]]$rel
[1] "first"

$links[[2]]$href
[1] "https://dadosabertos.camara.leg.br/api/v2/deputados/204379/discursos?dataInicio=2025-04-01&dataFim=2025-04-30&pagina=1&itens=100"

$links[[3]]
$links[[3]]$rel
[1] "last"

$links[[3]]$href
[1] "https://dadosabertos.camara.leg.br/api/v2/deputados/204379/discursos?dataInicio=2025-04-01&dataFim=2025-04-30&pagina=1&itens=100"

```

Por fim, vamos extrair dos dados em colunas:

```

df_discursos <- df_discursos |>
  unnest_wider(dados_discursos) |> # desaninhar resposta em colunas (dados e links)
  unnest(dados) |> # desaninhar dados de discursos em linha (um deputado por linha)
  unnest_wider(dados) |> # desaninhar dados em colunas
  unnest_wider(faseEvento, names_sep = "_") |> # desaninhar "faseEvento"
  select(
    # colunas anteriores
    colnames(df_deputados),
    # novas colunas
    hora_inicio = dataHoraInicio,
    fase_evento = faseEvento_titulo,
    tipo_discurso = tipoDiscurso,
    keywords,
    sumario,
    transcricao
  )
head(df_discursos)

```

A tibble: 6 x 22

id	nome	uf	partido	uri	nome_civil	sexo	cpf	data_nascimento
<int>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>

```

1 204379 Acácio Fava~ AP      MDB      http~ ACÁCIO DA~ M      7428~ 1983-09-28
2 204379 Acácio Fava~ AP      MDB      http~ ACÁCIO DA~ M      7428~ 1983-09-28
3 204379 Acácio Fava~ AP      MDB      http~ ACÁCIO DA~ M      7428~ 1983-09-28
4 220714 Adail Filho  AM      REPUB~ http~ ADAIL JOS~ M      7726~ 1992-02-16
5 221328 Adilson Bar~ SP      PL       http~ ADILSON B~ M      0558~ 1964-06-14
6 221328 Adilson Bar~ SP      PL       http~ ADILSON B~ M      0558~ 1964-06-14
# i 13 more variables: data_falecimento <chr>, uf_nascimento <chr>,
#   municipio_nascimento <chr>, escolaridade <chr>, situacao <chr>,
#   data_situacao <chr>, ultimo_partido <chr>, hora_inicio <chr>,
#   fase_evento <chr>, tipo_discurso <chr>, keywords <chr>, sumario <chr>,
#   transcricao <chr>

```

Por fim, podemos extrair estatísticas sobre os oradores e salvar os dados em uma planilha CSV para usos futuros.

```
print(str_c("Número total de discursos: ", nrow(df_discursos)))
```

```
[1] "Número total de discursos: 1849"
```

```
print(str_c("Número total de oradores: ", df_discursos |> select(nome) |> n_distinct()))
```

```
[1] "Número total de oradores: 317"
```

```
print(str_c("Número total de partidos: ", df_discursos |> select(partido) |> n_distinct()))
```

```
[1] "Número total de partidos: 21"
```

```
print(str_c("Número total de estados: ", df_discursos |> select(uf) |> n_distinct()))
```

```
[1] "Número total de estados: 27"
```

Principais oradores:

```
df_discursos |>
  group_by(nome) |> # agrupar por nome do orador
  count(sort = TRUE) # contar e ordenar

```

```
# A tibble: 317 x 2
# Groups:   nome [317]
  nome          n
  <chr>        <int>
1 Cabo Gilberto Silva    57
2 Luiz Lima              48
3 Hildo Rocha            39
4 Marcel van Hattem      39
5 Bibi Nunes             36
6 Erika Kokay            36
7 Coronel Assis          30
8 Gilson Marques         30
9 Chico Alencar          29
10 Otoni de Paula        28
# i 307 more rows
```

Principais partidos:

```
df_discursos |>
  group_by(partido) |> # agrupar por partido do orador
  count(sort = TRUE)  # contar e ordenar
```

```
# A tibble: 21 x 2
# Groups:   partido [21]
  partido      n
  <chr>    <int>
1 PL        455
2 PT        373
3 NOVO      136
4 PSOL      129
5 UNIÃO     117
6 MDB       105
7 PSD        94
8 REPUBLICANOS  92
9 PSB        82
10 PCdoB      57
# i 11 more rows
```

Para facilitar novos processamentos, podemos salvar os dados em formato CSV:


```
write_csv(df_discursos, "discursos.csv")
```

Unindo todo o processo:

```
library(tidyverse)
library(httr2)

obter_discursos_camara <- function(data_inicio, data_fim, id_legislatura = 57) {
  print("1/3 - Buscando lista de deputados")
  url_api <- "https://dadosabertos.camara.leg.br/"

  extrair_dados_resposta <- function(resp) {
    resposta <- resp_body_json(resp, simplifyDataFrame = TRUE)
    return(resposta$dados)
  }

  df_deputados_partido <- request(url_api) |>
    req_url_path("api", "v2", "deputados") |>
    req_url_query(
      idLegislatura = id_legislatura, # número da legislatura
      nome = "", # parte do nome
      siglaUF = "", # sigla UF eleito
      siglaPartido = "", # sigla do partido
      siglaSexo = "", # sexo (M ou F)
    ) |>
    req_perform() |> # executa a requisição
    extrair_dados_resposta() |> # extrai item "dados"
    select( # seleciona colunas relevantes
      id,
      nome,
      uf = siglaUf,
      partido = siglaPartido,
      uri
    )

  print("2/3 - Buscando detalhes de cada deputado")

  df_deputados <- df_deputados_partido |>
    mutate(req_detalhes = map(uri, request)) |> # montar requisições
    mutate(resp_detalhes = req_perform_parallel(req_detalhes)) |> # buscar detalhes
    mutate(detalhes = map(resp_detalhes, extrair_dados_resposta)) |> # extrair detalhes das 1
    unnest_wider(detalhes, names_sep = "_") |> # desaninhar detalhes
```

```

unnest_wider(detalhes_ultimoStatus, names_sep = "_") |> # desaninhar último status
select(
  # colunas antigas
  colnames(df_deputados_partido),
  # novas colunas
  nome_civil = detalhes_nomeCivil,
  sexo = detalhes_sexo,
  data_nascimento = detalhes_dataNascimento,
  data_falecimento = detalhes_dataFalecimento,
  uf_nascimento = detalhes_ufNascimento,
  municipio_nascimento = detalhes_municipioNascimento,
  escolaridade = detalhes_escolaridade,
  situacao = detalhes_ultimoStatus_situacao,
  data_situacao = detalhes_ultimoStatus_data,
  ultimo_partido = detalhes_ultimoStatus_siglaPartido
) |>
filter(partido == ultimo_partido) |> # considerar apenas último partido
distinct() # filtrar dados duplicados

print("3/3 - Buscando discursos de cada deputado")

req_discursos_deputado <- function(id_deputado) {
  req <- request(url_api) |>
  req_url_path("api", "v2", "deputados", id_deputado, "discursos") |>
  req_url_query(
    dataInicio = data_inicio,
    dataFim = data_fim,
  )
  return(req)
}

resp_discursos_deputados <- df_deputados |>
mutate(req_discursos = map(id, req_discursos_deputado)) |>
mutate(resp_discursos = req_perform_parallel(req_discursos))

df_discursos <- resp_discursos_deputados |>
mutate(discursos = map(resp_discursos, extrair_dados_resposta)) |> # respostas
filter(lengths(discursos) > 0) |> # filtrar linhas sem discurso
unnest(discursos) |> # desaninhar dados de discursos
unnest_wider(faseEvento, names_sep = "_") |> # desaninhar dados de evento
select(
  # colunas anteriores
  # selecionar colunas relevantes

```

```

        colnames(df_deputados),
        # novas colunas
        hora_inicio = dataHoraInicio,
        fase_evento = faseEvento_titulo,
        tipo_discurso = tipoDiscurso,
        keywords,
        sumario,
        transcricao
    )

    return(df_discursos)
}

```

```

dt_discursos_camara <- obter_discursos_camara(
    id_legislatura = 57,
    data_inicio = "2025-04-01",
    data_fim = "2025-04-30"
)

```

```

[1] "1/3 - Buscando lista de deputados"
[1] "2/3 - Buscando detalhes de cada deputado"

```

```

[working] (699 + 0) -> 8 -> 15 | =>----- 2%

```

```

[working] (670 + 0) -> 10 -> 42 | ==>----- 6%

```

```

[working] (628 + 0) -> 11 -> 83 | ===>----- 11%

```

```

Waiting 28s for rate limit =>-----

```

```

Waiting 28s for rate limit ====>-----

```

```

Waiting 28s for rate limit =====>-----

```

```

Waiting 28s for rate limit =====>-----

```

```

Waiting 28s for rate limit =====>-----

```

```

Waiting 28s for rate limit =====>-----

```

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>

Waiting 28s for rate limit =====>

[working] (628 + 0) -> 11 -> 83 | ===>----- 11%

[working] (576 + 1) -> 10 -> 136 | =====>----- 19%

Waiting 28s for rate limit =>-----

Waiting 28s for rate limit ===>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

[working] (576 + 1) -> 10 -> 136 | =====>----- 19%

[working] (522 + 2) -> 10 -> 190 | =====>----- 26%

[working] (478 + 2) -> 10 -> 234 | =====>----- 32%

Waiting 28s for rate limit =>-----

Waiting 28s for rate limit ==>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

[working] (478 + 2) -> 10 -> 234 | =====>----- 32%

[working] (422 + 3) -> 8 -> 292 | =====>----- 40%

[working] (379 + 3) -> 10 -> 333 | =====>----- 46%

[working] (338 + 3) -> 9 -> 375 | =====>----- 52%

[working] (294 + 3) -> 10 -> 418 | =====>----- 58%

```

[working] (250 + 3) -> 10 -> 462 | =====>----- 64%
[working] (205 + 3) -> 10 -> 507 | =====>----- 70%
[working] (160 + 3) -> 10 -> 552 | =====>----- 76%
[working] (115 + 3) -> 10 -> 597 | =====>----- 83%
[working] (70 + 3) -> 10 -> 642 | =====>----- 89%
[working] (27 + 3) -> 10 -> 685 | =====>----- 95%
[working] (0 + 3) -> 1 -> 721 | =====>----- 100%
[working] (0 + 3) -> 0 -> 722 | =====>----- 100%

```

[1] "3/3 - Buscando discursos de cada deputado"

```

[working] (616 + 0) -> 10 -> 19 | ==>----- 3%
[working] (583 + 0) -> 10 -> 52 | ==>----- 8%
[working] (550 + 0) -> 10 -> 85 | ====>----- 13%
[working] (518 + 0) -> 10 -> 117 | =====>----- 18%
[working] (487 + 0) -> 10 -> 148 | =====>----- 23%
[working] (459 + 0) -> 10 -> 176 | =====>----- 27%
[working] (427 + 0) -> 10 -> 208 | =====>----- 32%
[working] (393 + 0) -> 10 -> 242 | =====>----- 38%
[working] (378 + 0) -> 10 -> 257 | =====>----- 40%
[working] (339 + 0) -> 10 -> 296 | =====>----- 46%
[working] (308 + 0) -> 10 -> 327 | =====>----- 51%
[working] (282 + 0) -> 10 -> 353 | =====>----- 55%
[working] (246 + 0) -> 10 -> 389 | =====>----- 60%
[working] (217 + 0) -> 10 -> 418 | =====>----- 65%
[working] (172 + 0) -> 11 -> 462 | =====>----- 72%
[working] (132 + 0) -> 10 -> 503 | =====>----- 78%
[working] (85 + 0) -> 10 -> 550 | =====>----- 85%
[working] (45 + 0) -> 10 -> 590 | =====>----- 91%
[working] (8 + 0) -> 10 -> 627 | =====>----- 97%
[working] (0 + 0) -> 0 -> 645 | =====>----- 100%

```

```
head(dt_discursos_camara)
```

```
# A tibble: 6 x 21
```

	id	nome	uf	partido	uri	nome_civil	sexo	data_nascimento
	<int>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>
1	204379	Acácio Favacho	AP	MDB	http~	ACÁCIO DA~	M	1983-09-28
2	204379	Acácio Favacho	AP	MDB	http~	ACÁCIO DA~	M	1983-09-28
3	204379	Acácio Favacho	AP	MDB	http~	ACÁCIO DA~	M	1983-09-28
4	220714	Adail Filho	AM	REPUBLICA~	http~	ADAIL JOS~	M	1992-02-16

```

5 221328 Adilson Barroso SP      PL      http~ ADILSON B~ M      1964-06-14
6 221328 Adilson Barroso SP      PL      http~ ADILSON B~ M      1964-06-14
# i 13 more variables: data_falecimento <chr>, uf_nascimento <chr>,
#   municipio_nascimento <chr>, escolaridade <chr>, situacao <chr>,
#   data_situacao <chr>, ultimo_partido <chr>, hora_inicio <chr>,
#   fase_evento <chr>, tipo_discurso <chr>, keywords <chr>, sumario <chr>,
#   transcricao <chr>

```

```
write_csv(dt_discursos_camara, "discursos_camara_abril_2025.csv")
```

A.2 Coletando dados da API do Senado

De maneira análoga, podemos definir uma função para obter dados de discursos do senado:

```

obter_discursos_senado <- function(data_inicio, data_fim) {
  url_discursos <- "https://legis.senado.leg.br/dadosabertos/plenario/lista/discursos/"
  req_discursos <- request(url_discursos) |>
    req_url_path_append(data_inicio, data_fim) |>
    req_headers("Accept" = "application/json")

  resp_discursos <- req_perform(req_discursos)

  print("1/3 - Obtendo dados de discursos")

  dt_discursos <- resp_discursos |>
    resp_body_json() |>                                     # extrair dados da resposta em JSON
    pluck("DiscursosSessao", "Sessoes") |>                 # acessar dados dentro de DiscursosSessao
    as_tibble() |>                                           # transformar em tabela
    unnest_wider(Sessao) |>                                   # desaninhar dados das sessões
    unnest(Pronunciamentos) |>                               # desaninhar pronunciamentos
    unnest_longer(Pronunciamentos) |>                       # desaninhar pronunciamentos da sessão
    unnest_wider(Pronunciamentos) |>                       # desaninhar dados do pronunciamento
    unnest_wider(TipoUsoPalavra, names_sep = '_') |>        # desaninhar tipo de uso da palavra
    filter(!is.na(CodigoParlamentar)) |>                   # filtrar discursos de não-parlamentares
    select(                                                  # selecionar colunas
      id,
      codigo_parlamentar = CodigoParlamentar,
      nome = NomeAutor,
      partido = Partido,
      uf = UF,
      tipo_sessao = DescricaoSessao,

```

```

    tipo_uso_palavra = TipoUsoPalavra_Descricao,
    data = Data,
    sumario = Resumo,
    keywords = Indexacao,
    transcricao = TextoIntegralTxt
  )

print("2/3 - Obtendo inteiro teor")

dt_discursos_inteiro <- dt_discursos |>
  mutate(transcricao = map(transcricao, request)) |> # requisição da transcrição
  mutate(transcricao = req_perform_parallel(transcricao)) |> # buscar transcrições
  mutate(transcricao = map(transcricao, resp_body_string)) # ler transcrições das respostas

req_detalhes_senador <- function(codigo_parlamentar) {
  url_senadores <- "https://legis.senado.leg.br/dadosabertos/senador/"
  req <- request(url_senadores) |>
    req_url_path_append(codigo_parlamentar) |>
    req_headers(Accept = "application/json")
  return(req)
}

print("3/3 - Obtendo detalhes dos senadores")

dt_detalhes_senadores <- dt_discursos_inteiro |>
  select(codigo_parlamentar) |>
  distinct() |>
  mutate(req_senador = map(codigo_parlamentar, req_detalhes_senador)) |>
  mutate(resp_senador = req_perform_parallel(req_senador)) |>
  mutate(detalhes = map(resp_senador, resp_body_json)) |>
  unnest(detalhes) |> # desaninhar detalhes
  unnest_wider(detalhes) |> # desaninhar dados de detalhes
  unnest_wider(Parlamentar) |> # desaninhar dados do parlamentar
  unnest_wider(DadosBasicosParlamentar) |> # desaninhar dados básicos
  unnest_wider(IdentificacaoParlamentar) |> # desaninhar identificação
  select( # selecionar colunas
    codigo_parlamentar,
    nome_civil = NomeCompletoParlamentar,
    sexo = SexoParlamentar,
    data_nascimento = DataNascimento,
    uf_nascimento = UfNaturalidade,
    municipio_nascimento = Naturalidade
  )

```

```

    )

    dt_discursos_senado <- dt_discursos_inteiro |>
      inner_join(dt_detalhes_senadores, by = join_by(codigo_parlamentar))

    return(dt_discursos_senado)
  }

```

```

df_discursos_senado <- obter_discursos_senado(
  data_inicio = "20250401",
  data_fim = "20250430"
)

```

```

[1] "1/3 - Obtendo dados de discursos"
[1] "2/3 - Obtendo inteiro teor"

```

```

Waiting 15s for rate limit ==>-----
Waiting 15s for rate limit ==>-----

```

```

Waiting 15s for rate limit =====>-----

```

```

Waiting 15s for rate limit =====>-----

```

```

Waiting 15s for rate limit =====>-----

```

```

Waiting 15s for rate limit =====>----

```

```

Waiting 15s for rate limit =====>

```

```

[working] (153 + 8) -> 10 -> 188 | =====>----- 54%

```

```

Waiting 15s for rate limit ==>-----

```

```

Waiting 15s for rate limit =====>-----

```

```

Waiting 15s for rate limit =====>-----

```

```

Waiting 15s for rate limit =====>-----

```


Waiting 15s for rate limit =====>----

Waiting 15s for rate limit =====>

```
[working] (153 + 8) -> 10 -> 188 | =====>----- 54%
[working] (21 + 11) -> 10 -> 320 | =====>---- 91%
[working] (0 + 11) -> 0 -> 351 | =====> 100%
```

[1] "3/3 - Obtendo detalhes dos senadores"

Waiting 15s for rate limit ==>-----

Waiting 15s for rate limit =====>-----

Waiting 15s for rate limit =====>-----

Waiting 15s for rate limit =====>-----

Waiting 15s for rate limit =====>----

Waiting 15s for rate limit =====>

```
head(df_discursos_senado)
```

A tibble: 6 x 16

	id	codigo_parlamentar	nome	partido	uf	tipo_sessao	tipo_uso_palavra
	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>
1	513287	5976	Eduardo ~	NOVO	CE	Sessão Del~	Discurso
2	513284	1173	Wellingt~	PL	MT	Sessão Del~	Pela Liderança
3	513283	6304	Margaret~	PSD	MT	Sessão Del~	Discurso
4	513280	4560	Sérgio P~	PSD	AC	Sessão Del~	Discurso
5	513279	4531	Jayme Ca~	UNIÃO	MT	Sessão Del~	Pela Liderança
6	513278	5906	Dra. Eud~	PL	AL	Sessão Del~	Discurso

i 9 more variables: data <chr>, sumario <chr>, keywords <chr>,
transcricao <list>, nome_civil <chr>, sexo <chr>, data_nascimento <chr>,
uf_nascimento <chr>, municipio_nascimento <chr>

```
write_csv(df_discursos_senado, "discursos_senado_abril_2025.csv")
```